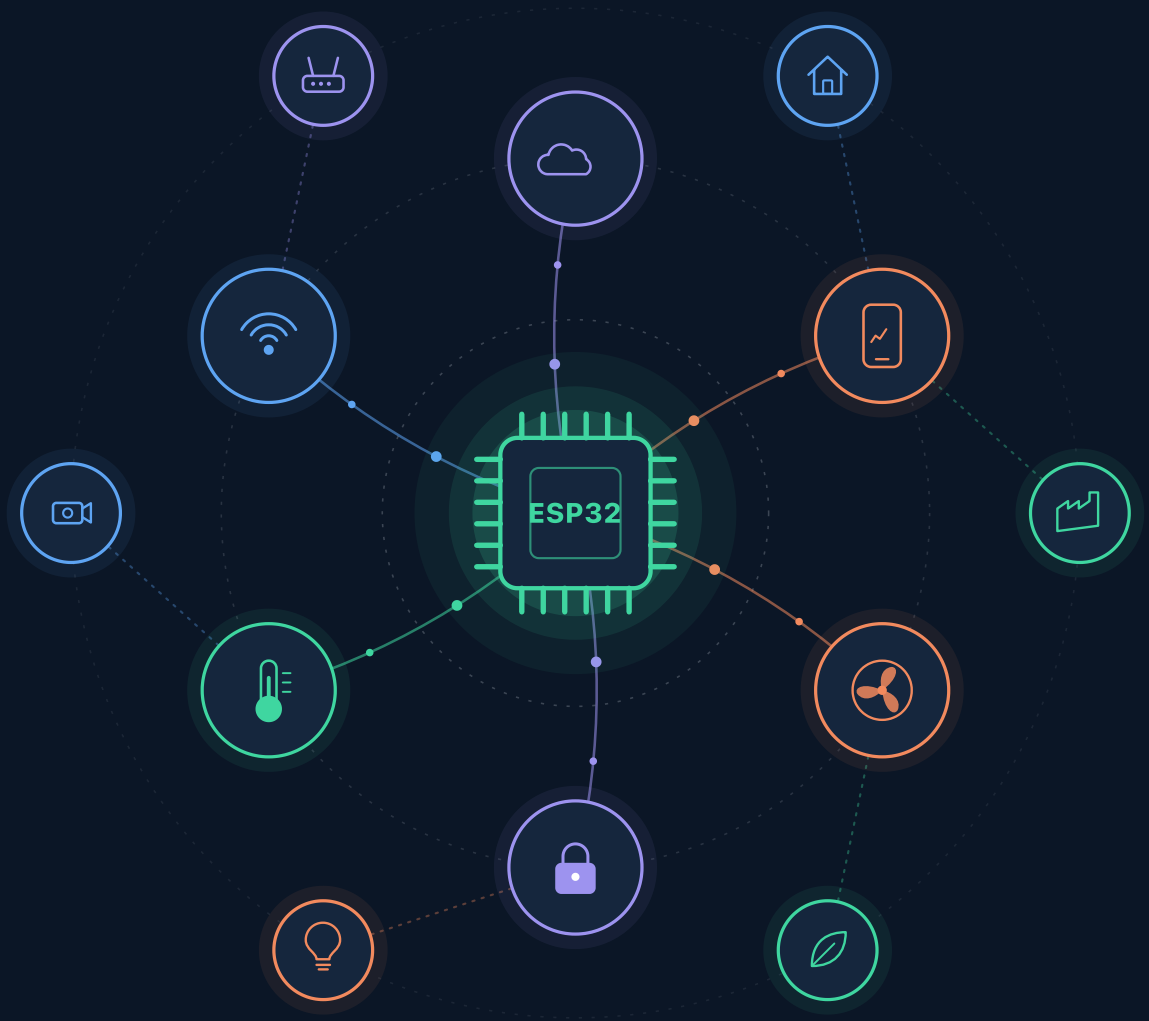
A PRACTICAL GUIDE

Introduction to the Internet of Things

From “what is IoT?” to a working smart device. Learn how connected devices sense, communicate, decide and act — then build one yourself, right in your web browser.



Mehdi Lotfinejad

Introduction to the Internet of Things

A practical, hands-on guide — from “what is IoT?” to a working smart device

Mehdi Lotfinejad

Introduction to the Internet of Things
A practical, hands-on guide — from "what is IoT?" to a working smart device

Mehdi Lotfinejad

First edition, September 2026

Copyright © 2026 Mehdi Lotfinejad.

Hands-on labs, code and circuit files: github.com/datatweets/intro-to-iot-workshop

Diagrams were created for this book. Photographs are reproduced from Wikimedia Commons under free licences (CC0, CC BY and CC BY-SA); authors and licences are listed in *Image credits* at the back of the book.

Product and company names mentioned in this book are trademarks of their respective owners. They are used only to identify products and services; no endorsement is implied.

Every effort has been made to ensure that the information in this book is accurate as of September 2026. Technology, prices, free-plan limits, standards and laws change over time — always check the latest official sources before relying on them. The information is provided without warranty, and the author accepts no liability for any loss or damage arising from its use.

Safety: the labs in this book run in a simulator and are completely safe. Never work on mains electricity unless you are qualified to do so.

Contents

Preface iv

Chapter 1 What Is the Internet of Things? 1

Chapter 2 How an IoT System Is Built 17

Chapter 3 Sensors, Actuators, and Reading Data 31

Chapter 4 Getting Online: Connectivity and Communication Protocols 52

Chapter 5 IoT Cloud Platforms and Data Visualisation 73

Chapter 6 Data Processing and Automation 87

Chapter 7 IoT Security and Best Practices 100

Chapter 8 Real-World IoT Applications and Project Showcase 116

The hands-on labs 134

Glossary 135

Image credits 146

Preface

You probably used a dozen connected devices before lunch today without thinking about it: the phone that woke you, the card you tapped to pay, the traffic lights that changed as you arrived, the electricity meter that reported itself. Together they make up the **Internet of Things** — ordinary objects that sense the world, share what they sense, and act on it.

This book explains how those things work, and then shows you how to build one yourself.

Who this book is for

This book is for anyone who is curious about the Internet of Things (IoT) and wants to understand it properly — not just the buzzwords.

- **Complete beginners.** You don't need to know electronics, networking, or programming. Every new idea is explained simply the first time it appears, usually with an everyday comparison.
- **Students and career-changers** who want a clear map of the whole field before specialising in one part of it.
- **Engineers and professionals from other fields** who need to talk confidently with IoT teams, suppliers, or customers.
- **Teachers and trainers** looking for a ready-made, tested path through the subject.

What you'll build

The book follows one running example from start to finish: a **smart room monitor**. It measures the temperature and humidity of a room, shows live readings on a small screen, switches on a fan by itself when the room gets too hot, and sends its data across the internet so you can watch it from anywhere.

You'll build it one piece at a time, in six short hands-on labs, and finish with the complete **Smart Environment Station** in Chapter 8.

What you need

- **A computer with a web browser** (Chrome, Edge, or Firefox) and an internet connection.
- **Two free online accounts**, which take a couple of minutes each: Wokwi, a simulator that runs a real ESP32 microcontroller in your browser, and ThingSpeak, a free cloud dashboard.

- **That's all.** No hardware to buy, no software to install, and no code to write from a blank page. Every lab provides complete, tested code; you'll read it, run it, and change small parts of it.

If you later want to build with real parts, the same code runs on a real ESP32 board wired the same way — for the Wi-Fi labs, you'd just put your own network name and password in place of the simulator's. Chapter 8 suggests where to go next.

The hands-on labs

The six labs, with step-by-step instructions, screenshots, and all the code and circuit files, are free online:

github.com/datatweets/intro-to-iot-workshop

Each chapter ends by pointing you to its lab. Every lab has been compiled and run end to end in the simulator before being written up, so if something doesn't work, the troubleshooting notes in the lab will usually tell you why.

How the book is organised

Chapter	What it covers	Hands-on
1. What Is the Internet of Things?	What IoT is, where it came from, where it's used	—
2. How an IoT System Is Built	The five layers, microcontrollers, the data pipeline	—
3. Sensors, Actuators, and Reading Data	Sensors, actuators, analog and digital signals	Labs 1 and 2
4. Getting Online	Wi-Fi, Bluetooth, LoRaWAN, cellular; HTTP and MQTT	Lab 3
5. IoT Cloud Platforms and Data Visualisation	Cloud platforms, analytics, dashboards	Lab 4
6. Data Processing and Automation	Events, rules, edge vs cloud decisions	Lab 5
7. IoT Security and Best Practices	Threats, defences, standards and laws	Group activity
8. Real-World IoT Applications	Designing complete systems, prototyping, deployment	Lab 6 — the capstone

Read the chapters in order the first time — each one builds on the last. At the back of the book you'll find a **glossary** of every key term and the **image credits**.

How to read each chapter

Every chapter follows the same pattern, so you always know where you are:

- It opens with **what you'll learn** and a quick reminder of **where we are** in the journey.
- **The big picture** gives you one diagram and a short story before the details.
- The main sections introduce a few ideas at a time, each with an everyday comparison, a diagram, and an example.
- **Pause and check** questions appear along the way. Try to answer each one before you read on — even a wrong guess helps you remember.
- **Common misconceptions** clears up the ideas that trip most people up.
- **Key takeaways**, **Check your understanding**, and **Key terms** close the chapter. The answers to every question are in the **answer key** at the end of the chapter.

Conventions used in this book

Diagrams use the same colours throughout, so you can read them at a glance:

Colour	Always means
Blue	Data moving from devices towards people
Orange	Commands and actions, such as switching a fan on
Aqua	Work done on or near the device (the "edge")
Violet	The cloud
Green ✓ / red ✕	OK / problem — always with an icon and words, never colour alone

Charts (such as the connectivity map in Chapter 4) have their own legend.

You'll also see boxed notes:

> **NOTE**
>
> Useful background or a side detail.

> **TIP**
>
> A "pause and check" question, or a helpful hint.

> **IMPORTANT**
>
> Something worth remembering.

> **WARNING**
>
> A safety or security caution.

A word on safety: everything in this book's labs is simulated, so it is completely safe. If you go on to build real projects, never work on mains electricity (the power from a wall socket) unless you are qualified to do so.

Enjoy the journey — by the last page, you'll have built a working IoT system from end to end.

Mehdi Lotfinejad

CHAPTER 1

What Is the Internet of Things?

You probably used several IoT devices today without noticing. This chapter explains what the Internet of Things is, where it came from, what makes something an "IoT device", and where it's being used around the world — and around you.

At a glance	
Reading time	About 20 minutes
Before this	Nothing. This is where the book begins.
You'll need	Nothing. Just think about the devices you've used since you woke up.

What you'll learn

By the end of this chapter, you'll be able to:

1. **Explain**, in your own words, what the Internet of Things is.
2. **Describe** how the internet grew from connecting computers to connecting everyday objects.
3. **List** the key characteristics that make a device an "IoT device".
4. **Describe** why IoT is growing so fast, and the opportunities that creates.
5. **Give examples** of IoT in six application areas: homes, cities, industry, healthcare, agriculture, and transport.

Welcome — and a quick look ahead

Welcome! Over the next eight chapters, you'll go from "what is IoT?" to building a working smart device that measures a room, shows live readings on a small screen, reacts on its own when it gets too hot, and sends its data across the internet.

You don't need any hardware, and you don't need to be a programmer. Every hands-on lab runs in a free online simulator called Wokwi, and all the code is provided for you. You'll read it, run it, and change small parts of it.

Here's the road ahead:

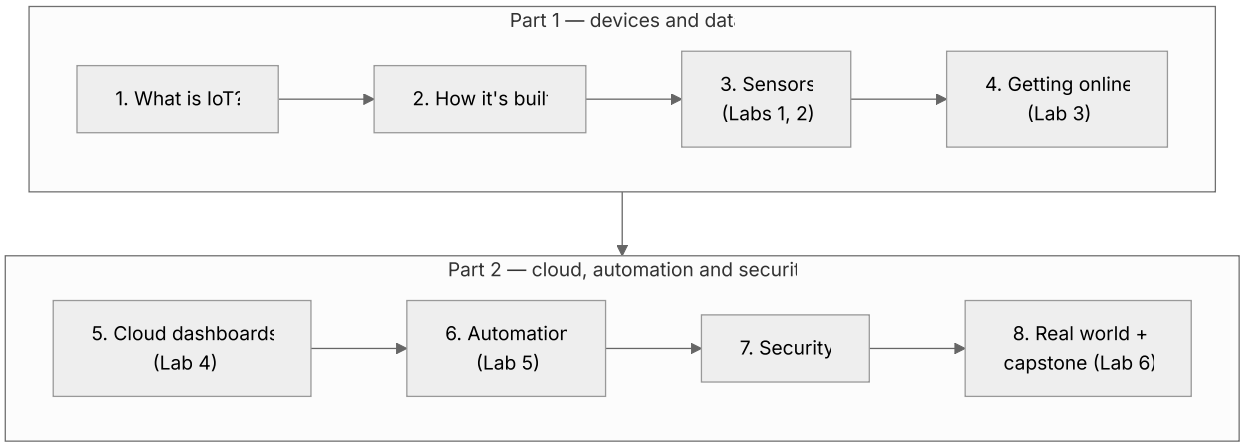


Figure 1.1 — the eight chapters in order. Chapters 1 to 4 cover devices and data; Chapters 5 to 8 cover the cloud, automation and security. Hands-on labs come with Chapters 3 to 6 and Chapter 8. (In a one-day workshop, Part 1 is the morning and Part 2 the afternoon.)

This first chapter is all about the big picture. There's no coding yet — just ideas you'll build on in every chapter.

A morning full of IoT

Let's start with an ordinary morning in Kuala Lumpur, or any big city.

Your phone alarm wakes you up. A fitness band on your wrist has already measured how well you slept. You pay for breakfast by tapping your phone. On the way to work, the traffic lights adjust their timing to the flow of cars, and a sign at the train station tells you exactly when the next train will arrive. At home, the electricity meter on the wall sends its reading to the power company by itself — nobody needs to come and read it.

Every one of those things involves a physical object that **measures something**, **sends that information over a network**, and lets **a person or a computer act on it**. That's the Internet of Things.

What is the Internet of Things?

Here's a simple definition you can use:

> **The Internet of Things (IoT)** is the network of physical objects — "things" — that have sensors, software, and a network connection, so they can collect data about the world, share it, and act on it, often without a person having to do anything.

A "thing" can be almost anything: a light bulb, a fridge, a water pump, a tractor, a streetlight, a heart monitor, a shipping container, or a cow wearing a smart collar.

The IoT loop: sense, send, decide, act

The easiest way to understand any IoT device is to look for four steps. We'll use this loop throughout:

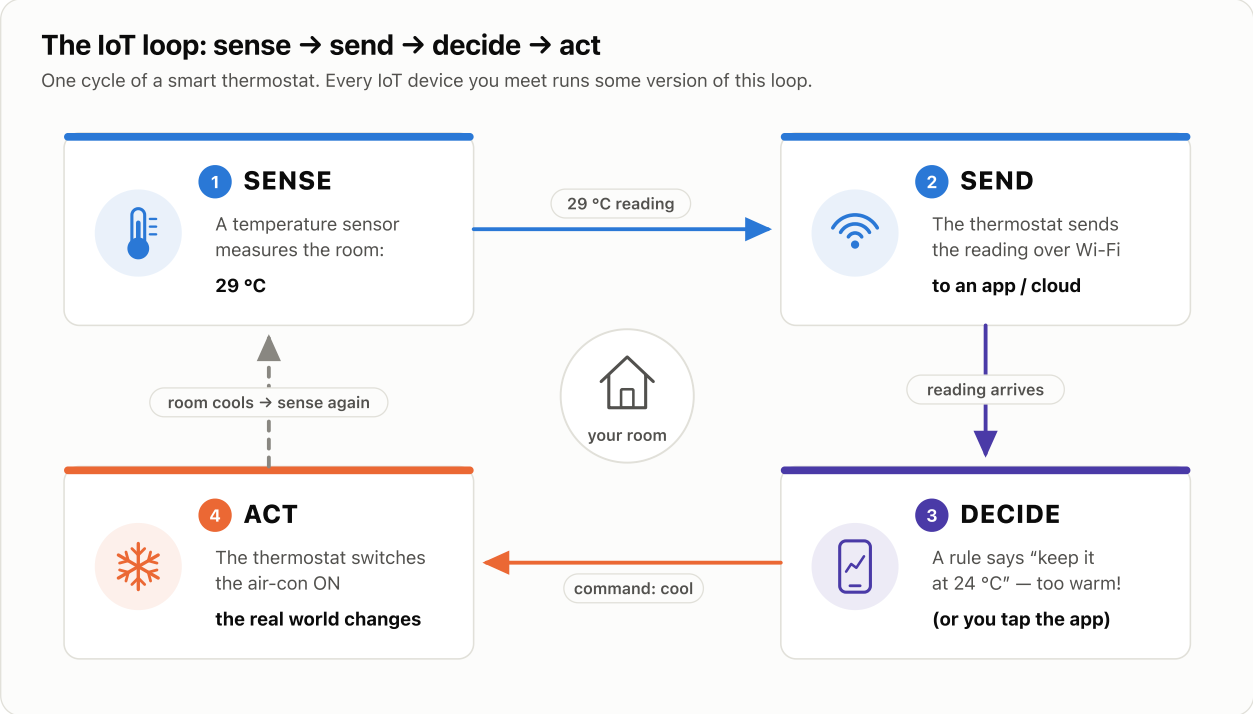
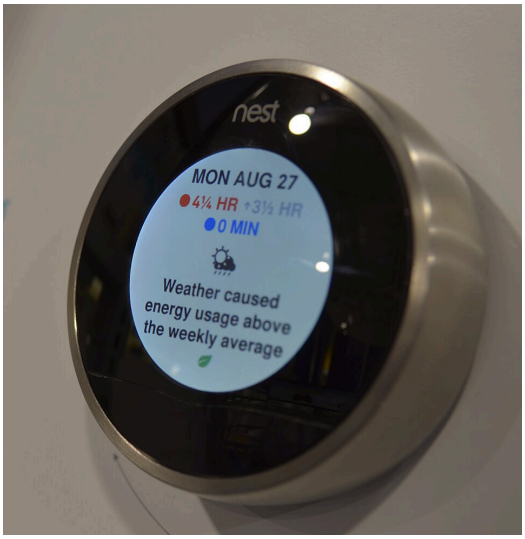


Figure 1.2 — the IoT loop. Every IoT device follows these four steps, then starts again, because acting changes the world.

Let's walk through it with a **smart thermostat**, the device that controls a home's heating or air-conditioning:



A real smart thermostat. It senses the room, connects to Wi-Fi, and switches heating or cooling on and off. Photo: Raysonho @ Open Grid Scheduler / Grid Engine, CC0, via Wikimedia Commons.

1. **Sense.** A temperature sensor inside measures the room: 29 °C.
2. **Send.** The thermostat sends that reading over Wi-Fi to an app.
3. **Decide.** A rule says “keep the room at 24 °C”, so the system decides the air-conditioning should come on. (Or you decide yourself, by tapping the app.)
4. **Act.** The thermostat switches the air-conditioning on.

Then the loop starts again. As the room cools, the thermostat senses the change, and eventually switches the air-conditioning off.

> **TIP**
>
> **Pause and check.** A smart doorbell has a camera and a button. When someone presses the button, you get a video call on your phone and can unlock the door remotely. Can you find the four steps?
>
> *→ Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

How we got here: from the internet of computers to the Internet of Things

IoT didn’t appear overnight. It grew out of the internet itself, step by step.

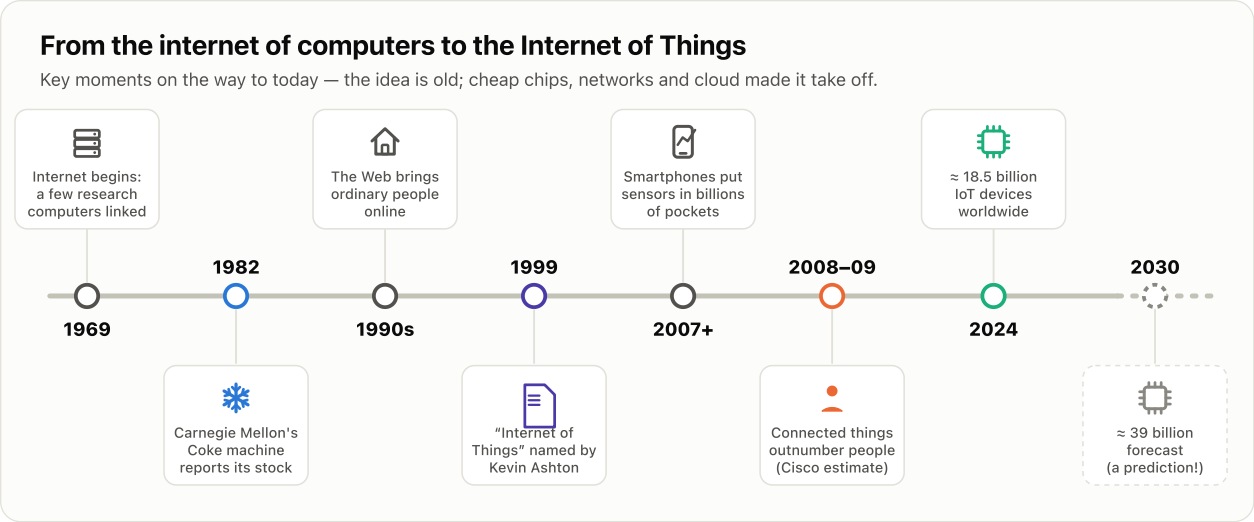


Figure 1.3 — from a handful of research computers to tens of billions of connected things.

The internet of computers (1960s–1990s)

The internet began in the late 1960s as a way to connect a small number of big research computers. For decades, the things on the internet were **computers**, and the people using them were mostly scientists, engineers, and universities. In the 1990s, the World Wide Web arrived, and ordinary people started getting online from home.

A curious early experiment (1982)

One of the first "things" ever connected to a network was a drinks machine. In 1982, students at Carnegie Mellon University in the United States were tired of walking to a Coke machine only to find it empty, or full of warm bottles. So they wired switches inside it to the university's computer network. Anyone on the network could check, from their desk, whether there were cold drinks before making the trip.

That's the IoT loop in miniature: **sense** (switches detect bottles), **send** (over the network), **decide** (is it worth the walk?), **act** (go and get a drink).

A name is born (1999)

The phrase **"Internet of Things"** was coined in 1999 by Kevin Ashton, who was working at the consumer-goods company Procter & Gamble. He used it as the title of a presentation about putting small radio tags (called RFID tags) on products, so that computers could track them through the supply chain by themselves, instead of relying on people typing in data. Ashton went on to co-found the Auto-ID Center at the Massachusetts Institute of Technology (MIT), which developed much of that tracking technology.

His big idea was simple: computers depended on humans to feed them information about the physical world. If things could report on themselves, computers could "see" the world directly.



Kevin Ashton, who named the “Internet of Things” in 1999. Photo: Larry D. Moore, CC BY 4.0, via Wikimedia Commons.

The internet of everyone (2000s)

In the 2000s, smartphones put an internet-connected computer — packed with sensors — into billions of pockets. Wi-Fi became standard in homes and offices.

The networking company Cisco estimated that somewhere around **2008 to 2009**, the number of things connected to the internet overtook the number of people on Earth. Many people treat that moment as the “birthday” of the Internet of Things.

The Internet of Things (2010s–today)

Since then, connecting everyday objects has become normal. Today there are many more connected things than connected people — and most of them aren’t phones or computers at all.

What makes something an IoT device?

Not every electronic gadget is an IoT device. A basic digital thermometer that only shows a number on its own screen isn’t. A thermometer that sends its reading to your phone is.

Here are the characteristics that IoT devices and systems share:

Characteristic	What it means	Example
Sensing	It can measure something in the physical world.	A smart meter measures electricity use.
Connectivity	It can send and receive data over a network.	The meter sends readings to the power company.
Unique identity	Each device can be told apart from every other one.	Each meter has its own ID, so your bill is yours.
Intelligence and automation	It can process data and act, sometimes without a person.	A smart plug turns off a heater after an hour.
Ability to act	Many devices can change the physical world, not just measure it.	A smart valve shuts off a leaking water pipe.
Scale	IoT systems often involve huge numbers of devices.	A city with tens of thousands of streetlights.
Limited resources	Many devices are small and cheap, with little power, memory, or processing.	A soil sensor that must last years on one battery.
Variety	Devices come in many shapes and from many makers, and must still work together.	Lights, locks, and cameras from different brands in one home.

If you remember only two, remember these: an IoT device **senses (or acts on) the physical world** and it is **connected**. The rest follows from those.

> **NOTE**
>
> "IoT" doesn't always mean "on the public internet". Many IoT systems — in factories, for example — run on private networks. What matters is that the things are **networked**, so data can flow between them and to the systems that use it.

Why is IoT growing so fast?

Ideas like the Coke machine have existed since the 1980s. So why has IoT exploded only in the last 15 years? Mostly because four things became very cheap and very good at the same time:

1. **Sensors got cheap and small.** Smartphones created a massive market for tiny sensors — accelerometers, cameras, temperature sensors — which drove their prices down for everyone.
2. **Chips got cheap and capable.** A microcontroller with Wi-Fi and Bluetooth built in, like the ESP32 you'll use in the labs, now costs very little.

3. **Connectivity got everywhere.** Wi-Fi is in nearly every home and office, mobile networks cover most populated areas, and new low-power networks can reach sensors in remote fields.
4. **The cloud made storage and computing cheap.** Instead of buying and running their own servers, companies can rent as much computing power as they need, by the minute.

How big is it?

The research firm IoT Analytics counted about **18.5 billion** connected IoT devices in the world in 2024. It estimated about **21.1 billion** by the end of 2025, and forecasts around **39 billion by 2030**. That's several connected things for every person on the planet. Of those devices, Wi-Fi was the most common way to connect, followed by Bluetooth and cellular networks.

Businesses are spending heavily, too. IoT Analytics estimated that companies spent about **US$324 billion** on IoT in 2025 — 13% more than the year before — and expects roughly 14% growth in 2026.

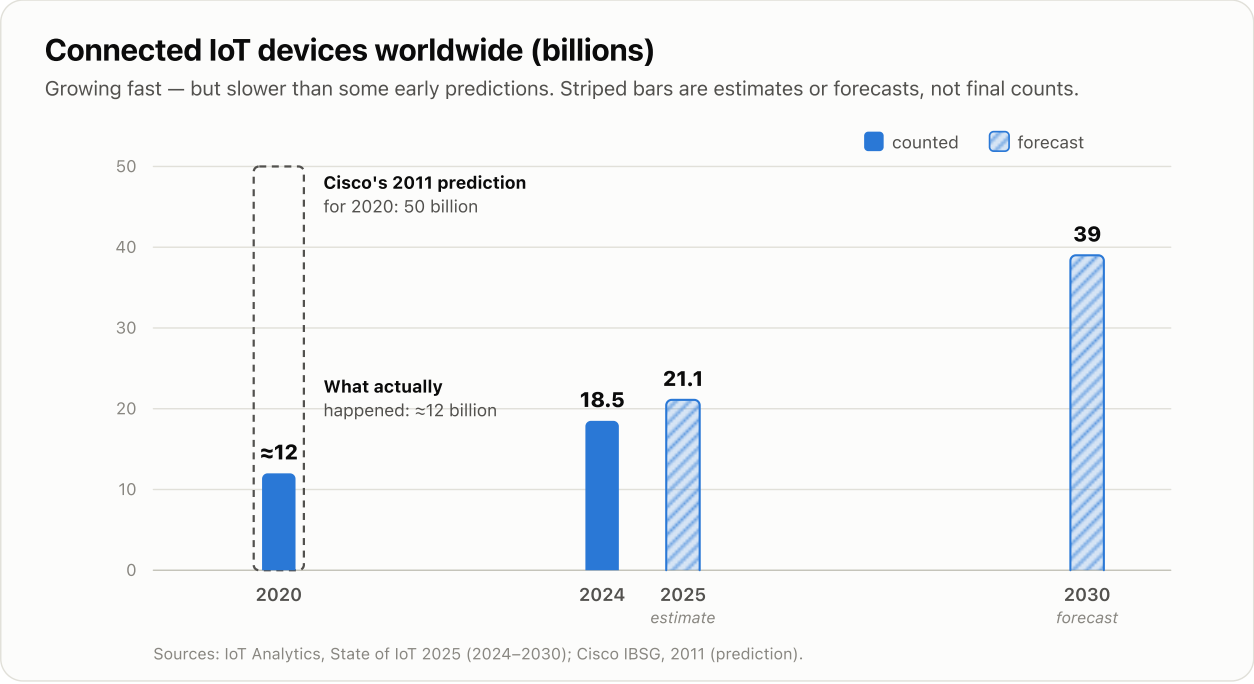


Figure 1.4 — IoT devices worldwide. The dashed outline shows how far above reality one famous early forecast was.

IMPORTANT

Treat forecasts with healthy doubt. In 2011, Cisco predicted there would be **50 billion** connected devices by 2020. The real number in 2020 was closer to 12 billion. IoT is growing fast, but predictions about technology are often too optimistic. When you read a big number, ask who made it, and how they counted.

Trends shaping IoT right now

Here are a few trends you'll hear about:

- **Smarter devices at the edge.** More devices now make decisions themselves, and some even run small artificial intelligence (AI) models on the device. This is sometimes called **AIoT** (AI + IoT). You'll learn about the "edge" in Chapters 2 and 6.
- **Devices that work together.** A standard called **Matter**, backed by Apple, Google, Amazon, Samsung and many others, lets smart-home devices from different brands work together.
- **New networks built for things.** Low-power, long-range networks like **LoRaWAN** and **NB-IoT** connect sensors that run for years on a battery. You'll compare these in Chapter 4.
- **Security is becoming the law.** Governments are passing rules that ban weak default passwords and require security updates. You'll learn about these in Chapter 7.
- **Sustainability.** IoT is widely used to save energy and water, and to cut waste — for example, turning streetlights down when streets are empty.

Opportunities — including for you

IoT creates opportunities at every layer. Businesses use it to cut costs, prevent breakdowns, and offer new services. And it creates jobs: embedded (firmware) developers, cloud and data engineers, network specialists, security experts, data analysts, product managers, and technicians who install and maintain devices.

You don't have to be an expert in all of it. Many people build whole careers around one piece — and this book gives you a map of all the pieces.

Where is IoT used? Six application areas

IoT shows up almost everywhere, but it's useful to group it into a few **application domains** — areas of use. Here are the six we'll look at.

1. Smart homes

IoT in the home is about **comfort, convenience, safety, and saving energy**.

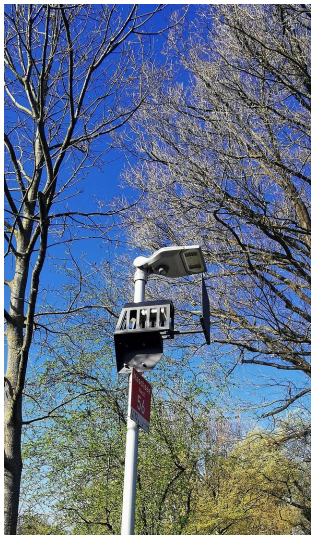
- Smart thermostats and air-conditioners that learn your schedule.
- Smart lights and plugs you can control from your phone or by voice.
- Video doorbells, cameras, smart locks, and alarm sensors.
- Robot vacuum cleaners, smart speakers, and connected appliances.

2. Smart cities

Cities use IoT to run **services more efficiently** and make life better for residents.

- **Smart streetlights** that report their own faults and dim when streets are empty. Los Angeles, for example, converted almost all of its streetlights to LEDs and connected tens of thousands of them to a monitoring system. The city reports saving around 64% of the energy the old lights used, and it now knows about broken lights straight away.
- **Smart parking** sensors that show free spaces on an app.
- Traffic sensors that adjust traffic-light timing.
- Sensors that monitor air quality, flooding, and rubbish-bin levels.

In Malaysia, Putrajaya runs its own low-power LoRaWAN network (you'll meet LoRaWAN in Chapter 4) for smart-city services such as smart parking.



A smart streetlight pilot in Hannover, Germany. The box on the pole adds sensors and a network connection. Photo: Ra Boe / Wikipedia, CC BY-SA 3.0 DE, via Wikimedia Commons.

3. Industrial IoT (IIoT)

Industrial IoT (often shortened to **IIoT**) is IoT in factories, power plants, oil and gas sites, and mines. The focus is on **efficiency, safety, and avoiding breakdowns**.

- **Predictive maintenance:** sensors listen for vibration or heat that shows a machine is about to fail, so it can be fixed *before* it breaks. The consultancy McKinsey reports that this typically cuts machine downtime by 30% to 50%.
- Tracking every part and product as it moves through a factory.
- Monitoring energy use across a whole site.

4. Healthcare

In healthcare, IoT helps **monitor patients and manage equipment**.

- Wearables that track heart rate, activity, and sleep.
- Remote patient monitoring — for example, a connected blood-pressure cuff or glucose monitor that sends readings to a doctor, so patients can stay at home.
- Tracking expensive hospital equipment, like wheelchairs and infusion pumps, so staff can find them fast.
- Temperature monitoring for vaccine and medicine fridges.

5. Agriculture

Smart agriculture (or **precision agriculture**) uses IoT to grow more food with **less water, fertiliser, and labour**.

- Soil-moisture sensors that tell farmers exactly when and where to water.
- Weather stations in the field.
- Automatic irrigation and fertigation (feeding plants through their water).
- Tracking the health and location of livestock.
- Monitoring greenhouses, and conditions in large plantations.

6. Transportation and logistics

Here, IoT is about **knowing where things are and how they're doing**.

- GPS trackers on lorries, ships, and shipping containers.
- Temperature monitoring for food and medicine being transported (the "cold chain").
- Fleet management: fuel use, driver behaviour, and vehicle health.
- Connected cars that report faults and receive software updates.
- Live arrival times for buses and trains.

A Malaysian example you probably know: smart electricity meters

Tenaga Nasional Berhad (TNB), Malaysia's main electricity company, has been replacing old meters with **smart meters**. A smart meter measures how much electricity a home uses and sends the readings to TNB automatically. That means no more estimated bills, faster detection of power cuts, and the option for customers to see their own usage in an app. TNB reports that more than **5.6 million customers** already have smart meters, with about 4.5 million more planned by 2027.

It's a perfect example of the IoT loop: the meter **senses** electricity use, **sends** it to TNB, TNB's systems **decide** (on a bill, or on where the power cut is), and crews or customers **act**.



A smart electricity meter (this one is in the United States). It measures use and reports it automatically. Photo: 277volts, CC BY-SA 4.0, via Wikimedia Commons.

Common misconceptions

"IoT just means controlling things from your phone." Phone control is only one small part. Most IoT data never reaches a person's phone at all. It's used by other computers — to run factories, balance the power grid, or schedule repairs automatically.

"IoT is only about gadgets for the home." Smart homes are the most visible part, but industry, cities, agriculture, healthcare, and logistics make up a huge share of IoT — and much of the money spent on it.

"An IoT device needs to be powerful." The opposite is often true. Many IoT devices are deliberately tiny, cheap, and low-power, so they can run for years on a battery in a remote place.

"IoT is a single technology." IoT is a combination of many technologies — sensors, microcontrollers, networks, cloud computing, data analysis, and security — working together. That's why it takes eight chapters to cover it properly, not one.

Activity: where do you encounter IoT in daily life?

Try this on your own, or in a small group if you're working through the book with others. Give yourself about five minutes.

1. List **every** connected device or system you used or passed since you woke up today. Think about home, transport, payments, work, and your health.
2. For each one, try to spot the **sense → send → decide → act** loop.
3. Sort your list into the six application areas above.

4. Pick one, and ask: *what data is it collecting about me, and who can see it?* (We'll come back to this question in Chapter 7, on security.)

Common answers include: fitness trackers and smartwatches, contactless card and phone payments, ride-hailing apps tracking the car, smart traffic lights, electricity and water meters, lift (elevator) monitoring, CCTV cameras, parking sensors, and supermarket self-checkouts.

Key takeaways

- The **Internet of Things (IoT)** is the network of physical objects with sensors, software, and connectivity, that collect and share data and act on it.
- Every IoT device follows the loop: **sense → send → decide → act**.
- IoT grew from the internet of computers. The name dates from **1999** (Kevin Ashton), and connected things outnumbered people around **2008–2009**.
- Key characteristics: **sensing, connectivity, unique identity, automation, acting, scale, limited resources, and variety**.
- IoT is booming because **sensors, chips, connectivity, and cloud computing** all became cheap. There were about **18.5 billion** IoT devices in 2024 — but always treat forecasts with care.
- Six major application areas: **homes, cities, industry, healthcare, agriculture, and transport**.

Check your understanding

1. In one sentence, what is the Internet of Things?

→ *Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

2. A digital kitchen scale shows the weight on its own screen. A "smart" kitchen scale sends the weight to a recipe app. Which one is an IoT device, and why?

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

3. Name the four steps of the IoT loop, and apply them to an automatic garden sprinkler.

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

4. Give two reasons why IoT has grown so quickly in the last 15 years.

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

5. In 2011, a big company predicted 50 billion IoT devices by 2020. The real number was about 12 billion. What lesson does that teach?

→ Answer A6 is in the answer key at the end of this chapter. Try it yourself first!

6. Match each example to an application area: (a) a sensor that detects a factory motor overheating; (b) a GPS tracker on a lorry; (c) a smart parking sensor; (d) a connected glucose monitor.

→ Answer A7 is in the answer key at the end of this chapter. Try it yourself first!

Key terms

Term	Meaning
Application domain	An area where IoT is used, such as homes, cities, or agriculture.
Cloud computing	Renting computing power and storage over the internet instead of owning it.
Connectivity	A device's ability to send and receive data over a network.
Industrial IoT (IIoT)	IoT used in factories, energy, mining, and other industries.
Internet of Things (IoT)	The network of physical objects with sensors, software, and connectivity that collect, share, and act on data.
IoT loop	The four steps every IoT system follows: sense, send, decide, act.
Matter	A smart-home standard that lets devices from different brands work together.
Precision agriculture	Using sensors and data to give crops exactly the water and nutrients they need.
Predictive maintenance	Using sensor data to fix machines before they break down.
RFID	Radio-frequency identification — small tags that let objects be identified by radio.
Smart meter	An electricity (or water) meter that measures use and reports it automatically.

Up next

In Chapter 2 — How an IoT System Is Built, you'll open up the "black box". You'll learn the five layers every IoT system is built from, meet sensors, actuators, and microcontrollers properly, and follow a single reading on its journey from a device to a dashboard.

Going further (optional)

- Kevin Ashton — "That 'Internet of Things' Thing" — a short, readable article by the person who named IoT, explaining what he meant.
- IoT Analytics — Number of connected IoT devices — the latest counts and forecasts, updated regularly.

- Los Angeles Bureau of Street Lighting — connected infrastructure — a real smart-city project, with its results.

Answer key

A1 · Pause and check — A smart doorbell has a camera and a button. When someone presses the button, you get a video call on your phone and can unlock the door remotely. Can you find the four steps?

Sense: the button detects a press, and the camera captures video. **Send:** the doorbell sends the alert and video over the internet to your phone. **Decide:** you look at the video and decide whether to let the person in. **Act:** you tap “unlock”, and the smart lock opens.

A2 · Question 1 — In one sentence, what is the Internet of Things?

Something like: *the network of everyday physical objects that have sensors, software, and a network connection, so they can collect and share data and act on it — often without a person getting involved.*

A3 · Question 2 — A digital kitchen scale shows the weight on its own screen. A "smart" kitchen scale sends the weight to a recipe app. Which one is an IoT device, and why?

The **smart scale** is an IoT device, because it’s **connected** — it shares its data over a network so another system can use it. The basic scale senses the weight, but keeps the data to itself.

A4 · Question 3 — Name the four steps of the IoT loop, and apply them to an automatic garden sprinkler.

Sense: a soil-moisture sensor measures that the soil is dry. **Send:** it sends the reading to a controller or app. **Decide:** a rule says “water if moisture is below 30%”. **Act:** the sprinkler valve opens.

A5 · Question 4 — Give two reasons why IoT has grown so quickly in the last 15 years.

Any two of: sensors became cheap and small; microcontrollers with built-in wireless became cheap; Wi-Fi and mobile networks became available almost everywhere; and cloud computing made storing and processing data cheap and easy.

A6 · Question 5 — In 2011, a big company predicted 50 billion IoT devices by 2020. The real number was about 12 billion. What lesson does that teach?

Technology forecasts are often **too optimistic**. It's wise to ask who made a prediction, how they counted, and whether they had a reason to make the number look big.

A7 · Question 6 — Match each example to an application area: (a) a sensor that detects a factory motor overheating; (b) a GPS tracker on a lorry; (c) a smart parking sensor; (d) a connected glucose monitor.

(a) **Industrial IoT**; (b) **transportation and logistics**; (c) **smart cities**; (d) **healthcare**.

CHAPTER 2

How an IoT System Is Built

Every IoT system, from a smart doorbell to a factory full of sensors, is built from the same few layers. Once you can see those layers, you can make sense of any IoT product you meet.

At a glance	
Reading time	About 20 minutes
Before this	Chapter 1 — What Is the Internet of Things?
You'll need	Nothing but curiosity. A pen and paper help for sketching diagrams.

What you'll learn

By the end of this chapter, you'll be able to:

1. **Name** the five layers of an IoT system and say what each one does.
2. **Explain** the difference between a sensor and an actuator, with an example of each.
3. **Compare** a microcontroller with a microprocessor, and pick the right one for a simple project.
4. **Trace** one piece of data as it travels from a sensor to a person's screen.
5. **Identify** the main groups of companies and people who make an IoT product happen.

Where we are

In Chapter 1, you met the simple loop at the heart of every IoT device: it **senses** something, **sends** that information somewhere, something **decides** what to do about it, and then the system **acts**.

That loop tells you *what* an IoT system does. This chapter is about *how* it's put together. Think of Chapter 1 as learning what a car is for, and this chapter as lifting the bonnet to see the engine, the wheels, and the steering.

Everything you build in the labs fits into the diagram you're about to learn. Keep it in mind — we'll come back to it in almost every chapter.

The big picture

Let's start with one example that we'll use for the rest of the book. Picture a **smart room monitor**: a small box on a shelf that measures the room's temperature and humidity. When it gets too hot, it switches a fan on by itself. It also sends its readings to the internet, so you can check your room from your phone while you're out.

Here's how that one small box breaks down into layers:

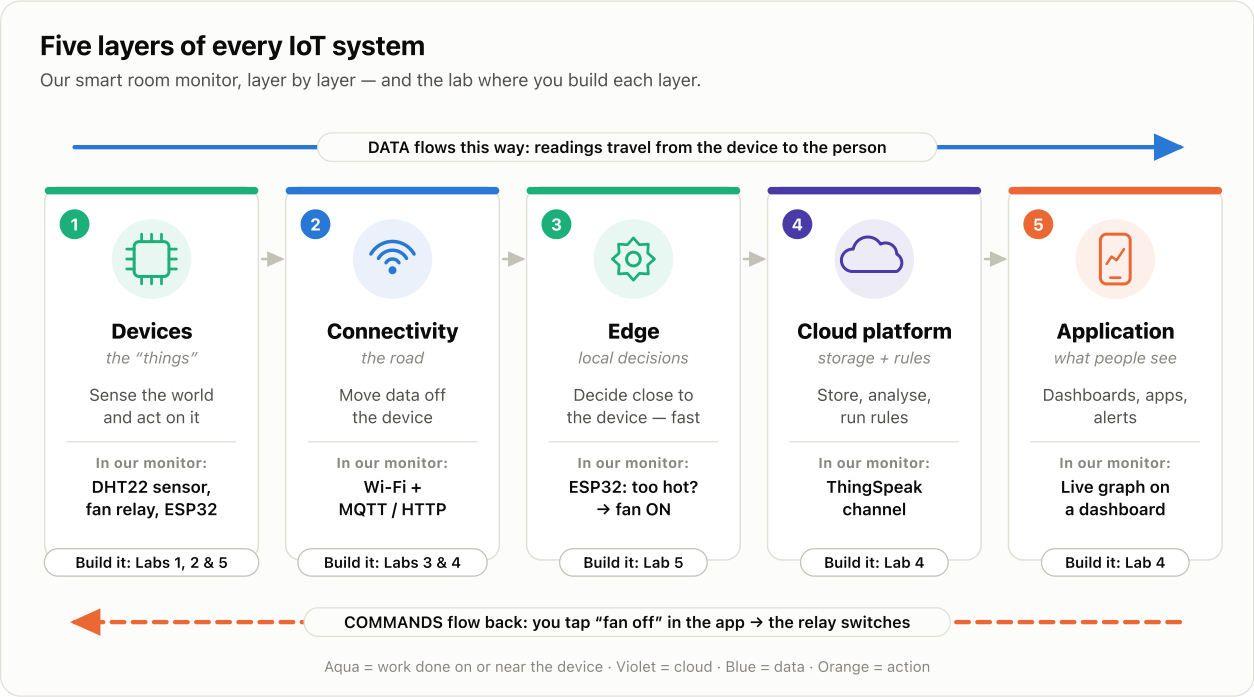


Figure 2.1 — the five layers of our smart room monitor. Data flows towards the person; commands flow back to the device.

Notice the two arrows. Data doesn't only flow *up* from the device. Commands can flow back *down* too — when you tap "fan off" in the app, that instruction travels back through the same layers to the box on the shelf.

You'll build every one of these layers yourself in the labs, in a simulator — the bottom of each card in Figure 2.1 shows which lab.

The five layers, one at a time

Different textbooks draw IoT architecture with three, four, five, or even seven layers. Don't let that worry you. They're all describing the same thing — they just slice it more or less finely. We'll use five layers, because they match what you'll build.

Layer 1: Devices (the "things")

This is the physical part — the hardware you can hold in your hand. A device usually has three ingredients:

- **Sensors** that measure something in the real world, like temperature.
- **Actuators** that do something in the real world, like switch on a fan.
- **A small computer** (usually a microcontroller) that reads the sensors, controls the actuators, and runs the device's program.

In our smart room monitor, the device is the box on the shelf. In the labs, your device is a simulated **ESP32** board — more on that chip later in this chapter.

Layer 2: Connectivity

A device on its own is just a gadget. It becomes part of the Internet of Things when it can **get its data out** and **receive instructions in**. Connectivity is the layer that does this.

It includes the radio technology (like Wi-Fi or Bluetooth) and the "language" the device speaks once connected (like MQTT or HTTP). Chapter 4 is entirely about this layer, so for now just remember: *connectivity is the road the data travels on.*

Sometimes the device connects straight to the internet, like our room monitor on home Wi-Fi. Other times it connects to a **gateway** first. A gateway is a middleman box that collects data from many nearby devices and forwards it to the internet. Your home Wi-Fi router is a kind of gateway. So is a smart-home hub that talks to lots of battery-powered light switches.

Layer 3: Edge computing

Edge computing means doing some of the thinking close to where the data is collected — on the device itself, or on a gateway nearby — instead of sending everything far away to the cloud first.

Why bother? Imagine our room monitor had to ask a server in another country "is 31 °C too hot?" every time it took a reading. If the internet dropped, the fan would never switch on. Making the decision locally, *at the edge*, is faster and keeps working when the connection

doesn't. The edge can live on the device itself or on a nearby gateway, which is why it sits between the device and the cloud.

> **NOTE**
>
> "Edge" simply means "the edge of the network" — the end where the devices are, far from the big data centres in the middle. You'll build a real edge decision in Lab 5, and Chapter 6 covers the idea in depth.

Layer 4: Cloud platform

The **cloud** is a set of powerful computers in data centres that you rent over the internet, instead of owning yourself. An **IoT cloud platform** is a service running there that's built specifically for device data.

A cloud platform typically:

- **Stores** readings over time, so you can see history, not just "now".
- **Processes** data — for example, working out a daily average.
- **Runs rules** — "if temperature is above 28 °C, send me a message".
- **Manages devices** — keeps track of which devices exist, and whether they're allowed to connect.

In Lab 4, you'll use a free cloud platform called ThingSpeak. Chapter 5 covers cloud platforms in detail.

Layer 5: Application

The application is the part people actually see and use. It might be a dashboard with graphs, a phone app with buttons, an email alert, or a report for a manager. It can also be another computer system, like a billing system that reads smart electricity meters.

This is where all the effort pays off. A farmer doesn't care about MQTT topics or data centres — they care that their phone says "Field 3 needs water".

> **TIP**
>
> **Pause and check.** Our room monitor switches the fan on by itself when it gets hot, *without* asking the cloud. Which layer is making that decision?
>
> *→ Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

Sensors and actuators: the device's senses and muscles

Your body has senses (eyes, skin, ears) that take in information, and muscles that act on the world. An IoT device works the same way.

A **sensor** turns something in the physical world — heat, light, movement, a gas — into an electrical signal that a computer can read. It's an *input*: information flows *into* the device.

An **actuator** does the opposite. It takes an electrical signal from the computer and turns it into a physical action — light, sound, movement, or switching something on. It's an *output*: action flows *out of* the device.

	Sensor (input)	Actuator (output)
Job	Measures the world	Changes the world
Body comparison	Your senses	Your muscles
Examples	Thermometer, light sensor, motion detector, gas sensor	LED, buzzer, motor, relay (an electric switch)
In our room monitor	Temperature and humidity sensor	Relay that switches the fan

A quick way to tell them apart: ask *"which way is the information going?"* If it's coming **in from the world**, it's a sensor. If it's going **out into the world**, it's an actuator.

You'll meet many kinds of sensors and actuators in Chapter 3, and wire up real (simulated) ones in Labs 1 and 2.

The brain: microcontroller vs microprocessor

Every IoT device needs a small computer to run it. There are two broad families, and choosing between them is one of the first decisions an IoT designer makes.

What's a microcontroller?

A **microcontroller** (often shortened to **MCU**, for microcontroller unit) is a complete, tiny computer on a single chip. The processor, memory, and the pins that connect to sensors are all packed into one small, cheap package.

It usually runs **one program, forever**. You don't install apps on it or log into it. You turn it on, and it immediately starts doing its one job — reading a sensor every two seconds, for example — until the power goes off.

What's a microprocessor?

A **microprocessor** (often shortened to **MPU**, for microprocessor unit) is a more powerful processor that needs extra chips around it — separate memory, storage, and so on — to work. It's the kind of chip inside a laptop or a phone.

It usually runs a full **operating system**, like Linux, Windows, or Android. That means it can run many programs at once, show a desktop, and connect a camera or a screen easily. A **Raspberry Pi** is a popular small board built around a microprocessor.

Side by side

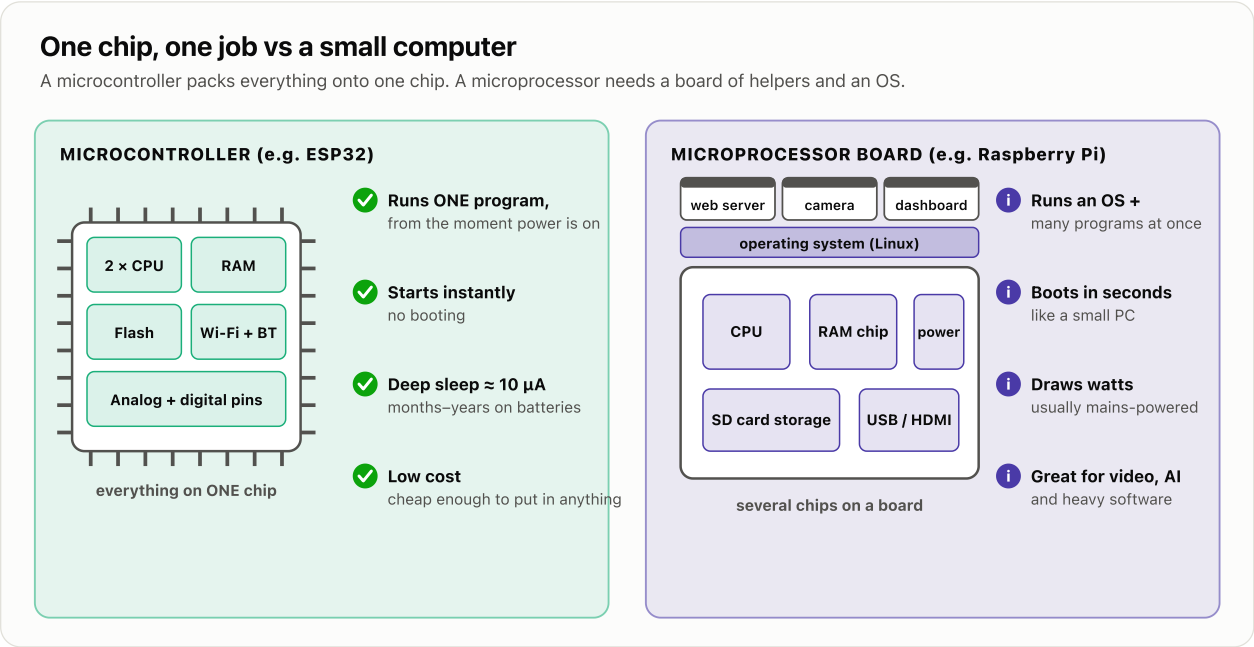
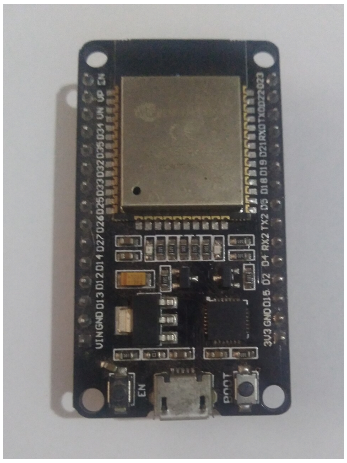
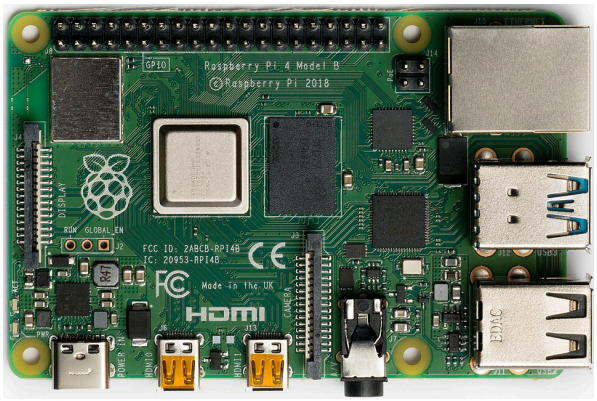


Figure 2.2 — a microcontroller puts everything on one chip; a microprocessor board spreads it across several chips.

	Microcontroller (MCU)	Microprocessor (MPU)
Everyday comparison	A kitchen timer — one job, done reliably	A laptop — many jobs, needs more looking after
Runs	One program, straight away at power-on	A full operating system and many programs
Power use	Very low — can run for months or years on batteries	Higher — usually needs mains power or a large battery
Start-up time	Almost instant	Seconds to a minute (it has to boot)
Cost	Low — a few US dollars	Higher — tens of US dollars and up
Good at	Reading sensors, switching things, low-power jobs	Video, heavy data processing, running a local dashboard
Examples	ESP32, Arduino boards, STM32	Raspberry Pi, the chip in your phone



A microcontroller board: the ESP32 DevKit. Photo: Edwiyanto, CC BY-SA 4.0, via Wikimedia Commons.



A microprocessor board: the Raspberry Pi 4. Photo: Laserlicht, CC BY-SA 4.0, via Wikimedia Commons.

Here's a simple rule of thumb: **if the device mostly reads sensors and flips switches, use a microcontroller. If it needs to process video or run lots of software, use a microprocessor.** Many real systems use both — lots of cheap microcontroller sensors, sending data to one microprocessor-based gateway.

Meet the device you'll use: the ESP32

All the labs use the **ESP32**, a microcontroller made by a company called Espressif. It's one of the most popular chips in the world for IoT, for hobbyists and professional products alike. Here's why:

- **Wi-Fi and Bluetooth are built in.** With many other microcontrollers, you'd have to add a separate radio chip. The ESP32 has both on the same chip.
- **It's cheap.** A development board costs very little, and the bare chip even less.
- **It's capable.** It has two processor cores running at up to 240 MHz, and 520 KB of working memory. That's tiny next to a laptop, but plenty for reading sensors, connecting to Wi-Fi, and sending data.
- **It sips power when idle.** In its deepest sleep mode the chip itself uses only about 10 microamps (millionths of an amp) — a whole development board, with its extra parts, draws more — so a battery-powered ESP32 can wake up, take a reading, send it, and go back to sleep for a long time.
- **It has lots of connection pins** for sensors and actuators, including pins that can read analog values (you'll see what that means in Chapter 3).

> **NOTE**
>
> The code you run in the Wokwi simulator would also run on a real ESP32 board with the same parts wired the same way. For the Wi-Fi labs, you'd only change the network name and password (`Wokwi-GUEST` exists only inside the simulator). The simulator is very close to the real chip, but real parts add electrical noise and limits it doesn't fully model.

Following one reading: the IoT data pipeline

Let's make the layers concrete by following a single temperature reading all the way through our smart room monitor. This journey is often called the **data pipeline** — a series of steps that data flows through, each one doing a little work on it.

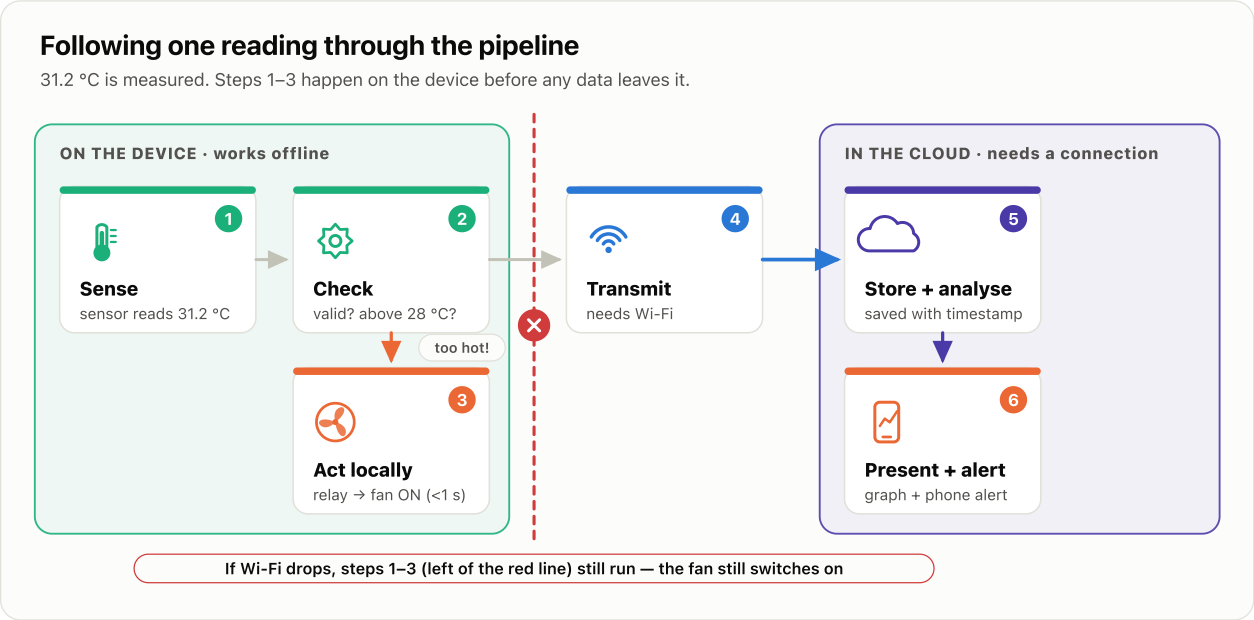


Figure 2.3 — one reading's journey. Steps 1–3, left of the red line, keep working if the Wi-Fi drops.

Step by step:

1. **Sense.** The temperature sensor measures the air. Inside, it produces an electrical signal. The microcontroller reads that and turns it into a number: 31.2 °C.
2. **Process locally.** The microcontroller checks the number. Is it a sensible value, or a glitch? Is it above our limit of 28 °C?
3. **Act locally.** Yes, it's too hot — so the microcontroller switches the relay, and the fan starts. This happens in a fraction of a second, with no internet needed.
4. **Transmit.** The microcontroller packages the reading into a small message and sends it over Wi-Fi to the cloud.
5. **Store and analyse.** The cloud platform saves the reading with a timestamp, next to thousands of earlier ones. It can now spot patterns: "this room gets hot every day at 3 pm".

6. **Present and alert.** Your phone dashboard updates its graph. If a rule says so, you also get a message: "Room is 31 °C — fan switched on."

Notice that steps 2 and 3 happen *before* the data ever leaves the device. That's edge computing at work. The cloud still gets the data (step 4 on), but it isn't needed for the urgent decision.

> **TIP**
>
> **Pause and check.** In the pipeline above, which step would still work if the Wi-Fi went down?
>
> *→ Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

Who makes it all happen: the IoT ecosystem

No single company builds an entire IoT system from scratch. Even a simple smart plug relies on a whole chain of companies and people. This network is called the **ecosystem**, and the people with an interest in the outcome are called **stakeholders**.

Here are the main groups, roughly following our five layers:

Group	What they do	Examples
Chip makers	Design the microcontrollers, sensors, and radio chips	Espressif (ESP32), Nordic Semiconductor, STMicroelectronics, Bosch Sensortec
Device makers	Build the finished product people buy	Smart-plug brands, industrial sensor makers, wearable brands
Connectivity providers	Provide the network the data travels on	Mobile network operators, LoRaWAN network operators, internet providers
Cloud and platform providers	Run the servers that store data and run rules	AWS, Microsoft, ThingSpeak (MathWorks), Blynk, Arduino
Application developers and integrators	Build the apps and connect everything into a working solution	Software companies, system integrators who install IoT for a factory or city
End users and businesses	Use the system and pay for it	Home owners, farmers, factory managers, hospitals, city councils
Regulators and standards bodies	Set the rules on safety, radio use, privacy, and security	National telecoms regulators, standards bodies that publish IoT security standards

Why does this matter to you, as a beginner? Two reasons.

First, it shows you **where the jobs are**. You don't need to know everything. People build whole careers in just one of these boxes — firmware engineers work on devices, cloud engineers work on platforms, data analysts work on applications.

Second, it explains **why IoT can be tricky**. Every link in the chain has to work together. When a smart camera stops working, the fault could be in the device, the Wi-Fi, the company's cloud, or the app. Thinking in layers helps you find it.

Common misconceptions

"An IoT device must connect directly to the internet." Not always. Many devices only talk to a nearby gateway using short-range radio, and only the gateway touches the internet. Plenty of battery-powered sensors work this way.

"The cloud does all the thinking." Not any more. More and more decisions are made at the edge, on the device or gateway, because it's faster and keeps working offline. The cloud is often used for the "big picture" jobs, like history and trends.

"A Raspberry Pi and an Arduino are basically the same thing." They look similar — both are small circuit boards — but they're different families. A Raspberry Pi is built around a microprocessor and runs a full operating system. A typical Arduino board is built around a microcontroller and runs one program. Each is better at different jobs. (Confusingly, the Raspberry Pi *Pico* is a microcontroller board.)

"More layers means a more advanced system." The number of layers in a diagram is just a way of explaining things. A three-layer diagram and a seven-layer diagram can describe exactly the same system.

Real-world example: a smart building

A large office building might have hundreds of IoT devices. Here's how its layers could look:

- **Devices:** temperature, occupancy, and air-quality sensors in every room; actuators that control air-conditioning vents and lights.
- **Connectivity:** battery sensors use a low-power radio to reach a gateway on each floor; the gateways connect to the building's network.
- **Edge:** each floor's gateway switches off the lights and air-con in empty rooms straight away, without waiting for the cloud.
- **Cloud platform:** stores months of data from every room, and spots which floors waste the most energy.
- **Application:** the facilities manager sees an energy dashboard; staff use an app to book meeting rooms that the sensors show are actually empty.

Same five layers as our tiny room monitor — just much bigger.

Key takeaways

- Every IoT system can be described with the same layers: **devices, connectivity, edge, cloud platform, and application**.
- Data flows **up** from devices to people; commands can flow back **down**.
- **Sensors** bring information *in* from the world; **actuators** push action *out* into the world.
- **Microcontrollers** are cheap, low-power, single-job computers — ideal for most IoT devices. **Microprocessors** are more powerful and run full operating systems.
- The **ESP32** is a popular microcontroller with Wi-Fi and Bluetooth built in, and it's what you'll use in every lab.
- A reading follows a **pipeline**: sense, process locally, act, transmit, store and analyse, present.
- IoT products depend on an **ecosystem** of chip makers, device makers, networks, cloud providers, developers, users, and regulators.

Check your understanding

Try to answer each question before opening the answer.

1. List the five layers of an IoT system, in order from the device to the person.

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

2. Is a smoke detector's alarm siren a sensor or an actuator? What about the part that detects the smoke?

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

3. You're designing a soil-moisture sensor that sits in a field and must run on a small battery for a year. Microcontroller or microprocessor? Why?

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

4. What's the main benefit of making a decision at the edge instead of in the cloud?

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

5. What is a gateway, and give one everyday example.

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

6. A smart doorbell stops showing video in its app. Using the layers, name three different places the fault could be.

→ *Answer A8 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
Actuator	A part that turns an electrical signal into a physical action, like light, sound, or movement.
Application layer	The part of an IoT system people see and use — dashboards, apps, alerts.
Cloud	Powerful computers in data centres that you use over the internet instead of owning.
Cloud platform (IoT platform)	A cloud service built to store, process, and act on device data.
Connectivity	How data gets off a device and commands get back to it — the radio and the "language" it speaks.
Data pipeline	The series of steps data passes through, from being sensed to being shown to a person.
Deep sleep	A very-low-power mode in which a microcontroller switches almost everything off between jobs.
Ecosystem	The network of companies and people who together make an IoT product work.
Edge computing	Processing data and making decisions close to the device, instead of in a distant cloud.
ESP32	A low-cost microcontroller from Espressif with Wi-Fi and Bluetooth built in. Used in all of the labs.
Gateway	A middleman device that connects nearby devices to the internet.
Microcontroller (MCU)	A complete, tiny, low-power computer on one chip that runs one program.
Microprocessor (MPU)	A more powerful processor that needs extra chips and usually runs a full operating system.
Raspberry Pi	A popular small computer board built around a microprocessor; it runs a full operating system.
Sensor	A part that measures something in the physical world and turns it into an electrical signal.
Stakeholder	Any person or organisation with an interest in how an IoT system turns out.

Up next

In Chapter 3 — Sensors, Actuators, and Reading Data, you'll zoom right in on Layer 1. You'll learn how sensors actually produce their signals, the difference between **analog** and **digital**, and then you'll open the simulator and wire up your first circuit in Lab 1.

Going further (optional)

- Espressif's ESP32 overview — the chip maker's own page, with the full specifications.
- Raspberry Pi documentation — if you're curious about the microprocessor side of the family.
- AWS: What is IoT? — a short, readable overview of IoT architecture from one of the big cloud providers.

Answer key

A1 · Pause and check — Our room monitor switches the fan on by itself when it gets hot, without asking the cloud. Which layer is making that decision?

The **edge** layer. The decision happens locally, on the device itself, not in the cloud.

A2 · Pause and check — In the pipeline above, which step would still work if the Wi-Fi went down?

Steps 1, 2 and 3 — sensing, checking, and switching the fan — all happen on the device, so they keep working. Steps 4 to 6 need the connection, so they'd pause until Wi-Fi came back. (A well-designed device would also save readings while offline and send them later.)

A3 · Question 1 — List the five layers of an IoT system, in order from the device to the person.

Devices → Connectivity → Edge → Cloud platform → Application.

A4 · Question 2 — Is a smoke detector's alarm siren a sensor or an actuator? What about the part that detects the smoke?

The siren is an **actuator** — it takes an electrical signal and turns it into sound, pushing action out into the world. The smoke-detecting part is a **sensor** — it brings information in from the world.

A5 · Question 3 — You're designing a soil-moisture sensor that sits in a field and must run on a small battery for a year. Microcontroller or microprocessor? Why?

A **microcontroller**. It uses very little power, starts instantly, and the job (read a sensor, send a small number) is simple. A microprocessor running a full operating system would drain the battery far too quickly.

A6 · Question 4 — What's the main benefit of making a decision at the edge instead of in the cloud?

It's **faster** (no round trip over the internet) and it **keeps working if the connection drops**. It can also reduce how much data needs to be sent.

A7 · Question 5 — What is a gateway, and give one everyday example.

A gateway is a middleman device that collects data from nearby devices and forwards it to the internet (and passes commands back). Your home Wi-Fi router, or a smart-home hub, are everyday examples.

A8 · Question 6 — A smart doorbell stops showing video in its app. Using the layers, name three different places the fault could be.

Any three of: the **device** (camera broken or no power), the **connectivity** (Wi-Fi down or weak), the **cloud platform** (the company's servers are having an outage), or the **application** (the phone app needs an update or you're logged out).

CHAPTER 3

Sensors, Actuators, and Reading Data

How does a device "feel" the temperature of a room, or switch on a fan? This chapter explains how sensors and actuators work, what analog and digital signals are, and how a microcontroller reads them. Then you'll try it yourself in Labs 1 and 2.

At a glance	
Reading time	About 20 minutes
Labs	Lab 1 — Digital vs Analog and Lab 2 — DHT22 Temperature & Humidity
Before this	Chapter 2 — How an IoT System Is Built, and the Wokwi quickstart
You'll need	A laptop with a browser, and your free Wokwi account

What you'll learn

By the end of this chapter, you'll be able to:

1. **Describe** five common types of sensors and what each one measures.
2. **Describe** four common actuators and when you'd use each one.
3. **Explain** the difference between an analog and a digital signal, with an example of each.
4. **Explain** how a microcontroller turns a sensor's signal into a useful number.
5. **Read** a short Arduino program and say what each part does.
6. **Connect and read** a real sensor in the Wokwi simulator.

Where we are

In Chapter 2, you learned the five layers of an IoT system. The bottom layer was **devices** — the physical "things", with **sensors** to take information in and **actuators** to act on the world.

Now we zoom right in on that bottom layer. This is where the physical world meets the digital one. Everything else in IoT — the Wi-Fi, the cloud, the dashboards — depends on getting this part right. A dashboard showing wrong temperatures is worse than no dashboard at all.

Our running example is still the **smart room monitor**: a small box that measures temperature and humidity and switches on a fan when it gets too hot. By the end of Lab 2, you'll have built its "senses".

The big picture

Every sensor reading follows the same short journey inside the device:

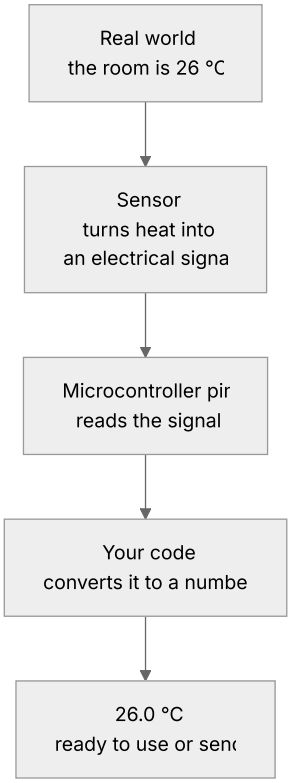


Figure 3.1 — a real-world quantity is turned into an electrical signal by a sensor, read by a pin on the microcontroller, and converted by the code into a meaningful number.

And every action follows the same journey in reverse: your code decides something, a pin sends out a signal, and an actuator turns that signal into light, sound, or movement.

Keep this picture in mind. The rest of the chapter fills in each box.

Sensors: how devices feel the world

A **sensor** measures something physical and turns it into an electrical signal. Most sensors work because some material changes its electrical behaviour when the world around it changes. For example, some materials let electricity through more easily when they get warmer.

You don't need to know the physics inside each sensor to use it. You just need to know **what it measures**, **what kind of signal it gives you**, and **how often you can read it**. Here are five of the most common types.

Temperature sensors

These measure how hot or cold something is. A common, simple type is the **thermistor** — a tiny part whose electrical resistance changes with temperature. More advanced temperature sensors have a small chip inside that does the measuring for you and sends back a ready-made number.

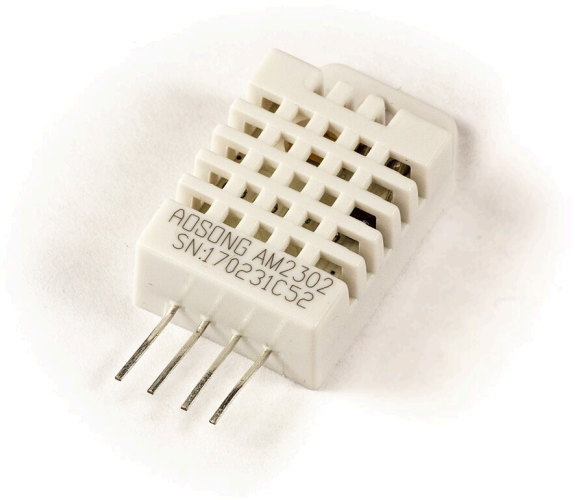
Where you'll find them: thermostats, fridges, weather stations, engines, server rooms.

Humidity sensors

These measure how much water vapour is in the air. They usually report **relative humidity**, as a percentage: 0% is completely dry air, and 100% means the air can't hold any more moisture (that's when you get fog or dew).

Where you'll find them: greenhouses, museums, food storage, bathrooms (to switch on the extractor fan).

> **NOTE**
>
> Temperature and humidity are so often measured together that many sensors do both. The **DHT22** you'll use in Lab 2 is one of these — one small part, two readings.



The DHT22 (also sold as AM2302): temperature and humidity in one small part — the same sensor you'll use in Lab 2. Photo: Ubahnverleih, CC0, via Wikimedia Commons.

Light sensors

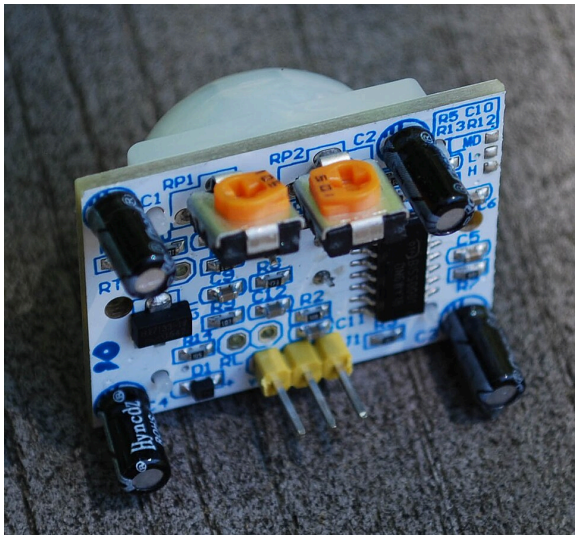
These measure how bright it is. The simplest is the **photoresistor** (also called a light-dependent resistor, or LDR), whose resistance drops as more light falls on it.

Where you'll find them: phone screens that dim in the dark, street lights that switch on at dusk, cameras.

Motion sensors

The most common type for rooms is the **PIR sensor** (passive infrared). Everything warm gives off invisible infrared light, including people. A PIR sensor notices when that pattern of warmth *changes* — which happens when a person walks past.

Where you'll find them: security lights, automatic doors, alarm systems, lights in office toilets that switch on when you walk in.



A PIR motion-sensor module. The two orange dials adjust sensitivity and how long it stays on. Photo: Sjilla, CC BY-SA 4.0, via Wikimedia Commons.

Gas sensors

These detect particular gases in the air, such as carbon monoxide, carbon dioxide, methane, or cooking gas (LPG). Many cheap hobby gas sensors (the "MQ" family) contain a small heated element that changes resistance when certain gases touch it.

Where you'll find them: carbon-monoxide alarms, gas-leak detectors, air-quality monitors, kitchens, factories and mines.

The sensor cheat sheet

Sensor type	Measures	Typical signal	Everyday example
Temperature	Heat	Analog or digital	Smart thermostat
Humidity	Water vapour in air (%)	Usually digital	Bathroom fan controller
Light	Brightness	Usually analog	Auto-dimming phone screen
Motion (PIR)	Movement of warm bodies	Digital (motion / no motion)	Security light
Gas	Specific gases in the air	Usually analog	Carbon-monoxide alarm

Don't worry about the "typical signal" column yet — that's exactly what the next section explains.

Actuators: how devices act on the world

An **actuator** takes a signal from the microcontroller and turns it into a physical action. Here are four of the most common.

LEDs

An **LED** (light-emitting diode) is a small, efficient light. It's the "hello world" of electronics: the first thing almost everyone learns to control. In IoT, LEDs are often used as **status lights** — green for "all good", flashing for "connecting", red for "problem".

One important detail: an LED needs a **resistor** in its circuit. A resistor limits how much electricity flows. Without it, an LED can draw too much current and burn out. You'll see one in Lab 1's circuit.

Buzzers

A **buzzer** makes a beep or tone when it receives a signal. It's the simplest way to get someone's attention — think of a microwave's "done" beep, or a smoke alarm. You'll use one in Lab 5 as an alarm.

Relays

A **relay** is an electrically controlled switch. It's how a tiny, low-power microcontroller can switch on something big — a fan, a water pump, a heater, or a light that runs on mains electricity.

The microcontroller can only safely provide a small amount of power from its pins. It can't run a fan directly. So instead, it sends a small signal to the relay module, whose built-in transistor powers the relay's coil — and the relay closes a separate, stronger circuit — like your finger flicking a wall switch that controls a much bigger current.

A relay also keeps the two circuits **physically separate**, which protects the delicate microcontroller from the high-power side. You'll use a relay in Labs 5 and 6.

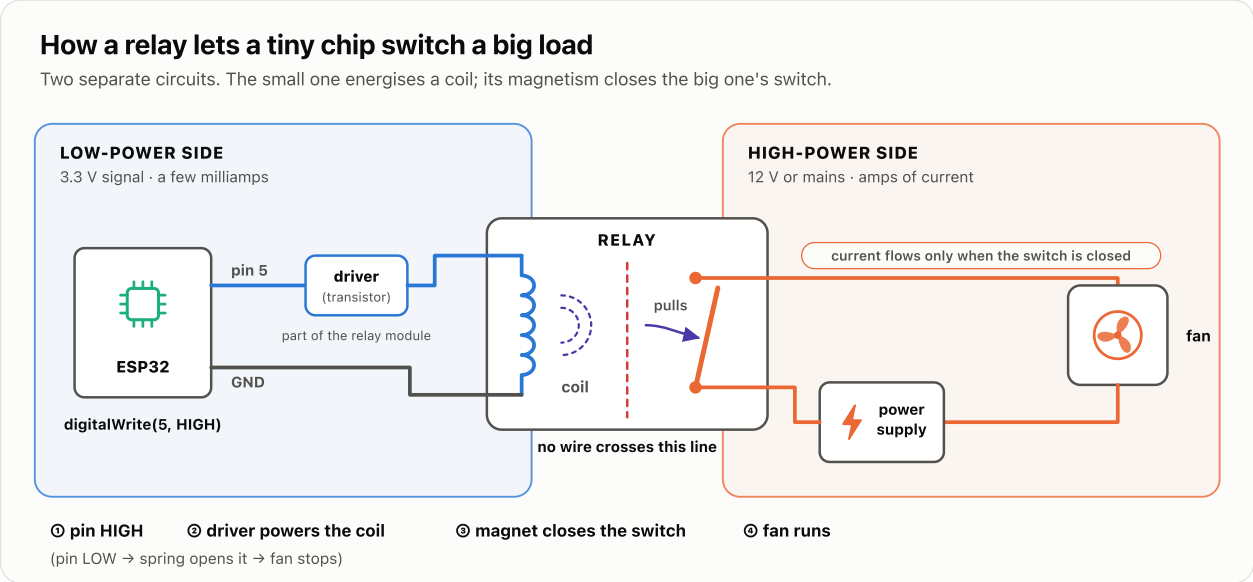


Figure 3.2 — how a relay works. No wire crosses from the small circuit to the big one — magnetism does the switching.



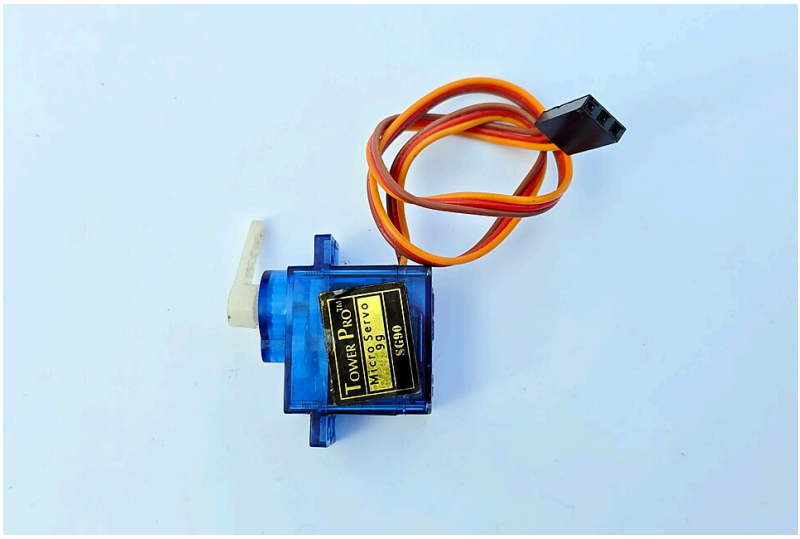
A typical 5 V relay module. The screw terminals connect the high-power circuit. Photo: Suyash Dwivedi, CC BY-SA 4.0, via Wikimedia Commons.

> **WARNING**
>
> Mains electricity (the power from a wall socket) can kill. In the labs, everything is simulated, so it's completely safe. If you later build real projects, use ready-made relay modules rated for the job, and never work on mains wiring unless you are qualified to do so.

Motors

Motors create movement: spinning a fan, opening a valve, moving a robot arm, or turning a lock. There are several types. **DC motors** spin continuously, **servo motors** turn to an exact angle (great for opening a vent halfway), and **stepper motors** move in precise small steps (used in 3D printers).

Like relays, motors need more power than a microcontroller pin can give, so they're connected through a **motor driver** — a helper circuit that does the heavy lifting. (A small hobby servo, like the one pictured below, has its driver built in: its signal wire connects straight to a pin, but it still needs its own power supply.)



A small SG90 servo motor. It turns to an exact angle — handy for opening a vent part-way. Photo: Suyash Dwivedi, CC BY-SA 4.0, via Wikimedia Commons.

The actuator cheat sheet

Actuator	Does	Controlled by	Typical IoT use
LED	Makes light	Directly from a pin (with a resistor)	Status indicator
Buzzer	Makes sound	Directly from a pin	Alarms, alerts
Relay	Switches a bigger circuit on/off	A pin, via a relay module	Fans, pumps, heaters, lights
Motor	Makes movement	A pin, via a motor driver	Fans, valves, locks, robots

TIP

Pause and check. You want your room monitor to open a window vent to exactly half-way. Which actuator would you choose?

→ *Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

Analog vs digital signals

This is the most important idea in this chapter, and it's simpler than it sounds.

Digital: only two states

A **digital** signal has only two possible values: **on or off**. In electronics, we call these **HIGH** and **LOW**, or **1** and **0**.

Think of an ordinary light switch. It's either up or down. There's no "somewhere in between".

- A **push button** is digital: pressed or not pressed.
- A **PIR motion sensor** is digital: motion or no motion.
- Switching an **LED** fully on or fully off is a digital *output*.

On the ESP32, HIGH means the pin is at about 3.3 volts, and LOW means it's at 0 volts.

Careful: "digital" is also used for sensors, like the DHT22, that send whole numbers as a stream of 1s and 0s. Either way, the wire itself only ever carries HIGH or LOW.

Analog: a smooth range

An **analog** signal can be any value within a range. Think of a dimmer switch or a volume knob. It isn't just "on" or "off" — it can be anywhere in between.

- A **potentiometer** (a knob) is analog: it can be turned to any position.
- A **light sensor** is analog: it reports *how* bright, not just "light or dark".
- A simple **thermistor** is analog: the voltage changes smoothly as the temperature rises.



A potentiometer. Turning the shaft changes its output smoothly — Lab 1 uses a simulated one. Photo: Iainf, CC BY 2.5, via Wikimedia Commons.

Side by side

	Digital	Analog
Values	Two: HIGH/LOW, 1/0, on/off	Any value in a range
Everyday comparison	Light switch	Dimmer switch, volume knob
Sensor examples	Push button, PIR motion sensor	Potentiometer, light sensor, thermistor
Read in code with	`digitalRead()`	`analogRead()`
Question it answers	"Is it happening?"	"How much?"

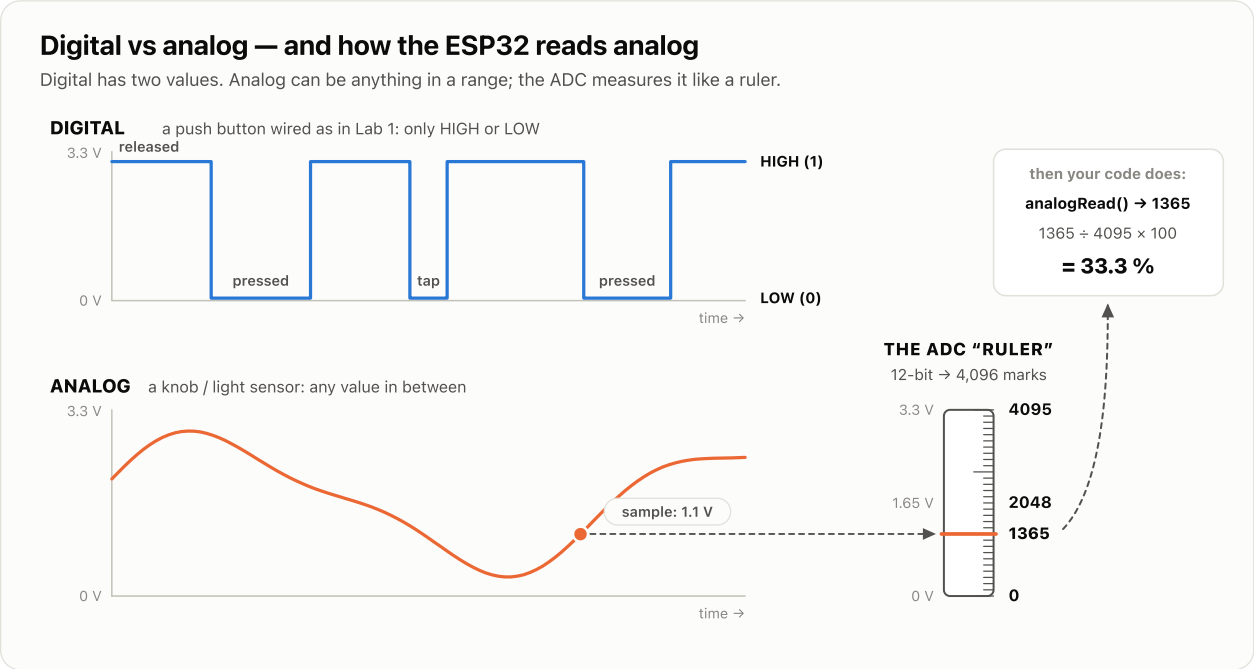


Figure 3.3 — digital vs analog, and how the ADC turns a voltage into a number your code can use.

How a microcontroller reads an analog signal

Here's a puzzle. A microcontroller is a digital computer — deep down, it only understands 1s and 0s. So how does it read a smooth, analog signal?

It uses a built-in part called an **ADC** — an **analog-to-digital converter**. An ADC measures the voltage on a pin and turns it into a whole number.

Think of it like measuring a person's height with a ruler. Their real height is a smooth value, but you write down the nearest millimetre mark. The more marks on your ruler, the finer your measurement.

The ESP32's ADC is **12-bit**, which means its "ruler" has 4,096 marks (that's 2 multiplied by itself 12 times). So `analogRead()` on the ESP32 gives you a number from **0 to 4095**:

Voltage on the pin	`analogRead()` returns
0 V (nothing)	0
About 1.65 V (half-way)	About 2048
About 3.3 V (full)	4095

On its own, "2048" doesn't mean much to a person. So your code usually **converts** the raw number into something useful. In Lab 1, you'll turn it into a percentage:

```
float potPercent = (potRaw / 4095.0) * 100.0;   // 2048 becomes about 50.0 %
```

This pattern — **read a raw number, then convert it into real units** — is at the heart of every sensor reading you'll ever do.

> **NOTE**
>
> Older Arduino boards, like the classic Arduino Uno (R3), have a 10-bit ADC, so their `analogRead()` returns 0 to 1023 instead. If you follow an online tutorial written for an Uno, remember that the numbers will be different on an ESP32.

> **IMPORTANT**
>
> **An ESP32 gotcha worth remembering.** The ESP32 has two groups of analog pins, called ADC1 and ADC2. The **ADC2 pins can't be read while Wi-Fi is switched on**, because the Wi-Fi radio uses that part of the chip. For any connected project, plug analog sensors into **ADC1 pins (GPIO 32 to 39)**. Lab 1 uses GPIO 34, an ADC1 pin, so it keeps working when you add Wi-Fi later. The ESP32's ADC is also a little less accurate right at the very bottom and top of its range, so don't rely on readings very close to 0 V or 3.3 V.

Can a microcontroller send out an analog signal?

Mostly, a microcontroller pin can only be fully on or fully off. But it can *fake* an in-between value using a trick called **PWM** (pulse-width modulation). It switches the pin on and off very fast — hundreds or thousands of times a second. If the pin is on for half of each cycle, an LED looks half as bright to your eye, and a motor spins at about half speed.

You won't need PWM in the labs, but it's how dimmable lights and variable-speed fans work. It's a good challenge to try at the end of Lab 1.

Sensor interfacing: connecting parts the right way

Interfacing means connecting a sensor or actuator to a microcontroller so they can work together correctly. Here are the principles that matter most for a beginner.

1. Every part needs power, ground, and a signal

Nearly every sensor has at least three connections:

- **Power** (often labelled VCC, 3V3, or 5V) — the supply that makes it work.
- **Ground** (GND) — the return path for electricity. Every part in a circuit must share the same ground.
- **Signal** (or DATA, OUT, SDA) — the wire that carries the reading to a microcontroller pin.

Look at the wiring table in any of the labs, and you'll see exactly this pattern: red wire to power, black wire to ground, and a coloured wire to a signal pin.

2. Match the voltage

The ESP32 works at **3.3 volts**. Some sensors are designed for 5 volts. A 5-volt signal going into a 3.3-volt pin can damage the chip. Always check that a part is suitable for 3.3 V, or use a small level-shifting board in between. (All of the labs' simulated parts are already correctly matched.)

3. Don't leave inputs "floating"

When a button isn't pressed, the pin it connects to isn't attached to anything. An unconnected input pin is said to be **floating**, and it picks up random electrical noise — so it may read HIGH, then LOW, then HIGH again, all on its own.

The fix is a **pull-up resistor**: a resistor that gently holds the pin at HIGH until the button pulls it firmly to LOW. The ESP32 has pull-up resistors built in, so you can switch one on in code:

```
pinMode(BUTTON_PIN, INPUT_PULLUP);   // HIGH when released, LOW when pressed
```

That's why, in Lab 1, a pressed button reads as **LOW**. It feels backwards at first, but it's the standard, reliable way to wire a button.

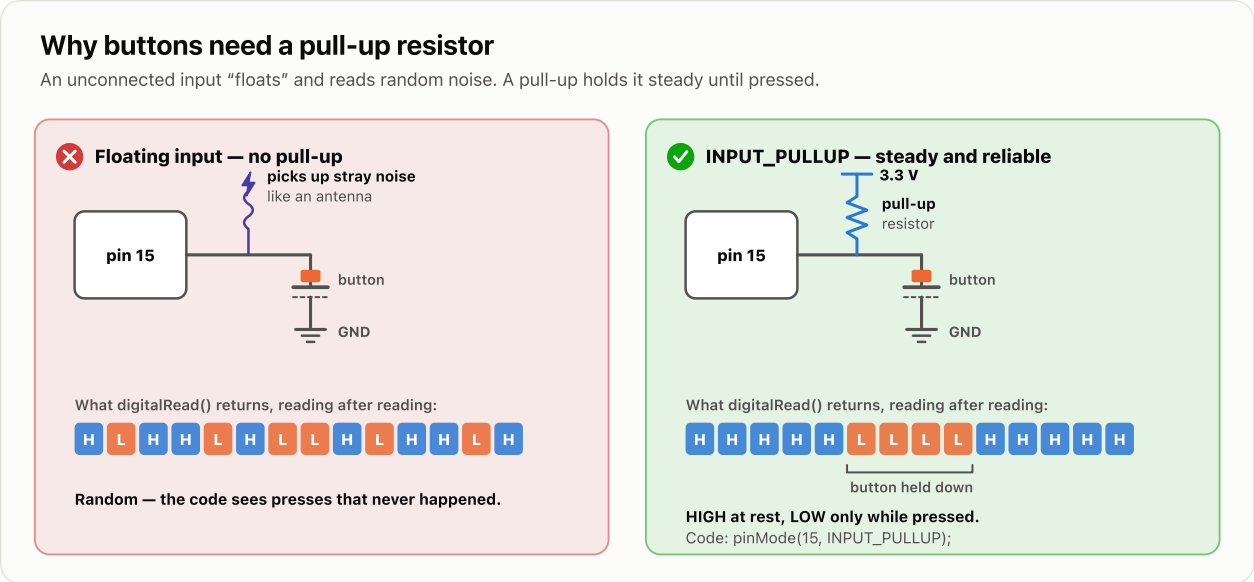


Figure 3.4 — without a pull-up the pin reads noise; with one, it reads HIGH until the button pulls it LOW.

4. Know how the sensor sends its data

Some sensors give a simple analog voltage or a single on/off signal. Others contain their own tiny chip and send a whole digital message — for example, "the temperature is 24.1 and the humidity is 40.0". These smart sensors use a **communication protocol**, a set of rules for sending data down a wire. The common ones are:

Protocol	How it connects	You'll meet it in
Single-wire (sensor-specific)	One data wire	Lab 2 — the DHT22 sensor
I²C (say "I-squared-C")	Two wires, shared by many devices	Lab 6 — the OLED display
SPI	Four wires, very fast	Displays, SD cards (not used in the labs)
UART (serial)	Two wires, one each way	GPS modules; also the Serial Monitor

You don't need to understand how these work inside. The good news is that for most parts, someone has written a **library** — ready-made code that handles the protocol for you. That's why Lab 2 uses a DHT library: instead of decoding the sensor's signal yourself, you call one function and get the temperature back.

5. Respect the sensor's limits

Every sensor needs time to take a measurement. The DHT22, for example, can only give a fresh reading about **once every two seconds**. If you ask faster, you just get old or invalid data. That's why Lab 2's code waits two seconds between readings.

Every sensor also has a limit on **how accurate** it is and **what range** it can measure. The sensor's **datasheet** — the technical document from its maker — tells you these limits. Here's what the DHT22's datasheet says:

What it measures	Range	Accuracy
Temperature	−40 °C to 80 °C	±0.5 °C
Relative humidity	0% to 100%	±2% (up to ±5% at worst)

So if your DHT22 reads 24.0 °C, the real temperature is probably somewhere between 23.5 °C and 24.5 °C. That's excellent for a room monitor — but you wouldn't use it for a scientific experiment that needs 0.1 °C precision. Choosing a sensor always means matching its limits to your job.

6. Expect a little noise, and be ready to smooth it

Real readings wobble slightly: 24.1, 24.3, 24.0, 24.2. That's normal, and it's called **noise**. A common fix is to take several readings and use their **average**. It's also why real products are **calibrated** — checked against a trusted reference, then adjusted — before they're relied on.

> **TIP**
>
> **Pause and check.** Your button randomly shows "pressed" even when nobody is touching it. What's the most likely cause, and the fix?
>
> *→ Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

Reading your first Arduino program

In the labs, you'll paste in code that's already written for you. You don't need to write code from scratch for the labs, but it helps a lot to understand what you're looking at. Here's a gentle tour.

The code for the ESP32 in Wokwi is written in the **Arduino** style, which is the C++ programming language with some helpful built-in commands. Every Arduino program (called a **sketch**) has two main parts:

```
void setup() {
  // Runs ONCE, when the device powers on.
  // Used to get things ready.
}

void loop() {
  // Runs AGAIN AND AGAIN, forever, after setup() finishes.
  // This is where the device does its job.
}
```

Think of `setup()` as a shop opening in the morning — unlocking the door, switching on the lights — and `loop()` as serving customers, one after another, all day long.

Here are the handful of commands you'll see in Labs 1 and 2:

Command	What it does	What it means
`pinMode(2, OUTPUT);`	Sets pin 2 as an output	"I'm going to control pin 2."
`digitalWrite(2, HIGH);`	Turns pin 2 on	"Switch the LED on."
`digitalRead(15)`	Reads pin 15 as HIGH or LOW	"Is the button pressed?"
`analogRead(34)`	Reads pin 34 as 0–4095	"How far is the knob turned?"
`delay(2000);`	Waits 2,000 milliseconds	"Wait two seconds."
`Serial.begin(115200);`	Opens a text channel to your screen	"Get ready to print messages."
`Serial.println("Hi");`	Prints a line of text	"Show 'Hi' in the Serial Monitor."

Two more things you'll notice:

- **Comments.** Anything after `//` is a note for humans. The computer ignores it. The lab code is full of comments to explain itself — read them!
- **The Serial Monitor.** The ESP32 has no screen. `Serial.println()` sends text back to the Serial Monitor panel in Wokwi, so you can see what the device is doing. It's your window into the device's mind, and it's the first place to look when something doesn't work.

Common misconceptions

"Digital means it's more accurate than analog." Not really. Digital and analog describe the *type* of signal, not its quality. An analog light sensor can tell you more (*how* bright) than a simple on/off light-detector module (just "light or dark").

“The microcontroller can power anything directly from its pins.” No. A pin can safely drive something small, like an LED with a resistor. Anything bigger — a fan, a motor, a pump — needs a relay or a driver in between. Trying without one can damage the chip.

“If the code shows a number, the reading must be right.” Sensors can be wrongly wired, faulty, uncalibrated, or read too quickly. Always ask: *is this number sensible?* A room reading of −40 °C probably means a broken wire, not a sudden ice age.

“A pressed button always reads HIGH.” It depends on the wiring. With the common `INPUT_PULLUP` setup used in Lab 1, a pressed button reads **LOW**. Both ways are correct, as long as your code matches your wiring.

Real-world example: a greenhouse monitor

A vegetable grower wants to protect their greenhouse crops. Here’s how this chapter’s ideas fit together in their system:

- **Sensors:** a temperature and humidity sensor (digital, via its own protocol), a light sensor (analog, read through the ADC), and a soil moisture sensor (analog).
- **Actuators:** a relay that switches a ventilation fan, a relay for the water pump, and a buzzer for an alarm if it gets dangerously hot.
- **Interfacing choices:** everything runs at 3.3 V; the pump and fan run on their own power supply, controlled safely through relays; readings are averaged over a minute to smooth out noise.

It’s our room monitor with a few more sensors — and it’s exactly what you’ll be able to build by the end of the book.

Key takeaways

- **Sensors** turn physical quantities into electrical signals; this chapter covered temperature, humidity, light, motion, and gas sensors.
- **Actuators** turn signals into actions; LEDs and buzzers can be driven directly, but relays and motors need helper circuits.
- **Digital** signals have two states (on/off). **Analog** signals can take any value in a range.
- The ESP32’s **ADC** turns an analog voltage into a number from **0 to 4095**, which your code then **converts** into real units.
- Good **interfacing** means: power, ground, and signal; matching voltages; no floating inputs; using libraries for smart sensors; respecting the sensor’s speed; and smoothing noise.
- Every Arduino sketch has a `setup()` that runs once and a `loop()` that runs forever. The **Serial Monitor** shows you what the device is doing.

Check your understanding

1. Name one sensor that gives a digital signal and one that gives an analog signal.

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

2. On an ESP32, `analogRead()` returns 1024. Roughly what percentage of the full range is that?

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

3. Why can't you connect a fan straight to an ESP32 pin?

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

4. What's the difference between `setup()` and `loop()`?

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

5. Your DHT22 code reads the sensor 10 times a second, and the readings look strange. What's wrong?

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

6. What does a library do for you when using a sensor like the DHT22?

→ *Answer A8 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
ADC (analog-to-digital converter)	A part of the microcontroller that turns an analog voltage into a number. On the ESP32, the number is 0–4095.
Analog signal	A signal that can take any value within a range, like a dimmer switch.
Buzzer	An actuator that beeps or sounds a tone when it receives a signal.
Calibration	Checking a sensor against a trusted reference and adjusting it so its readings are accurate.
Datasheet	The technical document from a part's maker, listing its limits and how to use it.
Digital signal	A signal with only two states: HIGH/LOW, 1/0, on/off.
Floating input	An input pin that isn't connected to anything, so it gives random readings.
Interfacing	Connecting a sensor or actuator to a microcontroller so they work together correctly.

Term	Meaning
I²C	A two-wire protocol that lets many devices share the same connection. Used by the OLED display in Lab 6.
LED	Light-emitting diode — a small, efficient light, often used as a status indicator.
Library	Ready-made code that handles a complicated job, like talking to a particular sensor.
Motor driver	A helper circuit that lets a microcontroller control a motor safely.
Noise	Small, random wobbles in sensor readings.
Photoresistor (LDR)	A light sensor whose resistance drops as more light falls on it.
PIR sensor	A passive infrared motion sensor that detects warm bodies moving.
Potentiometer	A knob or dial whose position gives an analog signal.
Pull-up resistor	A resistor that holds an input pin HIGH until something pulls it LOW.
PWM (pulse-width modulation)	Switching a pin on and off very fast to imitate an in-between value, e.g. to dim an LED.
Relative humidity	How much water vapour the air holds, as a percentage of the most it could hold.
Relay	An electrically controlled switch that lets a small signal switch a bigger circuit.
Resistor	A part that limits the flow of electricity, e.g. to protect an LED.
Serial Monitor	A panel that shows text messages sent from the device with `Serial.print()`.
Servo motor	A motor that turns to an exact angle.
Sketch	An Arduino program. It always has a `setup()` and a `loop()`.
SPI	A fast, four-wire protocol used by displays, SD cards and similar parts.
Thermistor	A part whose resistance changes with temperature, used as a simple temperature sensor.
UART	A simple two-wire serial link, one wire each way. The Serial Monitor uses it.
Wokwi	A free online simulator that runs a real ESP32 and its circuit in your web browser.

Into the labs

Time to get hands-on. Open the simulator and work through these two labs, in order:

1. **Lab 1 — Digital vs Analog** (about 25 minutes). You'll blink an LED (digital output), read a push button (digital input, using `INPUT_PULLUP`), and read a potentiometer (analog input, 0–4095). Watch the Serial Monitor as you press the button and turn the dial.
2. **Lab 2 — DHT22 Temperature & Humidity** (about 25 minutes). You'll read a real smart sensor using a library. While the simulation runs, click the sensor to open its sliders, change the temperature, and watch your readings follow along.

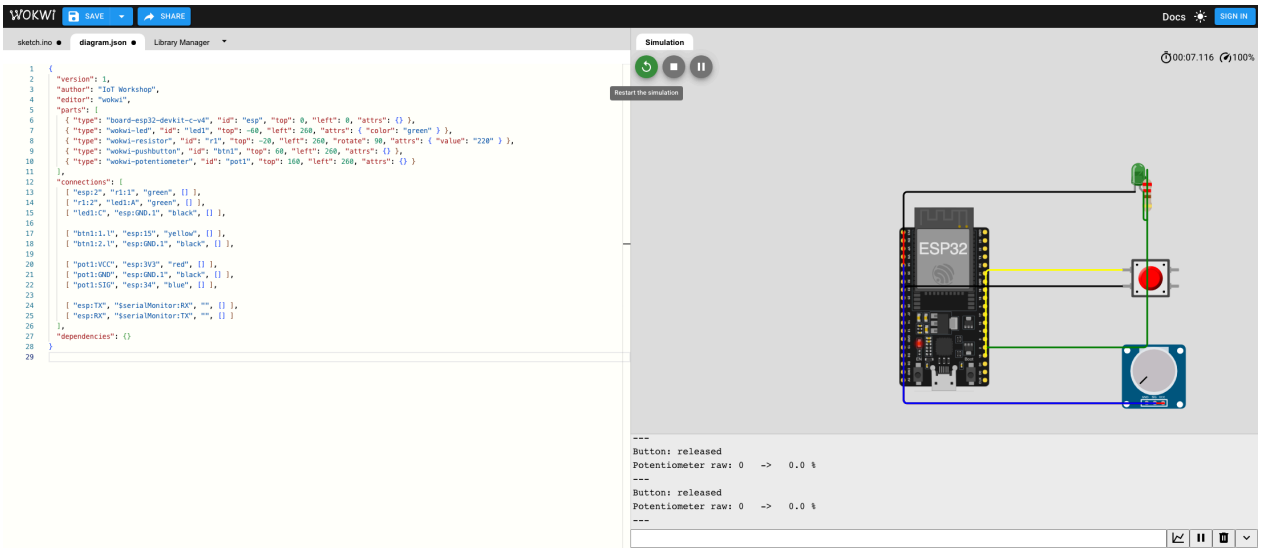


Figure 3.5 — what Lab 1 looks like in the Wokwi simulator: your code on the left, the circuit on the right, and the Serial Monitor underneath.

As you work, try to spot each idea from this chapter: the power, ground and signal wires; the raw number being converted; the delay that respects the sensor's speed limit.

Up next

In Chapter 4 — Getting Online: Connectivity and Communication Protocols, your device learns to talk to the world. You'll compare Wi-Fi, Bluetooth, LoRaWAN and other ways of connecting, meet the MQTT protocol, and send your DHT22 readings across the internet in Lab 3.

Going further (optional)

- Random Nerd Tutorials — ESP32 guides — hundreds of free, well-tested beginner projects for sensors and actuators.
- SparkFun — Analog vs Digital — a friendly, illustrated explanation of this chapter's key idea.
- SparkFun — Pull-up Resistors — more detail on floating inputs and how pull-ups fix them.

- Wokwi parts reference — every simulated sensor and actuator, with its pins and settings.

Answer key

A1 · Pause and check — You want your room monitor to open a window vent to exactly half-way. Which actuator would you choose?

A **servo motor**, because it can turn to a precise angle. (A relay could only switch something fully on or fully off.)

A2 · Pause and check — Your button randomly shows "pressed" even when nobody is touching it. What's the most likely cause, and the fix?

The input pin is probably **floating**. Fix it with a pull-up resistor — on the ESP32, just use `pinMode(pin, INPUT_PULLUP)`.

A3 · Question 1 — Name one sensor that gives a digital signal and one that gives an analog signal.

Digital: a push button, or a PIR motion sensor. Analog: a potentiometer, a light sensor (photoresistor), or a thermistor.

A4 · Question 2 — On an ESP32, analogRead() returns 1024. Roughly what percentage of the full range is that?

About **25%**. The full range is 0 to 4095, and 1024 is about a quarter of 4095 (1024 ÷ 4095 × 100 ≈ 25%).

A5 · Question 3 — Why can't you connect a fan straight to an ESP32 pin?

A microcontroller pin can only supply a very small amount of power — nowhere near enough for a fan. Connecting one directly could damage the chip. You need a **relay** (or a motor driver) to switch the fan's own, stronger power supply.

A6 · Question 4 — What's the difference between setup() and loop()?

`setup()` runs **once**, when the device starts, to get everything ready. `loop()` runs **over and over**, forever, and is where the device does its ongoing job.

A7 · Question 5 — Your DHT22 code reads the sensor 10 times a second, and the readings look strange. What's wrong?

The DHT22 can only give a fresh reading about **once every 2 seconds**. Reading it faster returns stale or invalid data. Add a delay of at least 2 seconds between readings.

A8 · Question 6 — What does a library do for you when using a sensor like the DHT22?

It handles the sensor's communication protocol for you. Instead of decoding the sensor's signal yourself, you call a simple function and get the temperature and humidity back as ready-to-use numbers.

CHAPTER 4

Getting Online: Connectivity and Communication Protocols

A sensor on its own is just a gadget. It becomes part of the Internet of Things when it can share what it knows. This chapter explains how devices connect — by Wi-Fi, Bluetooth, long-range radio or mobile networks — and the "languages" they speak once connected. Then you'll put your own device online in Lab 3.

At a glance	
Reading time	About 25 minutes
Lab	Lab 3 — Getting Online: Wi-Fi + MQTT
Before this	Chapter 3 — Sensors, Actuators, and Reading Data, and Lab 2
You'll need	Your Wokwi account, and a second browser tab for watching live messages

What you'll learn

By the end of this chapter, you'll be able to:

1. **Explain** the basic networking ideas IoT relies on: addresses, clients and servers, ports, and packets.
2. **Compare** Wi-Fi, Bluetooth Low Energy, Zigbee, LoRaWAN, and cellular IoT by range, power, and data rate.
3. **Choose** a suitable connectivity option for a given IoT project.
4. **Explain** how HTTP/REST and MQTT work, and when to use each. Describe what CoAP is for.
5. **Describe** the main security risks when devices communicate, and the basic defences.
6. **Connect** a simulated ESP32 to Wi-Fi and publish live sensor data over MQTT.

Where we are

In Chapter 3 and Labs 1–2, you gave your device its senses: it can read a button, a dial, and a real temperature and humidity sensor. But right now, those readings only appear in the Serial Monitor on your own screen.

In the IoT loop from Chapter 1 — **sense, send, decide, act** — you've done *sense*. This chapter is all about **send**. It's the "connectivity" layer from Chapter 2's five-layer diagram.

By the end of Lab 3, your smart room monitor will be broadcasting its temperature and humidity across the internet, where you — or anyone who knows your topic — can watch them arrive live.

The big picture: two separate decisions

Getting a device online always involves **two separate choices**:

1. **How it connects** — the **physical connection**, usually a radio technology like Wi-Fi or Bluetooth. This is the *road* the data travels on.
2. **What language it speaks** — the **protocol**, like MQTT or HTTP. A protocol is an agreed set of rules for how messages are formatted and exchanged. This is the *language* written on the parcels travelling down the road.

Think of posting a letter. You choose **how** to send it (by road, by air, by sea), and separately you choose **what language** to write it in. The two choices are independent: an English letter can travel by air or by sea.

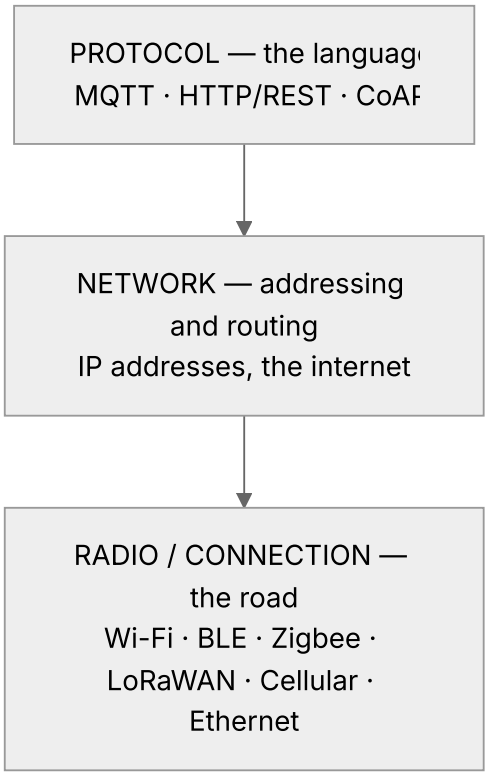


Figure 4.1 — three stacked layers. The protocol (the language) sits on top of the network (addressing and routing), which sits on top of the radio or wired connection (the road).

In Lab 3, your device uses **Wi-Fi** as its road and **MQTT** as its language. In Lab 4, it will still use Wi-Fi, but switch to **HTTP**.

Networking basics for IoT

Before we compare technologies, here are the handful of networking ideas you'll keep meeting. You don't need to be a network engineer — just know what these words mean.

Networks and the internet

A **network** is two or more devices connected so they can exchange data. Your home Wi-Fi is a **local area network (LAN)** — a small network in one place. The **internet** is a giant "network of networks" that connects millions of local networks around the world.

IP addresses: every device needs an address

To send a letter, you need an address. On a network, every device gets an **IP address** (IP stands for *Internet Protocol*), such as `192.168.1.42`. When your ESP32 joins Wi-Fi in Lab 3, it will print the IP address it has been given (in Wokwi, something like `10.13.37.2`).

People find numbers hard to remember, so the internet uses the **domain name system (DNS)** — a kind of phone book — to turn names like `broker.hivemq.com` into IP addresses automatically.

Routers and gateways

A **router** passes data between networks — for example, between your home network and the internet. Your home Wi-Fi box is a router. In IoT, you'll often hear **gateway**: a device that connects IoT devices to the internet, and sometimes translates between different radio technologies (for example, from Zigbee to Wi-Fi).

Clients and servers

Most communication on the internet follows the **client–server** pattern. A **client** makes a request ("please give me this web page", "please store this reading"). A **server** listens for requests and responds. In IoT, the device is usually the client, and a cloud platform is the server.

Ports: which door to knock on

One server can run many services at once. A **port** is a number that identifies which service you want — like a flat number in a block of flats. Some well-known ones: web pages use port **80** (HTTP) and **443** (secure HTTPS), and MQTT uses port **1883**, or **8883** when encrypted.

Packets, TCP, and UDP

Data doesn't travel as one big lump. It's split into small chunks called **packets**, which travel separately and are put back together at the other end. Two main rules govern how packets are sent:

- **TCP** (Transmission Control Protocol) checks that every packet arrives, in order, and re-sends any that go missing. It's **reliable**, like registered post. Classic HTTP and MQTT use TCP.
- **UDP** (User Datagram Protocol) just sends packets and hopes for the best. It's **lighter and faster**, but nothing is guaranteed, like ordinary post. CoAP uses UDP.

Bandwidth and latency

- **Bandwidth** (or **data rate**) is how much data can flow per second — the width of the road.
- **Latency** is how long a single message takes to arrive — the travel time.

A soil sensor sending one small number every hour needs very little bandwidth. A security camera streaming video needs a lot. This difference drives almost every connectivity decision in IoT.

TIP

Pause and check. Your ESP32 needs to send a reading to `broker.hivemq.com`. What turns that name into an address the network can use?

→ Answer A1 is in the answer key at the end of this chapter. Try it yourself first!

The big trade-off: range, power, and data rate

Every wireless technology is a compromise between three things:

- **Range** — how far the signal reaches.
- **Power** — how much battery it uses.
- **Data rate** — how much data it can carry per second.

Here's the catch: **you can't have all three.** Sending lots of data a long way takes lots of energy. So each technology picks a balance:

- **Wi-Fi** gives you lots of data across a building-sized range, but uses a lot of power.
- **Bluetooth Low Energy** uses very little power, but only reaches a short distance.
- **LoRaWAN** reaches kilometres on a tiny battery, but can only carry a few small messages.

Keep this triangle in mind. It explains every technology in the next section.

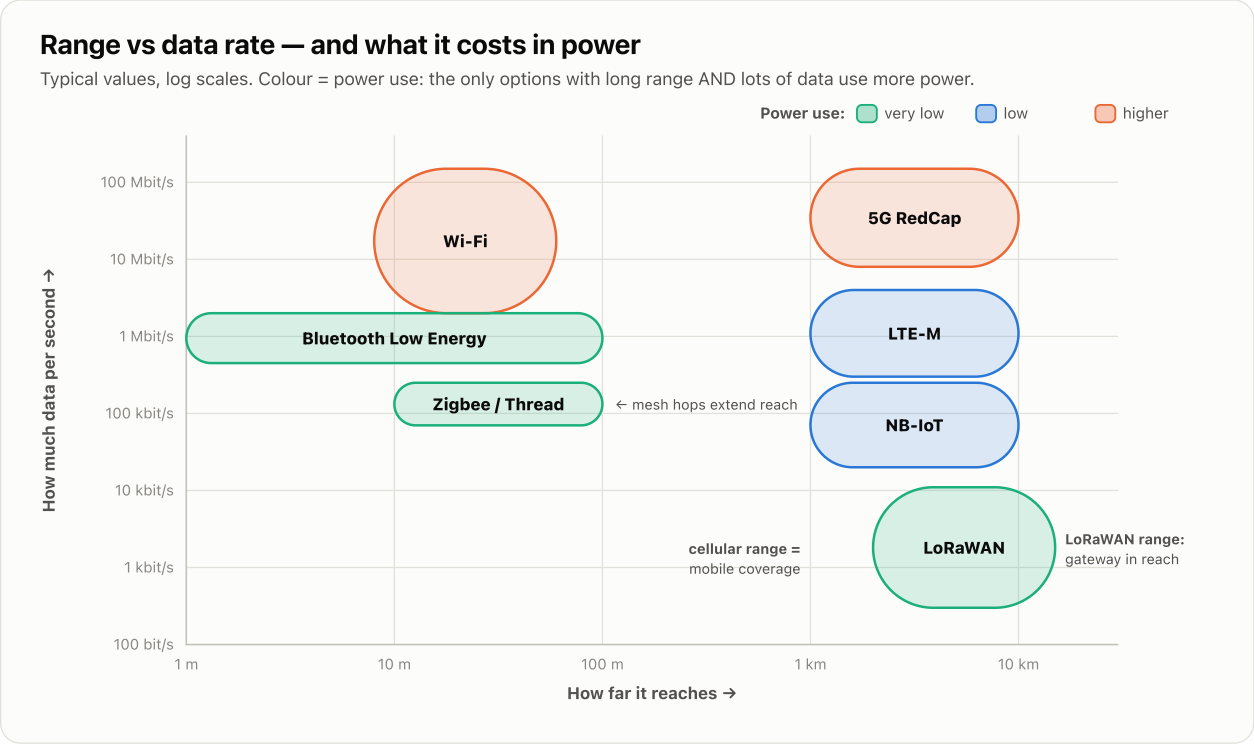


Figure 4.2 — every technology is a trade-off. The only long-range, high-data options use more power.

Communication technologies, one by one

Wi-Fi

Wi-Fi is the wireless networking you already use for your phone and laptop. Most IoT Wi-Fi uses the **2.4 GHz** radio band, because it passes through walls better than 5 GHz.

- **Range:** roughly 30–50 metres indoors, depending on walls.
- **Data rate:** high — tens of megabits per second or more; plenty for video.
- **Power:** high. Wi-Fi devices usually need mains power, or a large battery that's recharged often.
- **Good for:** devices plugged into the wall in homes and offices — smart plugs, cameras, speakers, thermostats — and anything needing lots of data.
- **Watch out for:** battery life, and the fact that each device has to be given the Wi-Fi password.

It's the most common way IoT devices connect today, and it's what your ESP32 uses in the labs. (Wokwi provides a simulated network called `Wokwi-GUEST` with real internet access.)

> **NOTE**
>
> A newer variant called **Wi-Fi HaLow** uses a lower radio frequency to reach up to about a kilometre with much lower power. It's aimed at IoT, but it's still far less common than ordinary Wi-Fi.

Bluetooth Low Energy (BLE)

Bluetooth Low Energy is a power-saving version of Bluetooth, designed for small battery-powered devices that send small amounts of data.

- **Range:** typically about 10 metres indoors, and more in the open. A long-range mode in newer versions can reach much further, at a lower data rate.
- **Data rate:** 1–2 megabits per second at most — enough for readings, not for video.
- **Power:** very low. Many BLE devices run for months or years on a coin battery.
- **Good for:** wearables, fitness trackers, medical sensors, smart locks, item trackers, and devices that talk to a nearby phone.
- **Watch out for:** short range. BLE devices usually need a phone or a gateway nearby to reach the internet.

The ESP32 has BLE built in, alongside its Wi-Fi.

Zigbee (and Thread)

Zigbee is a low-power radio technology designed for smart homes and buildings. Its superpower is the **mesh network**: each mains-powered device can pass messages on to the next, like people passing a note along a row, so the network can cover a whole building even though each hop is short.

- **Range:** about 10–100 metres per hop, extended by the mesh.
- **Data rate:** low (250 kilobits per second).
- **Power:** very low.
- **Good for:** smart lights, switches, door and window sensors, thermostats, and building automation.
- **Watch out for:** it needs a **hub** or gateway to reach the internet.

Thread is a newer mesh technology with a similar idea, and it's one of the networks used by **Matter**, the smart-home standard that lets devices from different brands work together. Matter can run over Wi-Fi, Thread, or Ethernet.

LoRaWAN

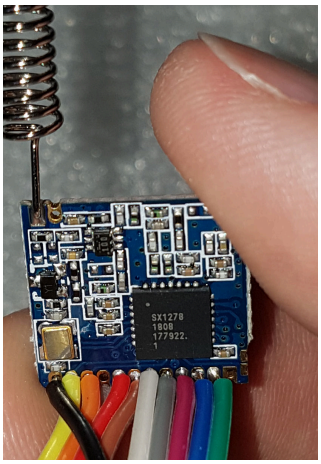
LoRaWAN (from "long range wide area network") is built for sensors that are **far away** and must run **for years on a battery**, sending only small amounts of data.

- **Range:** often over 10 km in the countryside, and a few kilometres in a dense city.
- **Data rate:** very low — from well under one kilobit to tens of kilobits per second. Think "a few short messages an hour", not "a stream".
- **Power:** extremely low. Many devices are designed to last up to 10 years on a battery.
- **Good for:** smart agriculture (soil, weather, water tanks), smart city sensors (parking, bins, water meters), and tracking over large sites.
- **Watch out for:** you can't send much data, and you need a LoRaWAN **gateway** within range. Many organisations set up their own gateways.

LoRaWAN uses radio bands that don't require a licence, and the frequencies differ by country. Malaysia uses the band around **919–924 MHz** (called **AS923**), so hardware must be the right regional version.



A LoRaWAN gateway high on a mast. One gateway can hear sensors kilometres away. Photo: Fabian Horst, CC BY 4.0, via Wikimedia Commons.



A LoRa radio module — the kind of part inside a battery-powered sensor. (This one is a 433 MHz version; Malaysia's AS923 band needs a module for about 920 MHz.) Photo: Cnd, CC0, via Wikimedia Commons.

Cellular IoT: NB-IoT, LTE-M, and 5G

Cellular IoT uses the same mobile networks as your phone, with a SIM card. Its big advantage is that it works **anywhere there's mobile coverage** — no Wi-Fi or gateway needed. Mobile operators have created special versions for IoT:

Type	What it's good at	Example uses
NB-IoT (Narrowband IoT)	Very low power, very small amounts of data, and excellent coverage deep inside buildings and basements	Water and gas meters, parking sensors
LTE-M	Low power, more data than NB-IoT, and it keeps working while moving	Vehicle and asset trackers, wearables, alarm panels
5G RedCap ("reduced capability")	A simpler, lower-power version of 5G for mid-range devices	Industrial sensors, wearables, video cameras

- **Range:** as far as the mobile network reaches.
- **Power:** low for NB-IoT and LTE-M; higher for 5G.
- **Watch out for:** a monthly data plan per device, and coverage in remote areas.

> **IMPORTANT**
>
> **Old networks are being switched off.** Around the world, operators are shutting down older 2G and 3G networks. Malaysia switched off its 3G networks in 2022. A device built for an old network will simply stop working when that network goes. New cellular IoT products should use NB-IoT, LTE-M, or 5G. In Malaysia, operators such as Maxis offer NB-IoT coverage for businesses.

Wired connections

Not everything has to be wireless. In factories and buildings, **Ethernet** cables (like the ones for office computers) and older industrial wiring standards are still very common. Wired connections are fast, reliable, and harder to interfere with. The downside, of course, is the cable.

Comparing short-range and long-range options

Here's everything side by side. The figures are **typical values** to help you compare — real results depend on the environment, antennas, and settings.

Technology	Range	Data rate	Power use	Needs a hub/gateway?	Typical use
Wi-Fi	Short (~30–50 m indoors)	High	High	Uses a Wi-Fi router	Cameras, smart plugs, home devices
Bluetooth LE	Very short (~10 m+)	Medium-low	Very low	Usually a phone or gateway	Wearables, trackers, locks
Zigbee / Thread	Short per hop, mesh extends it	Low	Very low	Yes, a hub	Smart lights, sensors, switches
LoRaWAN	Long (km)	Very low	Extremely low	Yes, a LoRaWAN gateway	Farms, cities, remote sensors
NB-IoT / LTE-M	Long (mobile coverage)	Low to medium	Low	No — uses the mobile network	Meters, trackers
Ethernet (wired)	Length of the cable	Very high	Mains-powered	Network switch	Factories, fixed equipment

How to choose: four questions

When you pick a connectivity option for a project, ask:

1. **How far** must the data travel to reach a network?
2. **How is it powered** — mains or battery? If battery, how long must it last?

3. **How much data**, and how often? (One number an hour, or live video?)
4. **What already exists** there — Wi-Fi, mobile coverage, a gateway? And what will it **cost** per device, per month?

> **TIP**
>
> **Pause and check.** Pick a technology for each: (a) a water-level sensor in a river 8 km from town, on a battery; (b) a security camera in an office; (c) a heart-rate chest strap that talks to a phone.
>
> *→ Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

IoT protocols: the languages devices speak

Once a device is connected, it needs a shared language with whatever it's talking to. Here are the three protocols in this book.

HTTP and REST: ask and answer

HTTP (Hypertext Transfer Protocol) is the language of the web. Every time you open a web page, your browser speaks HTTP. It follows a simple **request–response** pattern: the client asks, the server answers.

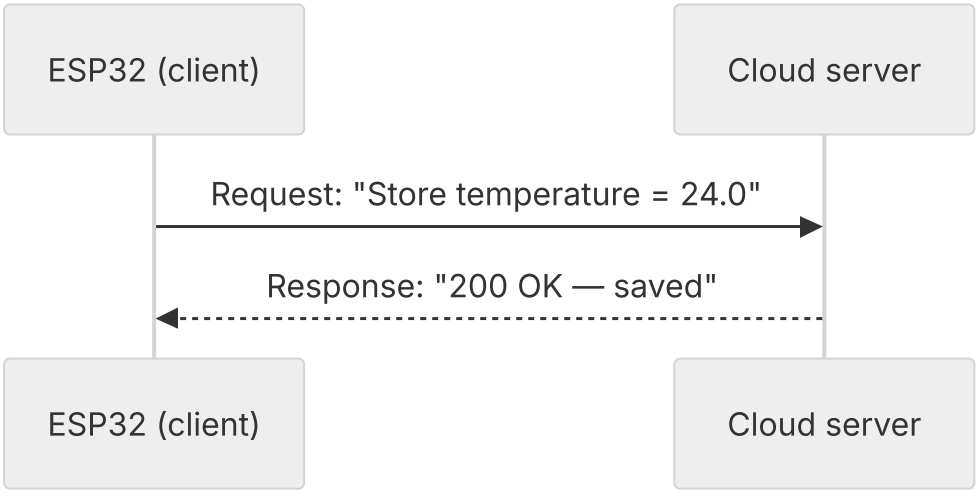


Figure 4.3 — the device sends a request to the server, and the server sends back a response.

REST is a popular style of using HTTP to build services (called **APIs** — application programming interfaces — which are doorways that let programs talk to each other). In REST, each thing you can work with has its own web address, and you use a small set of **methods** (verbs) to say what you want:

Method	Means	Example
GET	Read something	Get the latest reading
POST	Create or send something new	Send a new reading
PUT	Update something	Change a device's settings
DELETE	Remove something	Delete a device

The server replies with a **status code** telling you how it went: **200** means success, **400** means your request was wrong (ThingSpeak uses it for a bad API key; many other services use **401** or **403** for that), **404** means "not found", and **500** means the server itself had a problem.

Strengths: HTTP is everywhere, easy to understand, easy to test in a browser, and works with almost every cloud service.

Weaknesses: every message carries quite a lot of extra text (headers), and the device has to ask each time — the server can't easily push a message to the device. For tiny devices sending frequent readings, it's a bit heavy.

> **NOTE**
>
> You'll use HTTP in **Lab 4**, to send readings to the ThingSpeak cloud. If the API key is wrong, you'll see the server reply with status code **400** — HTTP telling you exactly what went wrong.

MQTT: publish and subscribe

MQTT (originally "MQ Telemetry Transport", after an IBM product called MQ, but today just "MQTT") is a protocol built specifically for IoT. It's lightweight, works well on slow or unreliable connections, and is the most widely used messaging protocol in IoT.

Instead of request and response, MQTT uses **publish/subscribe**. There are three roles:

- A **broker** — a central server that receives every message and passes it on. Think of it as a **post office**.
- **Publishers** — devices that send messages. Each message is labelled with a **topic**, like an address on an envelope.
- **Subscribers** — anything that has asked to receive messages on a particular topic.

Here's the important part: **publishers and subscribers never talk to each other directly, and don't even need to know the other exists.** They only talk to the broker.

A good everyday comparison is a **radio station**. The station broadcasts on a frequency (the topic). Anyone who tunes in (subscribes) hears it. The station doesn't know or care how many listeners there are.

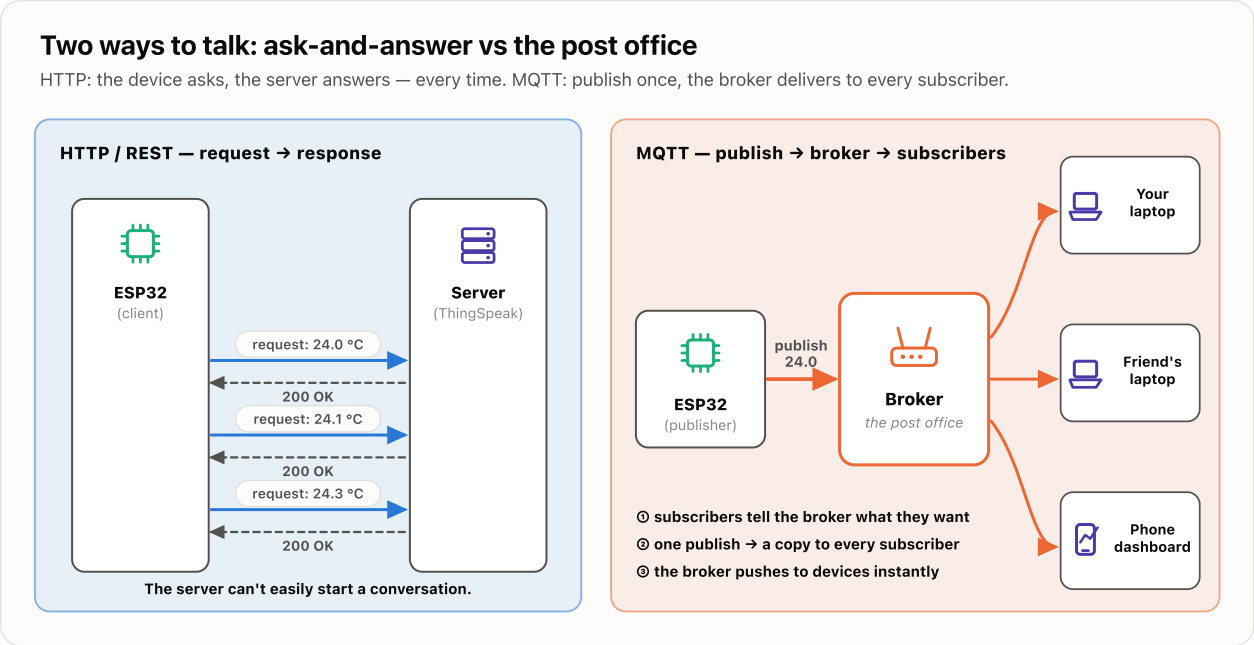


Figure 4.4 — HTTP is ask-and-answer; MQTT is a post office that delivers one message to every subscriber.

Topics

A **topic** is a text label organised like folders, using `/` to separate levels:

```
iot-workshop/aisha/temperature
iot-workshop/aisha/humidity
iot-workshop/ravi/temperature
```

Subscribers can use **wildcards** to listen to many topics at once:

- `+` matches exactly one level: `iot-workshop/+/temperature` gets *everyone's* temperature.
- `#` matches everything below: `iot-workshop/#` gets *every* message under that topic. A trainer — or you — could use this to see everyone's data on one screen.

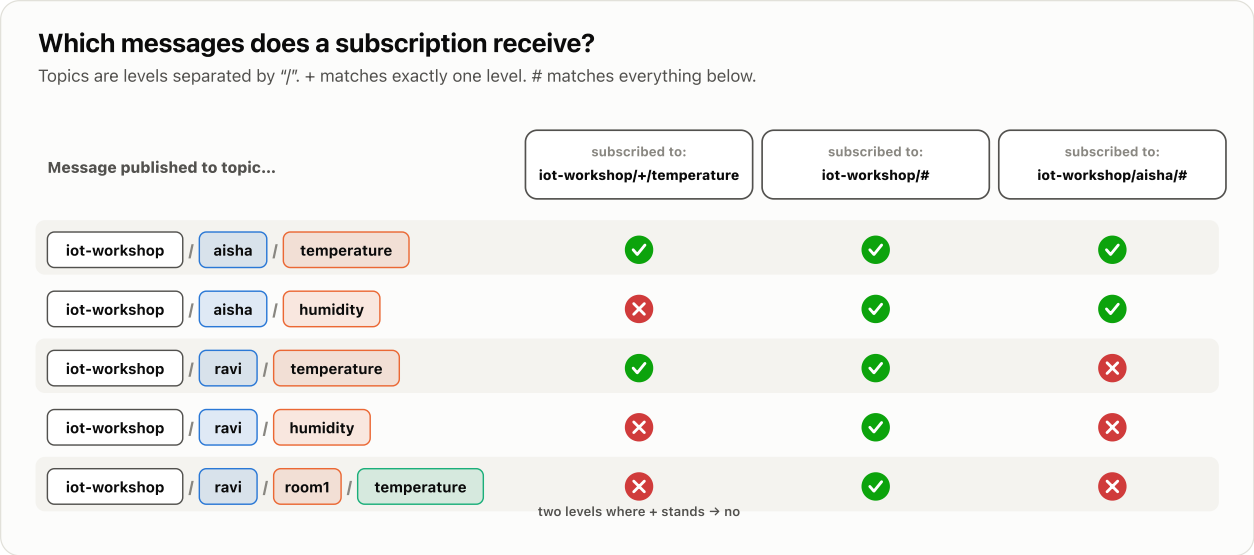


Figure 4.5 — how the + and # wildcards match topics. Note that + stands for exactly one level.

A few more MQTT features worth knowing

- **Quality of Service (QoS)** lets you choose how hard MQTT should try to deliver each message:

QoS	Promise	Like…
0	At most once — sent once, no confirmation	Shouting across a room
1	At least once — confirmed, but may arrive twice	A tracked parcel
2	Exactly once — a careful four-step handshake	Signed-for courier

Higher QoS is more reliable but slower and uses more data. Many sensor readings use QoS 0 or 1.

- **Retained messages.** The broker can keep the *last* message on a topic and hand it to anyone who subscribes later — so a new dashboard instantly shows the latest temperature, instead of waiting for the next reading.
- **Last Will and Testament.** A device can leave a message with the broker in advance ("I'm offline"). If the device drops off unexpectedly, the broker publishes it on the device's behalf — a neat way to spot dead devices.

MQTT has two main versions in use: **3.1.1** (made an official standard in 2014) and **5.0** (2019), which adds extra features. Both work the same way at the basic level you'll use in the labs.

Strengths: very small messages, a constant connection so the broker can push messages to devices instantly, one message can reach many subscribers, and it copes well with poor networks.

Weaknesses: you need a broker, and it's not as familiar or browser-friendly as HTTP.

CoAP: HTTP for the tiniest devices

CoAP (Constrained Application Protocol) is like a slimmed-down HTTP, designed for the smallest, lowest-power devices on weak networks. It uses the same request–response idea and similar methods (GET, POST, PUT, DELETE), but it runs on **UDP** instead of TCP, and its messages are much smaller. It uses port **5683**.

You won't use CoAP in the labs. Just remember: *CoAP is for very constrained devices that want web-style requests without the weight of HTTP.*

Choosing a protocol

	HTTP/REST	MQTT	CoAP
Pattern	Request–response	Publish–subscribe (via a broker)	Request–response
Runs on	TCP	TCP	UDP
Message size	Larger	Very small	Very small
Server can push to device?	Not easily	Yes, instantly	Yes, with "observe"
One message to many receivers?	No	Yes	Limited
Best for	Occasional uploads, talking to web services	Frequent readings, live updates, commands to devices	Tiny battery devices on constrained networks
In this book's labs	Lab 4 (ThingSpeak)	Labs 3 and 6	Not used

A useful rule of thumb: **use HTTP when a device occasionally reports to a web service; use MQTT when many devices exchange frequent, live messages.**

Security considerations when devices communicate

Every message a device sends travels through networks you don't control. That creates risks. Chapter 7 covers security properly, but here are the essentials to keep in mind right now:

- **Eavesdropping.** Anyone who can see the network traffic can read unencrypted data. **Encryption** (using TLS — the "S" in HTTPS) scrambles data so only the intended receiver can read it. For MQTT, that means using port **8883** instead of 1883.
- **Impersonation.** Without **authentication** — proof of who you are — a stranger could pretend to be your device and send fake readings, or pretend to be your server and send fake commands. Real systems give each device its own username and password, or better, its own digital certificate.

- **Unauthorised access.** Anyone who knows an unprotected topic or URL can read it or write to it. Real systems control **who may publish or subscribe to which topics**.
- **Leaked secrets.** Wi-Fi passwords and API keys written into code can leak — for example, if the code is shared online.

> **WARNING**
>
> **The labs take deliberate shortcuts.** In Lab 3, you'll publish to a **public** test broker, `broker.hivemq.com`, on the **unencrypted** port 1883, with **no password**. That means **anyone in the world can read your messages** if they know your topic — and anyone could publish fake data to it. That's fine for harmless, simulated temperature readings in a learning exercise, and it keeps the lab simple. **Never** do this with anything real, private, or important. Public test brokers are for learning only, with no guarantees.

Common misconceptions

"Wi-Fi is always the best choice because it's fastest." Speed isn't everything. Wi-Fi drains batteries and needs a router nearby. For a battery sensor in a field, a "slow" technology like LoRaWAN is the far better choice.

"Bluetooth and Wi-Fi do the same job." They overlap, but BLE is designed for very low power over short distances, usually with a phone nearby. Wi-Fi is designed for more data and direct internet access.

"MQTT devices send messages directly to each other." They don't. Every message goes through the **broker**. Publishers and subscribers only ever talk to the broker, never to each other.

"If my data is on the internet, it's automatically secure." No. Unless you use encryption and authentication, data can be read or faked on its way. The internet is more like a public road than a private tunnel.

Real-world example: a smart water meter network

A water company wants to read 200,000 household water meters automatically. Here's how the choices in this chapter play out:

- **Connectivity:** meters are often in basements or underground pits, run on batteries that must last over 10 years, and send one small reading a day. Wi-Fi is out (no power, poor reach). **NB-IoT** (excellent coverage inside buildings, no gateways needed) or **LoRaWAN** (with the company's own gateways) are the natural choices.
- **Protocol:** each meter sends a tiny message once a day, so a very lightweight protocol (such as MQTT, CoAP, or the network's own compact format) keeps the battery going.

- **Security:** readings are encrypted, and every meter has its own identity, so nobody can fake a neighbour's reading.

Key takeaways

- Getting online means two separate decisions: **how to connect** (the radio) and **what language to speak** (the protocol).
- Key networking ideas: **IP addresses**, **DNS**, **routers and gateways**, **clients and servers**, **ports**, **packets**, **TCP** vs **UDP**, **bandwidth**, and **latency**.
- Every wireless technology trades off **range, power, and data rate**:
- **Wi-Fi:** lots of data, high power.
- **BLE:** very low power, short range.
- **Zigbee/Thread:** low-power mesh for buildings.
- **LoRaWAN:** kilometres of range, tiny data, years on a battery.
- **Cellular IoT (NB-IoT, LTE-M, 5G):** works anywhere with mobile coverage.
- **HTTP/REST** is request–response, and great for occasional uploads to web services.
- **MQTT** is publish–subscribe through a **broker**, organised by **topics**. It's great for frequent, live messages.
- **CoAP** is a lightweight, UDP-based cousin of HTTP for the tiniest devices.
- Real systems need **encryption** and **authentication**. The public-broker lab is for learning only.

Check your understanding

1. What's the difference between a connectivity technology and a protocol? Give one example of each.

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

2. Which three things does every wireless technology trade off against each other?

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

3. A farm wants soil sensors spread over 5 km, running for years on batteries, sending one reading every 30 minutes. Which technology fits best, and why not Wi-Fi?

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

4. In MQTT, what does the broker do? Do publishers and subscribers talk directly?

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

5. You subscribe to `iot-workshop/+/humidity`. Which of these do you receive: (a) `iot-workshop/lee/humidity`, (b) `iot-workshop/lee/temperature`, (c) `iot-workshop/lee/room1/humidity`?

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

6. Your ESP32 sends a reading to a server over HTTP and gets back status code 400. What does that tell you?

→ *Answer A8 is in the answer key at the end of this chapter. Try it yourself first!*

7. Why is it risky to use a public MQTT broker on port 1883 for a real product?

→ *Answer A9 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
API	Application programming interface — a doorway that lets programs talk to each other.
Authentication	Proving who (or what) you are, e.g. with a password or certificate.
Bandwidth (data rate)	How much data can be sent per second.
BLE	Bluetooth Low Energy — a very-low-power, short-range radio.
Broker	The central MQTT server that receives messages and delivers them to subscribers.
Cellular IoT	IoT connectivity over mobile networks, such as NB-IoT, LTE-M and 5G RedCap.
Client / server	A client makes requests; a server listens and responds.
CoAP	Constrained Application Protocol — a lightweight, UDP-based request–response protocol for tiny devices.
DNS	Domain Name System — turns names like `broker.hivemq.com` into IP addresses.
Encryption	Scrambling data so only the intended receiver can read it.
HTTP / HTTPS	The request–response protocol of the web. HTTPS is the encrypted version.
IP address	A device's address on a network.
LAN	Local area network — a network in one place, such as your home Wi-Fi.
Last Will and Testament	An MQTT message a device leaves with the broker, which the broker sends for it if the device drops off unexpectedly.
Latency	How long a message takes to arrive.
LoRaWAN	A long-range, very-low-power wide-area network for small, infrequent messages.
Mesh network	A network where devices pass messages on for each other to extend coverage.
MQTT	A lightweight publish–subscribe messaging protocol widely used in IoT.

Term	Meaning
NB-IoT / LTE-M	Low-power versions of mobile (cellular) networks, designed for IoT.
Packet	A small chunk of data sent across a network.
Port	A number that identifies a particular service on a server.
Protocol	An agreed set of rules for how messages are formatted and exchanged.
Publish / subscribe	A messaging pattern where senders publish to topics, and receivers subscribe to topics.
QoS	Quality of Service — MQTT's setting for how hard to try to deliver a message (0, 1, or 2).
REST	A popular style of building web APIs using HTTP methods like GET and POST.
Retained message	The last message on an MQTT topic, kept by the broker and given to new subscribers straight away.
Router	A device that passes data between networks, such as your home network and the internet.
Status code	The number a server sends back to say how a request went, such as 200 (success) or 404 (not found).
TCP / UDP	Rules for sending packets: TCP is reliable and ordered; UDP is lighter with no guarantees.
TLS	Transport Layer Security — the standard way to encrypt internet connections.
Topic	The label on an MQTT message, organised in levels like `home/kitchen/temperature`.
Wildcard	A symbol in an MQTT subscription that matches several topics: + for exactly one level, # for everything below.
Zigbee / Thread	Low-power mesh radio technologies used in smart homes and buildings.

Into the lab

Time to go online. Open **Lab 3 — Getting Online: Wi-Fi + MQTT** (about 30 minutes).

You'll take your DHT22 setup from Lab 2 and add two things: a **Wi-Fi** connection (to the simulated `Wokwi-GUEST` network) and an **MQTT** client that publishes your temperature and humidity to the public broker `broker.hivemq.com` every 3 seconds.

Before you press Play, **change `"changeme"` on the `YOUR_NAME` line to something unique** — your first name is perfect. Everyone doing this lab, anywhere in the world, shares one public broker, and your name becomes part of your topic, so your data doesn't mix with anyone else's.

Then open the free HiveMQ web MQTT client in a second tab, **subscribe** to your topic, and watch your readings arrive live. Change the temperature on the simulated sensor, and see the new value appear in the other tab a moment later. That's the Internet of Things, working end to end.

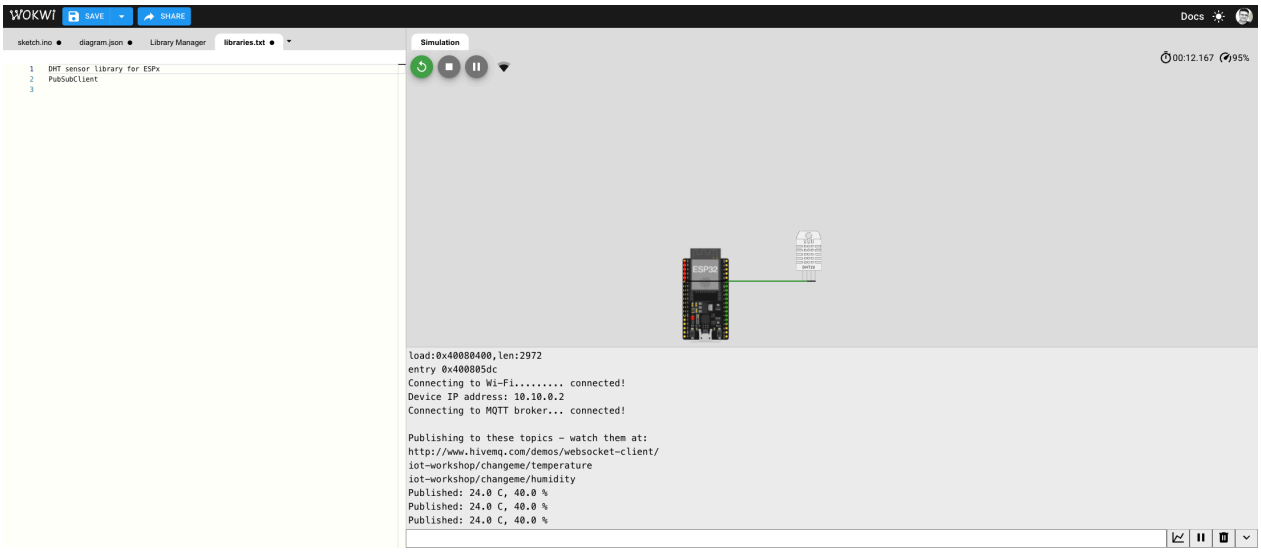


Figure 4.6 — Lab 3 running: the Serial Monitor shows the device joining Wi-Fi, connecting to the MQTT broker, and publishing a reading every 3 seconds.

As you work, spot the ideas from this chapter: the **IP address** your device is given, the **broker**, your **topics**, and the unencrypted **port 1883**.

Up next

In Chapter 5 — IoT Cloud Platforms and Data Visualisation, you'll give your data a home. You'll learn what IoT cloud platforms do, compare a few popular ones, and build a live graph of your readings in Lab 4 — using HTTP this time.

Going further (optional)

- HiveMQ — MQTT Essentials — a free, beginner-friendly series that goes deeper into topics, QoS, retained messages, and security.
- MDN — An overview of HTTP — a clear introduction to how HTTP works.
- The Things Network — What is LoRaWAN? — a community guide to long-range IoT networking.
- GSMA — LTE-M vs NB-IoT vs 3G/4G data rates — the mobile industry's own comparison of the two main cellular IoT options.

Answer key

A1 · Pause and check — Your ESP32 needs to send a reading to broker.hivemq.com. What turns that name into an address the network can use?

DNS (the domain name system) — the internet's phone book, which turns names into IP addresses.

A2 · Pause and check — Pick a technology for each: (a) a water-level sensor in a river 8 km from town, on a battery; (b) a security camera in an office; (c) a heart-rate chest strap that talks to a phone.

(a) **LoRaWAN** (or NB-IoT if there's good mobile coverage) — long range, tiny data, years on a battery. (b) **Wi-Fi** (or Ethernet) — lots of data, mains power, indoors. (c) **Bluetooth LE** — very low power, short distance to a nearby phone.

A3 · Question 1 — What's the difference between a connectivity technology and a protocol? Give one example of each.

A **connectivity technology** is *how* the device physically connects — the road (e.g. Wi-Fi, BLE, LoRaWAN). A **protocol** is the agreed *language* for the messages (e.g. MQTT, HTTP). They're independent choices.

A4 · Question 2 — Which three things does every wireless technology trade off against each other?

Range, **power** use, and **data rate** (bandwidth). You can't maximise all three at once.

A5 · Question 3 — A farm wants soil sensors spread over 5 km, running for years on batteries, sending one reading every 30 minutes. Which technology fits best, and why not Wi-Fi?

LoRaWAN (or NB-IoT if mobile coverage is good). It's built for long range, tiny amounts of data, and years of battery life. Wi-Fi only reaches tens of metres and would drain the batteries quickly.

A6 · Question 4 — In MQTT, what does the broker do? Do publishers and subscribers talk directly?

The **broker** receives every published message and delivers a copy to every client subscribed to that topic. Publishers and subscribers **never** talk directly — they only talk to the broker.

A7 · Question 5 — You subscribe to iot-workshop/+/humidity. Which of these do you receive: (a) iot-workshop/lee/humidity, (b) iot-workshop/lee/temperature, (c) iot-workshop/lee/room1/humidity?

Only **(a)**. The `+` wildcard matches exactly **one** level. (b) is a different final level, and (c) has two levels where `+` stands.

A8 · Question 6 — Your ESP32 sends a reading to a server over HTTP and gets back status code 400. What does that tell you?

A **400** means the server thinks **your request was wrong** — for example, a missing or incorrect API key, or badly formatted data. (A 200 would mean success.)

A9 · Question 7 — Why is it risky to use a public MQTT broker on port 1883 for a real product?

Port 1883 is **unencrypted**, so anyone can read the messages; a public broker has **no authentication**, so anyone who knows the topic can read or publish fake messages; and public test brokers offer **no guarantees** of availability. Real products use an encrypted connection (port 8883), per-device authentication, and topic permissions.

CHAPTER 5

IoT Cloud Platforms and Data Visualisation

Your device can now send its readings across the internet. But where do they go, and how does anyone make sense of thousands of numbers? This chapter explains what the cloud does for IoT, compares popular IoT platforms, and shows how to design a dashboard people can actually understand. Then you'll build a live graph of your own data in Lab 4.

At a glance	
Reading time	About 20 minutes
Lab	Lab 4 — Cloud Dashboard with ThingSpeak
Before this	Chapter 4 — Getting Online: Connectivity and Communication Protocols, and Lab 3
You'll need	Your Wokwi account, and your free ThingSpeak account (see the setup guide)

What you'll learn

By the end of this chapter, you'll be able to:

1. **Explain** the role cloud computing plays in an IoT system.
2. **Describe** how IoT data is stored, and the different kinds of analysis done on it.
3. **Compare** four IoT platforms — ThingSpeak, Arduino Cloud, Blynk, and AWS IoT Core — and say which suits which job.
4. **Choose** the right kind of chart or widget for a piece of IoT data.
5. **Apply** simple rules for designing a clear, useful dashboard.
6. **Build** a cloud dashboard that shows your live sensor data.

Where we are

In Lab 3, your smart room monitor started sending live temperature and humidity readings over MQTT. You watched them arrive in a web client — but only while you were looking. When you closed the tab, those readings were gone.

That's the gap this chapter fills. Real IoT systems need somewhere to **keep** data, **make sense of it**, and **show it** in a way people understand. That's the job of the **cloud platform** and **application** layers from Chapter 2's five-layer diagram.

The big picture

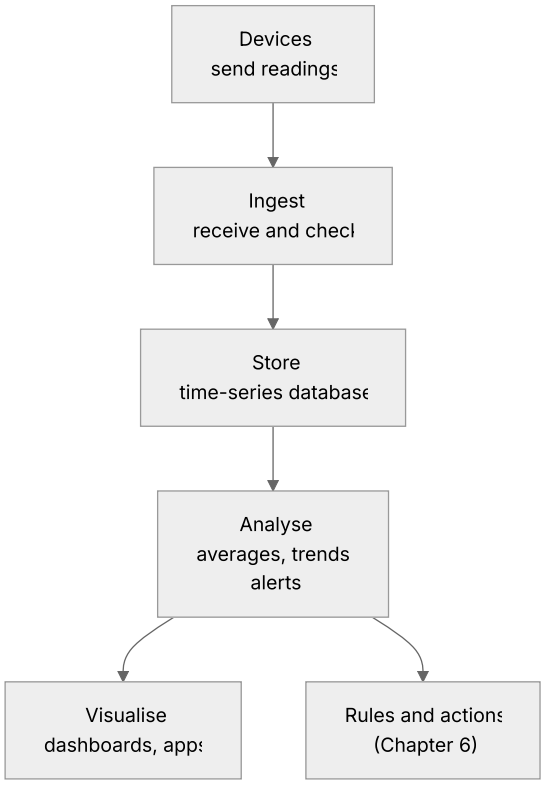


Figure 5.1 — data from devices is received (ingested), stored, and analysed in the cloud. From there it's shown on dashboards, and can also trigger rules and actions.

An IoT cloud platform takes care of every box after "Devices", so you don't have to build a database, a website, and a server yourself. In Lab 4, you'll send data to one — ThingSpeak — and within minutes you'll have a live, shareable graph.

What does the cloud do for IoT?

What is “the cloud”?

Cloud computing means renting computing power, storage, and software over the internet, from a provider who runs huge data centres, instead of buying and running your own servers. You pay for what you use, and you can grow from one device to a million without buying a single machine.

A helpful comparison is electricity. You don’t build your own power station to boil a kettle. You plug into the grid and pay for what you use. The cloud does the same for computing.

Why IoT and the cloud fit together

IoT devices are deliberately small and cheap. A microcontroller like the ESP32 has only a tiny amount of memory — nowhere near enough to store years of readings, analyse them, or serve a dashboard to hundreds of people. The cloud fills that gap. It provides:

What the cloud provides	Why it matters for IoT
Storage	Keeps history, so you can see trends, not just “right now”.
Computing power	Runs analysis far too heavy for a microcontroller.
Scale	Handles one device or a million, without you buying hardware.
Anywhere access	Lets you check your data from any phone or computer.
Integration	Connects IoT data to other systems — email, billing, maps, AI.
Device management	Keeps track of every device, its status, and its permissions.

What an IoT platform does

A general cloud provider offers raw building blocks. An **IoT platform** is a ready-made service, built on the cloud, specifically for device data. Most IoT platforms offer some or all of these features:

- **Device connection (ingestion):** a way for devices to send data in — usually over MQTT or HTTP, the protocols from Chapter 4.
- **Device management:** a list of registered devices, with their own credentials, status (online or offline), and settings.
- **Storage:** a database that keeps every reading with its time.
- **Processing and analytics:** tools to calculate averages, spot trends, or run custom code.
- **Rules and alerts:** “if this, then that” automation (you’ll learn about this in Chapter 6).
- **Visualisation:** ready-made charts, gauges, and dashboards.
- **APIs:** web addresses that let other software read or write your data.

Storing IoT data

Time-series data

Almost all sensor data has one thing in common: every value comes with a **time**. A list of values over time is called **time-series data**:

Time	Temperature (°C)	Humidity (%)
09:00:00	24.0	40.0
09:00:20	24.1	40.2
09:00:40	24.3	40.1

Without the time, a reading of 24.3 °C tells you very little. *With* the time, you can answer the questions people actually care about: Is it getting hotter? When does this room overheat? What was it like last night?

Because time-series data is so common, many IoT platforms use special **time-series databases**, built to store huge numbers of timestamped values and answer time-based questions quickly.

How much data is that?

IoT data adds up fast. One sensor sending a reading every 20 seconds produces about **4,300 readings a day**, and about **1.6 million a year**. Multiply that by thousands of devices, and storage becomes a real cost.

That's why platforms have **retention** limits (how long data is kept) and **message** limits (how many readings you can send). Real systems often keep detailed data for a short time, then keep only summaries — like hourly averages — for the long term.

Making sense of data: four kinds of analytics

Analytics means examining data to find useful information. It's helpful to think of four levels, each answering a harder question:

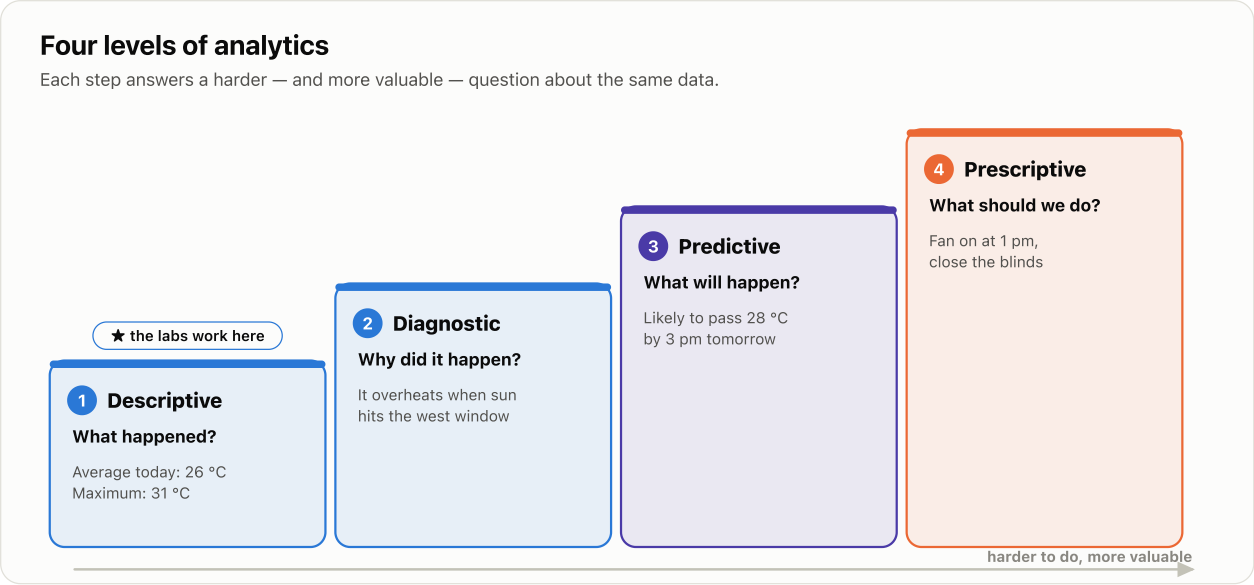


Figure 5.2 — four levels of analytics, applied to our room monitor. The labs in this book work on the first step.

Most beginner projects — including the labs in this book — work at the **descriptive** level: graphs of what happened. Big industrial systems use all four, often with **machine learning** (software that learns patterns from data) for the predictive and prescriptive levels. Predictive maintenance, which you met in Chapter 1, is predictive analytics in action.

> **TIP**
>
> **Pause and check.** A farm dashboard shows: "Soil in Field 2 will be too dry in about 6 hours." Which level of analytics is that?
>
> → *Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

Meet the platforms

There are many IoT platforms. This chapter introduces four, from the simplest to the most powerful. They all do the same basic job — receive, store, and show device data — but for different audiences and at different scales.

ThingSpeak — simple data logging and graphs

ThingSpeak is a free-to-start platform from MathWorks, the company behind the MATLAB analysis software. It's designed to make logging and graphing sensor data as simple as possible — which is why you'll use it in Lab 4.

In ThingSpeak, data is organised into:

- **Channels** — one channel per project or device, such as "My Room Monitor".
- **Fields** — up to 8 values per channel, such as Field 1 = Temperature and Field 2 = Humidity.

- **API keys** — secret codes that control access. The **Write API key** lets a device add data; the **Read API key** lets others read a private channel.

Devices send data with a simple HTTP request, and ThingSpeak draws a chart for each field automatically. It also includes MATLAB tools for deeper analysis, and simple rule features called “apps”.

Free plan (at the time of writing): about 3 million messages a year (roughly 8,200 a day), up to 4 channels, a minimum of **15 seconds** between updates to a channel, and **non-commercial use only**. Paid licences remove these limits.

Best for: learning, research, and simple data logging.

Arduino Cloud — for Arduino and ESP32 makers

Arduino Cloud (previously called Arduino IoT Cloud) is the online platform from Arduino, the company behind the popular hobby boards. It works with Arduino boards and ESP32 boards.

Its standout feature is how tightly it links the code and the cloud: you define **“Things”** and their **variables** (such as `temperature`) online, and Arduino Cloud helps generate the code to keep the device and the cloud in sync. You can also write and upload code from your browser, and build dashboards that work on phones.

Free plan (at the time of writing): 2 Things, 5 variables per Thing, and only 1 day of data history. Paid plans raise these limits.

Best for: makers, students, and schools already using Arduino.

Blynk — mobile apps without app development

Blynk focuses on building **phone apps** for your devices, without you having to learn mobile app development. You drag buttons, sliders, gauges, and charts onto a screen in the Blynk app, and link them to your device. It’s especially good for *controlling* devices from a phone, not just watching them.

Free plan (at the time of writing): up to 5 devices, 1 user, and a limited number of messages, with one week of data history.

Best for: phone-controlled projects and quick prototypes of commercial products.

AWS IoT Core — industrial scale

AWS IoT Core is part of Amazon Web Services (AWS), one of the largest cloud providers in the world. It’s built to connect **billions** of devices, and is used by large companies.

It includes a secure MQTT broker, a registry of devices, "device shadows" (a cloud copy of each device's latest state, even when the device is offline), and a rules engine that can send data on to dozens of other AWS services for storage, analytics, and machine learning. It's extremely powerful, but it takes much more setup and technical knowledge.

Pricing: pay-as-you-go — for example, about US$1 per million messages in its main region, plus small charges for connection time. There's no free "hobby" dashboard; you build your own from AWS building blocks.

Best for: professional products and large-scale deployments.

> **NOTE**
>
> Other large providers offer similar services — for example, Microsoft Azure IoT Hub. The concepts in this chapter — devices, messages, topics, storage, rules, dashboards — carry over to all of them.

Side by side

	ThingSpeak	Arduino Cloud	Blynk	AWS IoT Core
Aimed at	Learners, researchers	Makers, schools	App-style projects, prototypes	Businesses, large scale
Ease of starting	Very easy	Easy	Easy	Harder
Sends data using	HTTP (and MQTT)	Arduino's own library	Blynk's own library	MQTT, HTTP
Dashboards	Automatic charts	Web and phone dashboards	Phone app and web	Build your own
Strong point	Simplicity, MATLAB analysis	Code and cloud linked together	Phone control	Scale, security, integration
Free option	Yes, non-commercial	Yes, very limited	Yes, limited	Free-tier credits for new accounts

Free-plan details change often. Always check the provider's pricing page before you commit to a platform.

A word of caution: choosing a platform

Picking a platform is a long-term decision. Keep these in mind:

- **Lock-in.** Each platform has its own way of doing things. Moving thousands of devices to a new platform later can be painful. Using standard protocols like MQTT and HTTP makes switching easier.
- **Platforms can disappear.** Google shut down its IoT platform, *Google Cloud IoT Core*, in August 2023, and every customer had to move. It's a good reminder to plan for change.
- **Cost at scale.** A free plan for 5 devices tells you little about the bill for 50,000.
- **Where your data lives.** If you collect data about people — from cameras, wearables, or smart meters, for example — you must follow data protection laws, such as Malaysia's Personal Data Protection Act (PDPA). You'll learn more in Chapter 7.

Dashboards and data visualisation

A **dashboard** is a screen that shows the most important information at a glance — just like a car's dashboard shows speed and fuel, not the engine's internal data. **Data visualisation** means presenting data as pictures — charts, gauges, maps — so people can understand it quickly.

A good dashboard answers the viewer's questions in seconds. A bad one buries them in numbers.

Choosing the right chart

Different questions need different visuals:

If you want to show...	Use a...	Example
How something **changes over time**	**Line chart**	Temperature over the last 24 hours
The **current value** within a safe range	**Gauge** or big number	"Tank is 72% full"
A **comparison** between things	**Bar chart**	Energy used by each floor
On/off or good/bad **status**	**Status indicator** (with a label)	"Pump: RUNNING"
Where things are	**Map**	Locations of delivery lorries
Exact values to look up	**Table**	The last 10 readings, with times

For IoT sensor data, the **line chart** is the workhorse. Most of what you want to know about a sensor — is it rising, falling, steady, or spiking? — shows up best as a line over time. That's why ThingSpeak draws line charts by default.

Seven rules for a clear dashboard

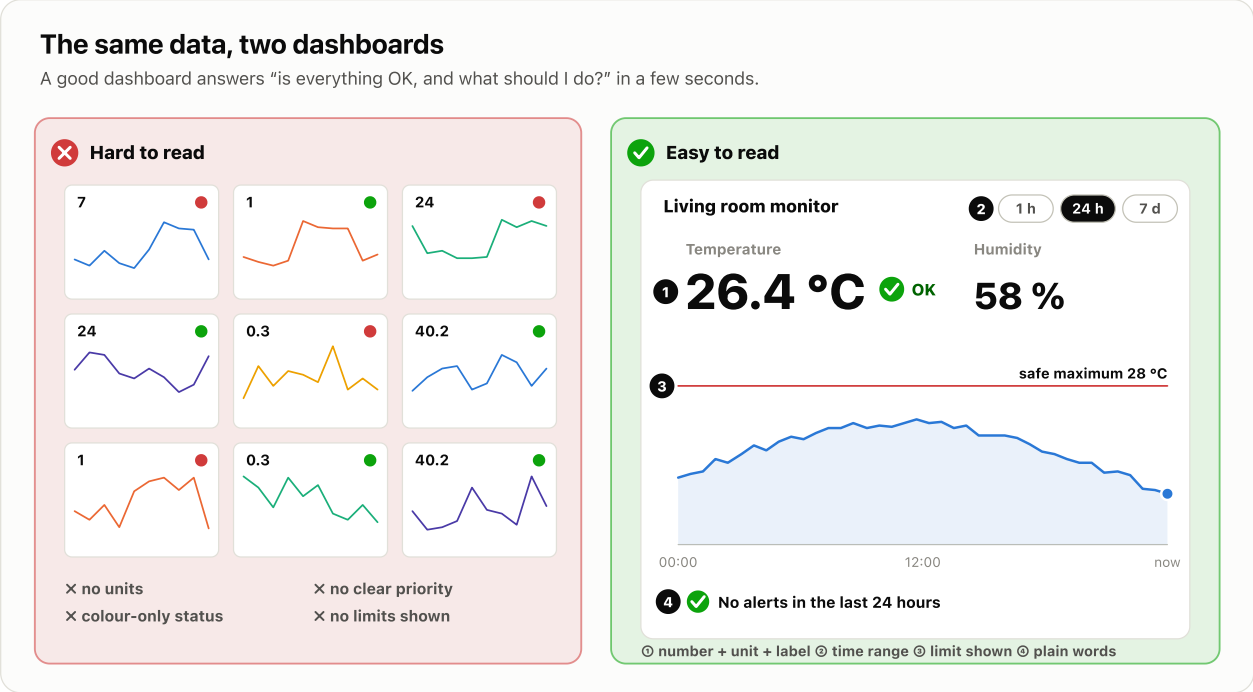


Figure 5.3 — the same data, two dashboards. Callout ① shows a big number with its unit and a word label (rules 2, 3 and 6), ② a sensible time range (rule 4), ③ the limit drawn on the chart (rule 5), and ④ a plain-language status (rule 1).

1. **Know your audience.** A farmer needs “Water Field 3 now”, not a table of raw numbers. An engineer might need both.
2. **Put the most important thing first**, and make it the biggest.
3. **Always show units** (°C, %, kWh) and clear labels. “24” on its own could mean anything.
4. **Choose a sensible time range.** The last hour for “right now” questions, the last week for patterns.
5. **Show limits and targets.** A line marking “safe maximum: 28 °C” makes a chart instantly meaningful.
6. **Don’t rely on colour alone.** About 1 in 12 men and 1 in 200 women have some colour blindness. Pair red and green with words or icons, like “⚠ HIGH” or “ ✓ OK”.
7. **Less is more.** Every extra chart competes for attention. If nobody acts on a chart, remove it.

TIP

Pause and check. A hospital wants a screen showing whether each of 20 medicine fridges is currently in its safe range. Which visual would you choose, and what would you avoid?

→ *Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

Common misconceptions

"The cloud is one big computer somewhere." It's a huge number of computers in data centres around the world, rented by the minute. Your data may be stored and copied across several of them.

"If it's in the cloud, it's automatically safe and permanent." Cloud providers are generally reliable, but free plans have retention limits, services can shut down, and a stolen API key can let someone else read or change your data. You still need to protect your data and plan for change.

"More charts make a better dashboard." Usually the opposite. The best dashboards show a few important things clearly, so the viewer knows what to do.

"All IoT platforms are basically the same." They share the same core ideas, but differ hugely in ease of use, scale, cost, and features. The right choice depends on who's using it and how big the project will get.

Real-world example: an energy dashboard for a school

A school installs smart meters and room sensors to cut its electricity bill.

- **Cloud storage:** every meter sends a reading every minute. Detailed data is kept for three months; hourly summaries are kept for years.
- **Analytics:** *descriptive* — "we used 4,200 kWh last month"; *diagnostic* — "the science block's air-conditioning runs all weekend"; *predictive* — "this month's bill will be about 10% higher".
- **Dashboard for the principal:** one big number (this month's cost vs last month), one line chart (daily use), and a short list of problem areas.
- **Dashboard for students:** a friendly screen in the lobby showing how much energy their school has saved, to encourage everyone to switch things off.

Same data, two different dashboards — each designed for its audience.

Key takeaways

- **Cloud computing** means renting computing power and storage over the internet. It gives tiny IoT devices the storage, processing, scale, and access they can't provide themselves.
- An **IoT platform** handles device connection, device management, storage, analytics, rules, visualisation, and APIs.
- IoT data is **time-series data**: values with timestamps. It adds up fast, so platforms limit **messages** and **retention**.

- Analytics comes in four levels: **descriptive, diagnostic, predictive, and prescriptive**.
- **ThingSpeak** is simple and great for learning; **Arduino Cloud** links code and cloud for makers; **Blynk** builds phone apps; **AWS IoT Core** works at industrial scale.
- Consider **lock-in, platform shutdowns, cost at scale, and data protection** when choosing a platform.
- Good dashboards use the **right chart**, show **units and limits**, suit their **audience**, and **don't rely on colour alone**.

Check your understanding

1. Name three things the cloud provides that a microcontroller like the ESP32 can't.

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

2. What is time-series data, and why is the timestamp so important?

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

3. In ThingSpeak, what's the difference between a channel and a field? And between the Write and Read API keys?

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

4. You want your family to switch a garden light on and off from their phones. Which platform from this chapter fits best?

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

5. Which chart type would you use to show how a room's humidity changed over the last week?

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

6. Give two risks to think about before choosing an IoT platform for a product.

→ *Answer A8 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
Analytics	Examining data to find useful information — descriptive, diagnostic, predictive, or prescriptive.
API key	A secret code that gives permission to read or write data through an API.
Channel (ThingSpeak)	A container for one project's data, holding up to 8 fields.

Term	Meaning
Cloud computing	Renting computing power, storage, and software over the internet.
Dashboard	A screen showing the most important information at a glance.
Data retention	How long a platform keeps your data.
Data visualisation	Presenting data as charts, gauges, maps, and other pictures.
Device management	Keeping track of devices, their credentials, status, and settings.
Device shadow	A cloud copy of a device's latest state (AWS IoT).
Field (ThingSpeak)	One value within a channel, such as temperature.
Ingestion	Receiving and accepting data from devices into a platform.
IoT platform	A cloud service built for connecting devices and handling their data.
Lock-in	Becoming so dependent on one provider that switching is hard.
Machine learning	Software that learns patterns from data to make predictions.
Time-series data	A sequence of values, each recorded with a timestamp.
Time-series database	A database built to store huge numbers of timestamped readings and answer time-based questions quickly.

Into the lab

Now give your data a home. Open **Lab 4 — Cloud Dashboard with ThingSpeak** (about 30 minutes).

It has two parts:

1. **Create your ThingSpeak channel.** Name it, switch on Field 1 (Temperature) and Field 2 (Humidity), save it, and copy your **Write API key**.
2. **Send data from the simulator.** Paste your Write API key into the code, then run it. Every 20 seconds, your ESP32 sends an **HTTP** request with your readings. Watch your channel's charts grow, point by point.

As you work, notice the ideas from this chapter:

- Your data becomes **time-series data** — each point has a time.
- The code waits **20 seconds** between updates because ThingSpeak's free plan needs at least 15.
- If the key is wrong, you'll see **HTTP code 400** — the cloud rejecting a request it can't authenticate.
- Your **Write API key** is a secret. Anyone who has it can write data to your channel.

Then try it: change the simulated temperature and watch your graph follow — from a simulated sensor, across the internet, into the cloud, and onto a chart.

Up next

In Chapter 6 — Data Processing and Automation, your device stops just reporting and starts **acting**. You'll learn how IoT systems make decisions by themselves, and make your ESP32 switch on a relay and a buzzer when it gets too hot — with no cloud needed.

Going further (optional)

- ThingSpeak documentation — channels, the REST API, and MATLAB analysis of your data.
- Arduino Cloud and its plans — try it with an ESP32.
- Blynk documentation — build a phone app for your device.
- AWS IoT Core — how it works — see how an industrial-scale platform is put together.

Answer key

A1 · Pause and check — A farm dashboard shows: "Soil in Field 2 will be too dry in about 6 hours." Which level of analytics is that?

Predictive — it's forecasting what will happen. If it added "so start irrigation at 2 pm", that would be **prescriptive**.

A2 · Pause and check — A hospital wants a screen showing whether each of 20 medicine fridges is currently in its safe range. Which visual would you choose, and what would you avoid?

A grid of **status indicators** (one per fridge), each with a clear label like "Fridge 4: OK" or "Fridge 7: TOO WARM", sorted so problems appear at the top. Avoid 20 separate line charts (too much to scan quickly) and avoid using colour alone to show a problem.

A3 · Question 1 — Name three things the cloud provides that a microcontroller like the ESP32 can't.

Any three of: long-term **storage** of history; heavy **computing power** for analysis; **scale** to many devices; **access from anywhere**; **integration** with other systems; and **device management**.

A4 · Question 2 — What is time-series data, and why is the timestamp so important?

Time-series data is a sequence of values, each with the **time** it was recorded. The timestamp lets you see **trends and patterns** — whether a value is rising or falling, when problems happen — which a single value can't tell you.

A5 · Question 3 — In ThingSpeak, what's the difference between a channel and a field? And between the Write and Read API keys?

A **channel** holds one project's data stream; a **field** is one value within it (up to 8 per channel), like temperature. The **Write API key** lets a device add data; the **Read API key** lets others read a private channel.

A6 · Question 4 — You want your family to switch a garden light on and off from their phones. Which platform from this chapter fits best?

Blynk, because it's designed for building simple phone apps with buttons that control devices. (Arduino Cloud's phone dashboards could also work.)

A7 · Question 5 — Which chart type would you use to show how a room's humidity changed over the last week?

A **line chart**, because it shows how a value changes over time.

A8 · Question 6 — Give two risks to think about before choosing an IoT platform for a product.

Any two of: **lock-in** (hard to move later); the platform **shutting down** (like Google Cloud IoT Core in 2023); **costs** growing at scale; and **data protection** and privacy rules for personal data.

CHAPTER 6

Data Processing and Automation

Collecting data is only half the story. The real power of IoT comes when a system acts on that data by itself — switching on a fan, sounding an alarm, or sending an alert — without anyone pressing a button. This chapter shows you how.

At a glance	
Reading time	About 20 minutes
Lab	Lab 5 — Automation: Threshold-Triggered Relay & Buzzer
Before this	Chapter 5 — IoT Cloud Platforms and Data Visualisation
You'll need	Your Wokwi account, and the DHT22 skills from Lab 2

What you'll learn

By the end of this chapter, you'll be able to:

1. **Describe** the typical steps data goes through, from raw reading to useful decision.
2. **Explain** what "event-driven" means, and why IoT systems are built that way.
3. **Write** simple automation rules in plain "if this, then that" form.
4. **Explain** what a rules engine is and where it runs.
5. **Compare** automation at the edge with automation in the cloud, and choose between them.
6. **Build** a device that reacts to a sensor threshold on its own.

Where we are

So far, your device can **sense** (Labs 1 and 2), **send** its data over the internet (Lab 3), and show it on a **cloud dashboard** (Lab 4).

That covers the first two steps of the IoT loop from Chapter 1: *sense* and *send*. But a dashboard still needs a person to look at it and decide what to do. What if nobody's watching when the server room overheats at 3 am?

This chapter is about the last two steps: **decide** and **act** — automatically. By the end of Lab 5, your smart room monitor will finally switch on its own fan.

The big picture

Here's the whole idea in one picture:

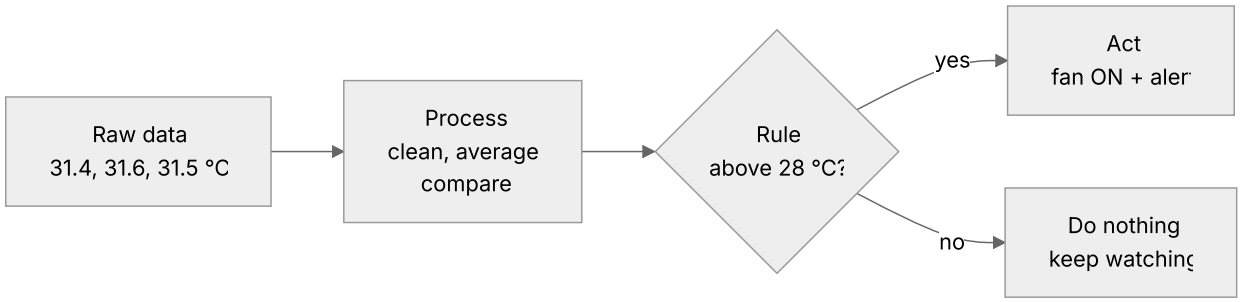


Figure 6.1 — raw readings are processed, compared against a rule, and either trigger an action or are simply recorded.

Three ideas make this work, and we'll take them one at a time:

1. **Processing** — turning a stream of raw numbers into something trustworthy.
2. **Events and rules** — noticing when something important happens, and deciding what to do about it.
3. **Where the decision happens** — on the device (the **edge**) or in the **cloud**.

From raw data to useful information

Raw sensor data is rarely ready to use as it is. Before it can drive a decision, it usually passes through a series of steps. This series is called a **data processing workflow** (you'll also hear **data pipeline**, the same word we used in Chapter 2).

A typical workflow

Step	What happens	Room monitor example
1. Collect	Read the sensor at a sensible rate	Read the DHT22 every 2 seconds
2. Validate	Throw away readings that can't be real	Ignore a reading of −40 °C or "not a number"
3. Clean / smooth	Reduce noise and random wobble	Average the last 5 readings
4. Transform	Convert to useful units or combine values	Work out "feels like" temperature from temperature + humidity
5. Aggregate	Summarise over time	Hourly average, daily maximum
6. Decide	Compare against rules	"Is the average above 28 °C?"
7. Store / act / send	Save it, act on it, or pass it on	Switch the fan, log the event, send an alert

Not every system uses every step, and the order can vary. But almost every real IoT system does some version of *collect, check, clean, decide*.

Why cleaning matters

Imagine a single faulty reading of 85 °C arrives, caused by a loose wire. Without validation, the system might sound a fire alarm and call the building manager at midnight. With a simple rule — *"ignore any reading more than 10 °C away from the last one"* — the glitch is quietly thrown out.

Smoothing helps in a similar way. If the temperature is sitting right around the limit, individual readings might jump above and below it: 27.9, 28.1, 27.8, 28.2. Averaging a few readings gives a steadier picture before any decision is made.

> **TIP**
>
> **Pause and check.** Your sensor reports 24.1, 24.3, 24.2, **97.0**, 24.2 °C. Which processing step should deal with the 97.0?
>
> *→ Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

Event-driven thinking

Readings vs events

There are two ways a system can handle data:

- **Polling** means checking again and again on a schedule: *"What's the temperature now? And now? And now?"* Every reading is treated the same.
- **Event-driven** means reacting when **something meaningful happens**: *"Tell me when the temperature goes above 28 °C."*

An **event** is a change that matters. "The temperature is 24 °C" is a reading. "The temperature just crossed 28 °C" is an event. So are "the door opened", "motion detected", "battery below 10%", and "device went offline".

Think of the difference between a child asking "are we there yet?" every five minutes (polling) and a driver who simply announces "we've arrived!" (an event).

Why IoT systems are event-driven

An **event-driven architecture** is a way of designing a system where parts react to events as they happen, rather than constantly checking on each other. It suits IoT very well:

- **It saves resources.** Sending "nothing changed" 1,000 times a day wastes battery, bandwidth, and money. Sending one message when something *does* change is far cheaper.
- **It's fast.** Things react the moment an event happens, rather than at the next scheduled check.
- **It scales.** One event can trigger many reactions — the fan, an alert, a log entry, and a dashboard update — without the device needing to know about any of them.

That last point should sound familiar. It's exactly how **MQTT** works, from Chapter 4: a device *publishes* an event to a topic, and any number of subscribers react to it. MQTT is a natural fit for event-driven IoT.

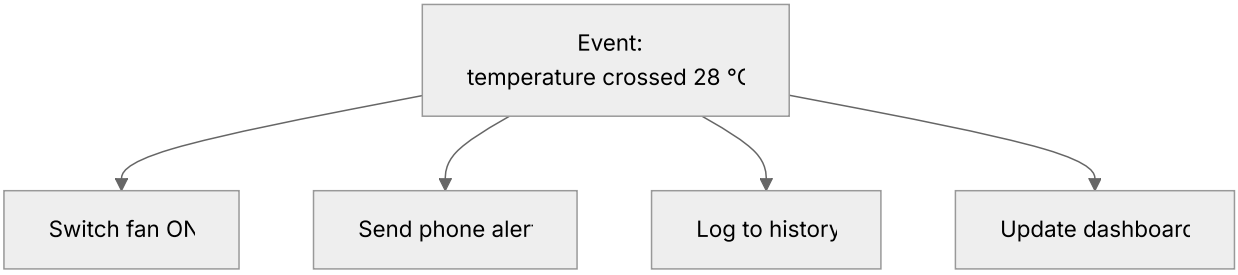


Figure 6.2 — one event fans out to four independent reactions — the fan, an alert, a log entry, and a dashboard update.

You'll see this in Lab 5

Lab 5's code reads the temperature every 2 seconds (that part is polling — sensors usually have to be polled). But look closely and you'll see a variable called `alarmIsOn`. The code uses it to print `THRESHOLD CROSSED` **only once, at the moment of change**, instead of repeating it

every loop. That's the code turning a stream of readings into an **event**.

Rules and rules engines

Rules in plain English

An **automation rule** is an instruction of the form:

> **IF** *(a condition is true)* **THEN** *(do an action)*

Some examples:

- IF temperature > 28 °C THEN switch the fan on.
- IF motion is detected AND it's after 10 pm THEN switch on the porch light.
- IF soil moisture < 20% THEN open the irrigation valve for 5 minutes.
- IF no data has arrived for 15 minutes THEN email the technician.

Notice how you can combine conditions with **AND** and **OR**, and include things other than sensor values — like the time of day, or whether a device has gone quiet.

The popular consumer service **IFTTT** is literally named after this pattern: "If This Then That".

What's a rules engine?

When you have a handful of rules, you can write them directly in the device's code, as you will in Lab 5. But imagine a factory with 5,000 sensors and 300 rules that managers want to change every week. Editing and re-uploading code for every change would be slow and risky.

A **rules engine** is software that stores and runs many rules for you, usually with a friendly screen where you can add or change rules without writing code. It watches incoming data, checks every rule, and fires the right actions.

You'll find rules engines in most IoT platforms:

Platform	Its rule feature (in plain terms)
ThingSpeak	"React" — run an action when a channel's value meets a condition
Blynk	"Automations" — if-then rules built in the app
AWS IoT Core	"Rules" — route and act on device messages as they arrive
Node-RED	A free, visual tool where you build rules by connecting boxes with wires
Home Assistant	A free smart-home hub with automations for home devices

The *idea* is always the same as the `if` statement in Lab 5 — only the place it runs and how you edit it change.

A classic rule problem: flickering

Here's a problem that catches out almost everyone who writes their first automation rule.

Say your rule is *IF temperature > 28 °C THEN fan on, ELSE fan off*. The room is sitting at almost exactly 28 °C, so the readings go: 28.1 (fan on), 27.9 (fan off), 28.1 (fan on), 27.9 (fan off)... The fan clicks on and off every few seconds. That's annoying, and it wears out the relay.

The standard fix is called **hysteresis**: use **two** limits instead of one.

- Switch the fan **on** when it goes **above 28 °C**.
- Only switch it **off** again when it drops **below 26 °C**.

The 2-degree gap in the middle is a "do nothing" zone that stops the flickering. Your home air-conditioner and fridge work this way. It's a great improvement to try once you've finished Lab 5.

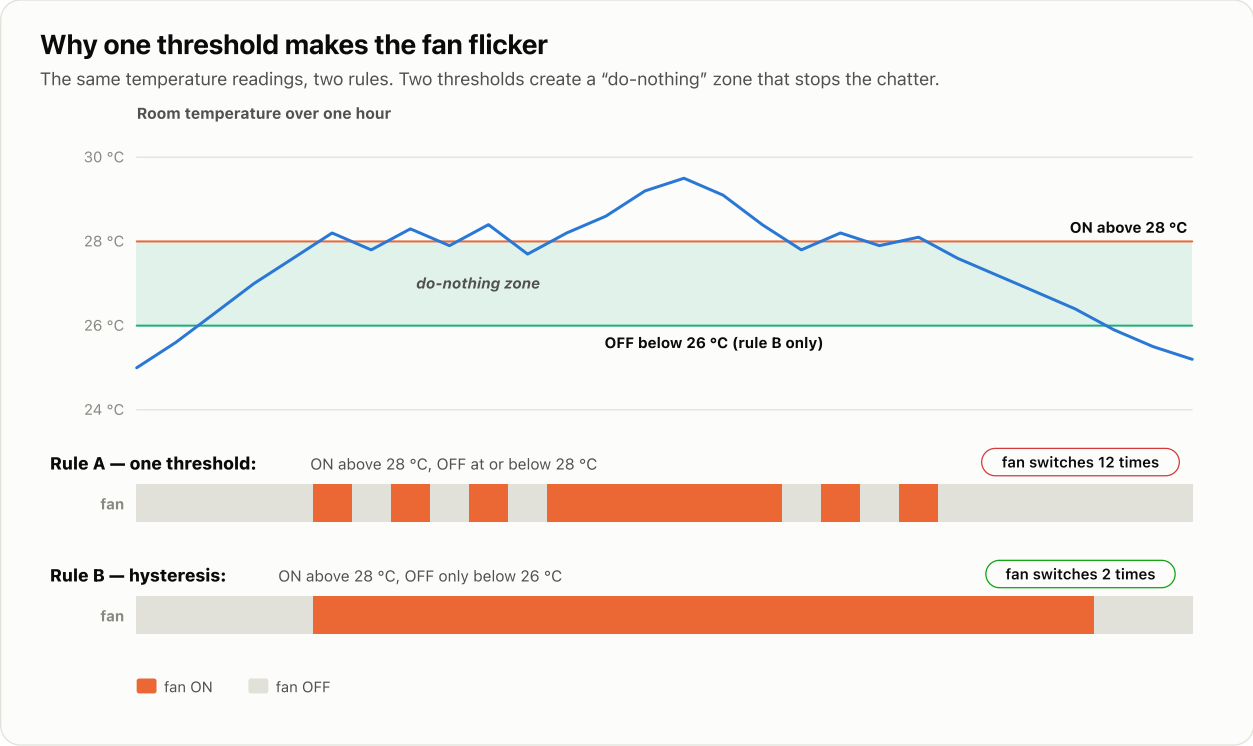


Figure 6.3 — the same readings under two rules. Hysteresis turns 12 annoying switches into 2.

> **TIP**
>
> **Pause and check.** Write a rule, in plain English, that sends an alert if a freezer's temperature rises above −10 °C, but only if it stays that warm for more than 10 minutes (so opening the door briefly doesn't trigger it).
>
> → *Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

Where should the decision happen? Edge vs cloud

In Chapter 2, you met **edge computing**: processing data and making decisions close to where the data is collected, instead of in a distant cloud. Now you can see why it matters.

The *same rule* — "IF temperature > 28 °C THEN fan on" — can run in two different places:

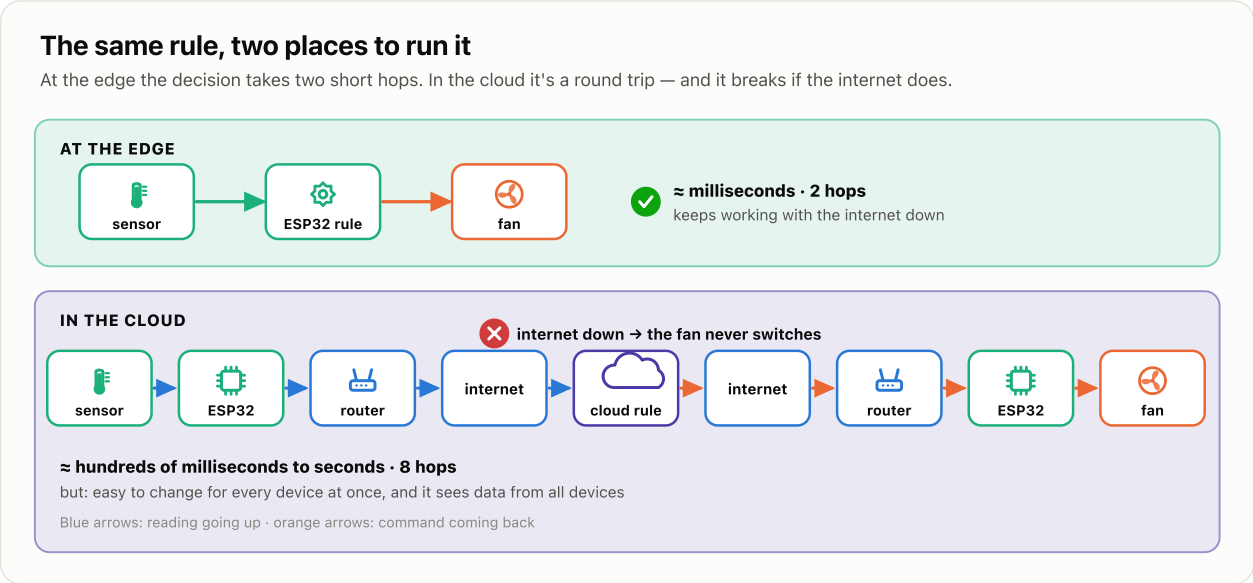


Figure 6.4 — the same rule running in two places. The edge is fast and works offline; the cloud sees every device.

Edge automation

Pros: - **Fast.** The decision takes a fraction of a second — no trip across the internet. - **Keeps working offline.** If the Wi-Fi drops, the fan still switches on. This can be critical for safety. - **Saves data and money.** The device doesn't need to send every reading. - **More private.** Sensitive data, like camera footage, can stay on the device.

Cons: - **Harder to change.** Updating the rule means updating every device's software. - **Limited view.** Each device only knows about itself. - **Limited power.** A microcontroller can't run very heavy processing.

Cloud automation

Pros: - **Easy to change.** Edit a rule once, in a web page, and it applies to every device straight away. - **The big picture.** The cloud sees data from *all* devices, so it can make smarter decisions: “three rooms are hot at once — maybe the main air-conditioning has failed.” - **Lots of computing power** for heavy analysis, history, and machine learning.

Cons: - **Slower.** Every decision needs a round trip over the internet. - **Stops if the connection drops.** No internet, no decision. - **Ongoing costs** for sending and processing all that data.

Choosing — and using both

Here’s a simple way to decide:

If the decision…	Put it…
Is urgent or safety-critical	At the **edge**
Must keep working without internet	At the **edge**
Needs data from many devices	In the **cloud**
Changes often, or is set by non-technical staff	In the **cloud**
Needs heavy analysis or long history	In the **cloud**

In practice, most good systems use **both**. The device handles urgent, simple rules locally, and the cloud handles the big-picture, long-term ones. For our room monitor: the ESP32 switches the fan the instant it gets hot (edge), while the cloud notices the room has overheated every afternoon this week and emails you a suggestion to close the blinds (cloud).

> **NOTE**
>
> Edge computing isn’t only for tiny microcontrollers. It also includes **gateways** and small local servers — for example, a gateway in a factory that analyses video from cameras on-site, rather than streaming it all to the cloud.

Common misconceptions

“Automation means artificial intelligence.” Most IoT automation is just clear, simple if-then rules — like the one you’ll write in Lab 5. AI and machine learning can add smarter predictions, but they’re built on top of these basics, not a replacement for them.

“Edge and cloud are competitors — you have to pick one.” They’re partners. Most well-designed systems split the work: urgent and local decisions at the edge, big-picture decisions in the cloud.

“Sending every reading to the cloud is the safest option.” It can actually be worse: more data costs more, drains batteries, creates more privacy risk, and still fails if the connection drops. Sending meaningful events and summaries is often smarter.

“One threshold is enough for a good rule.” A single threshold often causes flickering when the value hovers near it. Using **hysteresis** (two thresholds) or a time delay makes rules far more reliable.

Real-world example: a cold-chain vaccine fridge

Some vaccines must be kept between 2 °C and 8 °C at all times, including while being transported and stored. Many clinics and pharmacies monitor their fridges with IoT sensors. Here’s how this chapter’s ideas show up:

- **Processing:** readings every minute are validated (glitches thrown out) and averaged over five minutes, so opening the door briefly doesn’t cause a false alarm.
- **Events:** the system reacts to *changes that matter* — “left the safe range”, “power failed”, “device offline” — not to every normal reading.
- **Edge rules:** the device sounds a local buzzer immediately if the temperature leaves the safe range, even if the internet is down.
- **Cloud rules:** the cloud texts the pharmacist on duty, escalates to a manager if nobody responds in 15 minutes, and keeps a complete temperature record that inspectors can check.

That’s edge and cloud automation working together to protect something valuable.

Key takeaways

- Raw data usually goes through a **workflow**: collect, validate, clean, transform, aggregate, decide, and act or store.
- An **event** is a meaningful change. **Event-driven** systems react to events instead of constantly checking — saving power, data, and time.
- **Automation rules** follow the pattern **IF** condition **THEN** action, and can combine conditions with AND/OR.
- A **rules engine** stores and runs many rules, usually through a friendly interface, so they can be changed without rewriting code.
- **Hysteresis** (two thresholds) or time delays stop rules from flickering on and off.
- **Edge** automation is fast and works offline. **Cloud** automation is easy to change and sees the big picture. Most real systems use **both**.

Check your understanding

1. What's the difference between a reading and an event? Give an example of each.

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

2. Name two benefits of an event-driven design for a battery-powered sensor.

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

3. A greenhouse fan keeps switching on and off every few seconds when the temperature is near its limit. What's the problem called, and how do you fix it?

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

4. A gas-leak detector must shut a valve instantly, even if the internet is down. Edge or cloud? Why?

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

5. What is a rules engine, and why would a large company use one instead of writing rules into device code?

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

6. Give one decision in a smart home that belongs in the cloud rather than on a device.

→ *Answer A8 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
Aggregate	To summarise many readings into one value, like an hourly average.
Automation rule	An instruction of the form IF condition THEN action, run automatically.
Cloud automation	Rules that run on cloud servers, using data sent from devices.
Data processing workflow	The series of steps that turns raw readings into useful information.
Edge automation	Rules that run on the device or a nearby gateway, without needing the cloud.
Event	A meaningful change, such as a value crossing a limit or a door opening.
Event-driven architecture	A design where parts of a system react to events as they happen.
Hysteresis	Using two thresholds (an "on" point and an "off" point) to stop a rule flickering.
Polling	Checking a value again and again on a fixed schedule.

Term	Meaning
Rules engine	Software that stores and runs many automation rules.
Threshold	The limit a value is compared against in a rule, like 28 °C.
Validation	Checking that a reading is realistic, and discarding it if not.

Into the lab

Now build it. Open **Lab 5 — Automation: Threshold-Triggered Relay & Buzzer** (about 30 minutes).

You'll wire a DHT22 sensor, a relay (with an LED standing in for a real appliance), and a buzzer to your ESP32. The code has one rule: **if the temperature goes above 28 °C, switch on the relay and the buzzer.** While the simulation runs, you'll drag the temperature slider past the limit and watch the device react on its own.

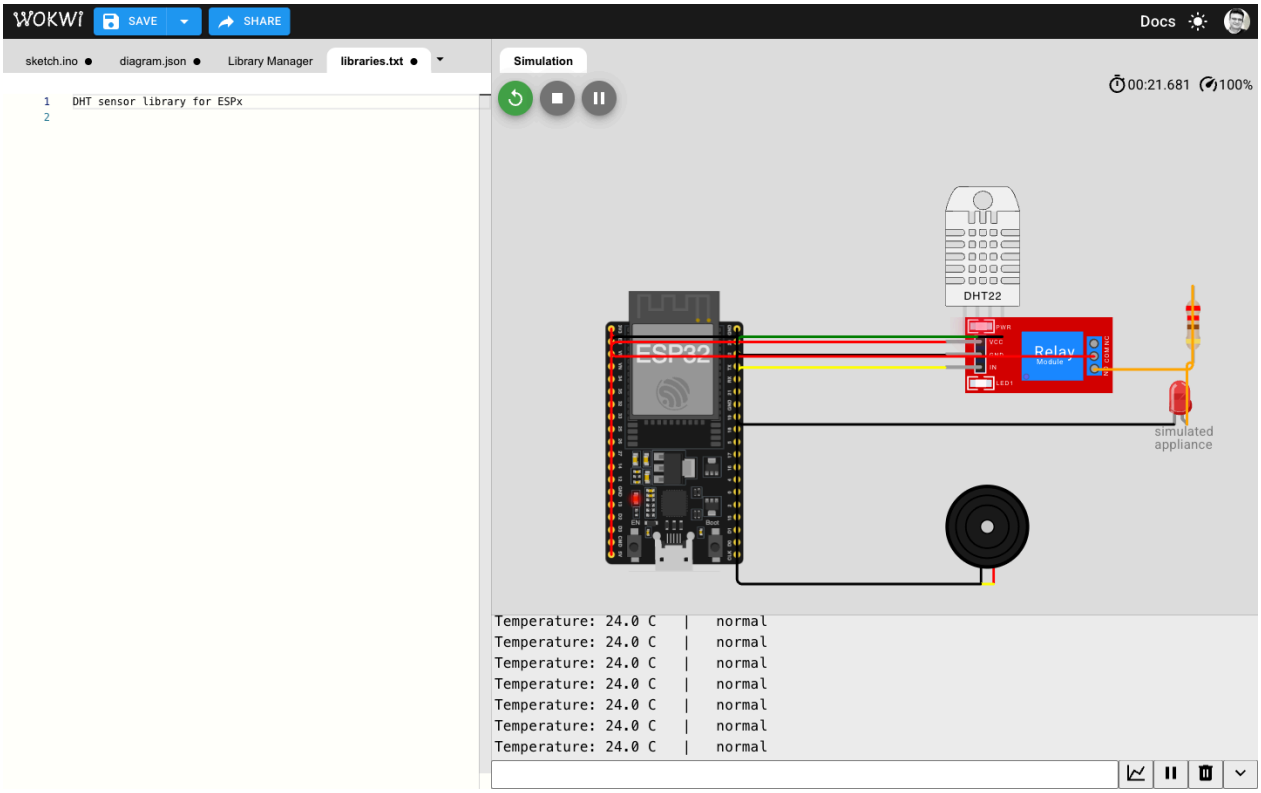


Figure 6.5 — Lab 5 running. Raise the DHT22's temperature above 28 °C and the relay, LED and buzzer switch on by themselves.

As you work, notice:

- The whole rule is one line: `TEMPERATURE_THRESHOLD_C = 28.0`.
- The `alarmIsOn` variable turns a stream of readings into a single **event** message.

- The code never touches Wi-Fi. This is **edge automation** — it would keep working with the internet unplugged.

Stretch goal: once it works, try adding **hysteresis** — switch on above 28 °C, but only switch off below 26 °C.

Up next

In Chapter 7 — IoT Security and Best Practices, you'll step back from building and look at what can go wrong. You'll learn the most common ways IoT devices get attacked — including some shortcuts we deliberately took in the labs — and how real products defend against them.

Going further (optional)

- Node-RED — a free, visual tool for building IoT rules and data flows by connecting boxes; a great next step after this book.
- ThingSpeak React app documentation — add cloud rules to the channel you built in Lab 4.
- AWS IoT Core rules — see how a large cloud platform handles rules at industrial scale.

Answer key

A1 · Pause and check — Your sensor reports 24.1, 24.3, 24.2, 97.0, 24.2 °C. Which processing step should deal with the 97.0?

Validation. A jump of over 70 °C between two readings isn't physically realistic for a room, so it's almost certainly a glitch and should be discarded, not averaged in.

A2 · Pause and check — Write a rule, in plain English, that sends an alert if a freezer's temperature rises above −10 °C, but only if it stays that warm for more than 10 minutes (so opening the door briefly doesn't trigger it).

One good answer: **IF** freezer temperature > −10 °C **AND** it has been above −10 °C continuously for 10 minutes **THEN** send an alert to the shop manager. Adding a time condition like this is another common way to avoid false alarms.

A3 · Question 1 — What's the difference between a reading and an event? Give an example of each.

A **reading** is a value at a moment in time, like “the temperature is 24 °C”. An **event** is a meaningful change, like “the temperature just went above 28 °C” or “the door opened”.

A4 · Question 2 — Name two benefits of an event-driven design for a battery-powered sensor.

Any two of: it sends far fewer messages, so it **saves battery**; it uses **less data and bandwidth** (and costs less); and it **reacts faster** to important changes.

A5 · Question 3 — A greenhouse fan keeps switching on and off every few seconds when the temperature is near its limit. What's the problem called, and how do you fix it?

It’s flickering (sometimes called chattering) caused by a single threshold. Fix it with **hysteresis**: switch the fan on above one limit (say 28 °C) and only switch it off below a lower limit (say 26 °C). A minimum time delay also helps.

A6 · Question 4 — A gas-leak detector must shut a valve instantly, even if the internet is down. Edge or cloud? Why?

Edge. The decision is safety-critical and urgent, and it must work with no internet connection. Waiting for a round trip to the cloud — or failing completely during an outage — isn’t acceptable.

A7 · Question 5 — What is a rules engine, and why would a large company use one instead of writing rules into device code?

A rules engine is software that stores and runs many if-then rules, usually through a friendly interface. A large company uses one because rules can be added or changed quickly, in one place, by non-programmers — without updating and re-uploading code to thousands of devices.

A8 · Question 6 — Give one decision in a smart home that belongs in the cloud rather than on a device.

Any decision needing the big picture or long history. For example: “if energy use this month is 20% higher than last month, send the owner a report”, or “if every window sensor shows closed and everyone’s phones have left home, turn off all the lights”.

CHAPTER 7

IoT Security and Best Practices

Every connected device is a small door into a network. This chapter explains how IoT devices get attacked, and the proven defences: strong identity, encryption, secure updates, and the security standards and laws that now apply to IoT products — including in Malaysia. There's no coding in this chapter. It ends with an activity — on your own or in a group — where you spot realistic vulnerabilities.

At a glance	
Reading time	About 25 minutes
Activity	Spot the Vulnerability
Before this	Chapter 6 — Data Processing and Automation, and Labs 3–5
You'll need	Nothing but a sceptical mind.

What you'll learn

By the end of this chapter, you'll be able to:

1. **Explain** why IoT devices are such attractive targets for attackers.
2. **Describe** the most common IoT threats and weaknesses, using the OWASP IoT Top 10.
3. **Explain** the difference between authentication and authorisation, and how devices prove who they are.
4. **Explain** how encryption protects data in transit and at rest.
5. **Describe** what makes a firmware update process secure.
6. **Name** the main IoT security standards and laws, including those relevant in Malaysia.
7. **Spot** common security mistakes in real code and configuration.

Where we are

Over the last few chapters, you've built a device that senses, connects, sends data to the cloud, and acts by itself. Along the way, we took some deliberate shortcuts to keep the labs simple:

- In **Lab 3**, you published to a **public** MQTT broker, with **no password** and **no encryption**.
- In **Lab 4**, you sent data over plain **HTTP**, not encrypted HTTPS, and put your API key directly in the code.
- In the labs, the Wi-Fi details are written straight into the code.

None of that mattered for harmless, simulated temperature readings. But in a real product, every one of those shortcuts could be a serious security hole. This chapter explains why — and what professionals do instead.

The big picture: why IoT is a target

Attackers love IoT devices for four reasons:

1. **There are so many of them.** Billions of devices means billions of possible targets. Even a small weakness, repeated across millions of identical devices, is a big prize.
2. **Many are never updated.** A laptop or phone updates itself regularly. Many cheap IoT devices are installed once and never updated again — so a weakness found in year one is still there in year five.
3. **Security was an afterthought.** In a rush to be cheap and quick to market, some makers ship devices with weak passwords, old software, and no way to fix them.
4. **They connect to the physical world.** Hacking a laptop can steal data. Hacking an IoT device can unlock a door, switch off a freezer, open a valve, or watch someone through a camera.

A wake-up call: the Mirai botnet (2016)

In 2016, a piece of malicious software called **Mirai** spread across the internet. It did something very simple: it scanned for IoT devices — mostly cameras and home routers — and tried to log in using a short list of **factory default usernames and passwords**, like `admin` / `admin`.

It worked on a huge scale. At its peak, Mirai controlled around **600,000** devices. Together, these hijacked devices formed a **botnet** — an army of infected machines controlled by an attacker. On 21 October 2016, part of this botnet flooded a major internet company called Dyn with traffic, which made websites including Twitter, Netflix, and Reddit hard to reach for many people in the United States and Europe.

No clever hacking was needed. The devices' owners had simply never changed the default passwords — and in many cases, didn't know they could.

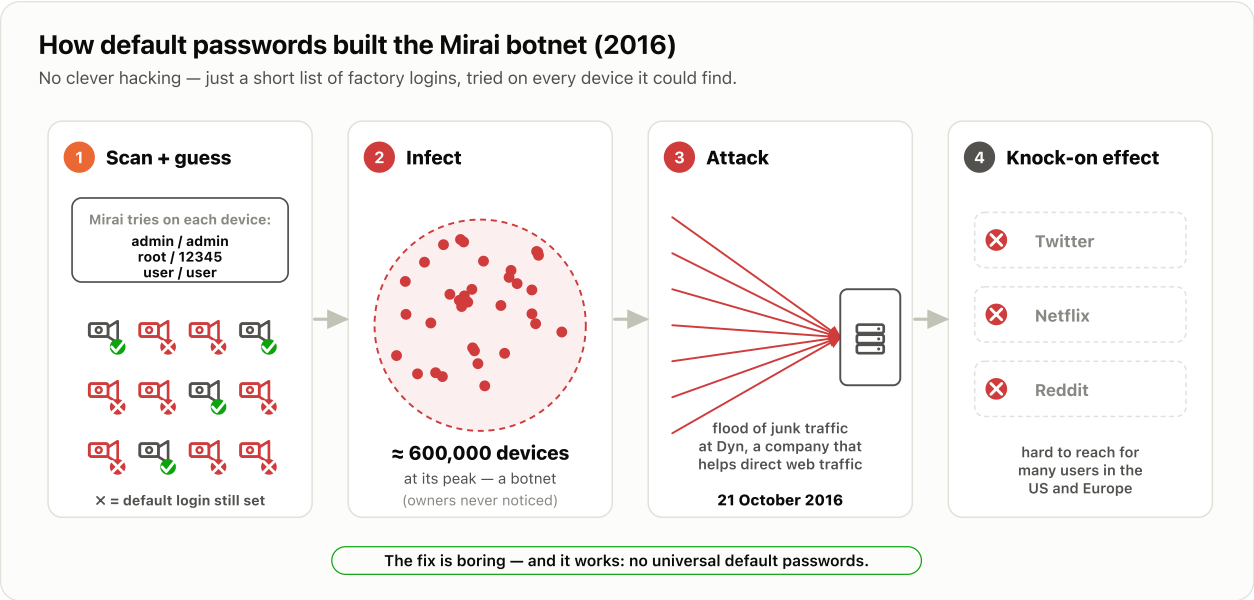


Figure 7.1 — how the Mirai botnet grew from default passwords into a major outage.

IMPORTANT

The lesson of Mirai is that **the most damaging IoT attacks often exploit the simplest mistakes.** Default passwords, missing updates, and unencrypted connections cause far more real-world harm than exotic, movie-style hacking.

What are we protecting? The CIA triad

Security professionals describe their goals with three words, known as the **CIA triad** (nothing to do with spies):

Goal	Means	IoT example of it going wrong
Confidentiality	Only the right people can **see** the data	A stranger watches your baby-monitor camera.
Integrity	Data and commands can't be secretly **changed**	Someone fakes a sensor reading so a factory shuts down, or changes a medicine pump's dose.
Availability	The system keeps **working** when needed	A smart lock won't open because its cloud service is down or under attack.

For IoT, there's often a fourth goal: **safety**. When devices control physical things, a security failure can put people in danger.

Common IoT threats and vulnerabilities

Where attacks can happen

An attacker can target any layer of an IoT system — the same five layers you learned in Chapter 2, plus the people who use it. The set of all possible points of attack is called the **attack surface**:

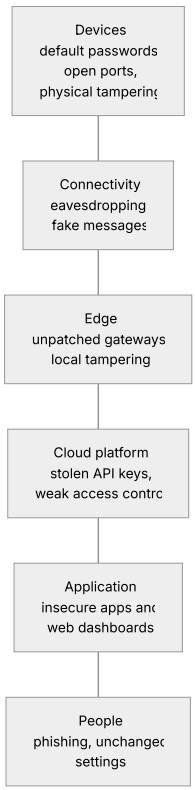


Figure 7.2 — the attack surface spans all five layers — devices, connectivity, edge, cloud platform and application — plus the people using the system, each with its own typical weaknesses.

The OWASP IoT Top 10

OWASP (the Open Worldwide Application Security Project) is a respected non-profit that publishes free security guidance. Its **IoT Top 10** lists the most common weaknesses found in IoT products. It was published in 2018, and it's still the current version at the time of writing — sadly, because the same problems keep appearing. Here it is, explained simply:

#	Weakness	What it means
1	**Weak, guessable, or hardcoded passwords**	Default logins like `admin` / `admin`, or passwords built into the software that can't be changed.
2	**Insecure network services**	The device leaves unnecessary "doors" (network services) open to the internet.

#	Weakness	What it means
3	**Insecure ecosystem interfaces**	The web dashboard, phone app, cloud API, or other systems around the device are poorly protected.
4	**Lack of secure update mechanism**	The device can't be updated, or updates aren't checked to make sure they're genuine.
5	**Use of insecure or outdated components**	Old software libraries or parts with known weaknesses.
6	**Insufficient privacy protection**	Personal data is collected, stored, or shared carelessly, or without permission.
7	**Insecure data transfer and storage**	Data isn't encrypted when sent or when stored.
8	**Lack of device management**	Nobody can see which devices are deployed, whether they're healthy, or switch off a bad one.
9	**Insecure default settings**	The device ships with unsafe settings, and users can't easily make it safer.
10	**Lack of physical hardening**	Someone with physical access can easily open the device and extract secrets.

Notice how many of these are about **defaults** and **neglect**, not cutting-edge attacks. That's good news: most IoT security problems can be prevented with well-known, simple practices.

> **TIP**
>
> **Pause and check.** Look back at the shortcuts we took in the labs (public broker, no encryption, secrets in code). Which OWASP items do they match?
>
> *→ Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

Defence 1: device authentication and authorisation

Two words sound similar but mean different things, and both matter:

- **Authentication** answers **"Who are you?"** — proving identity.
- **Authorisation** answers **"What are you allowed to do?"** — deciding permissions.

Think of a hotel. Showing your passport at check-in is **authentication**. Your room key card only opening *your* room, the gym, and the pool — but not anyone else's room — is **authorisation**.

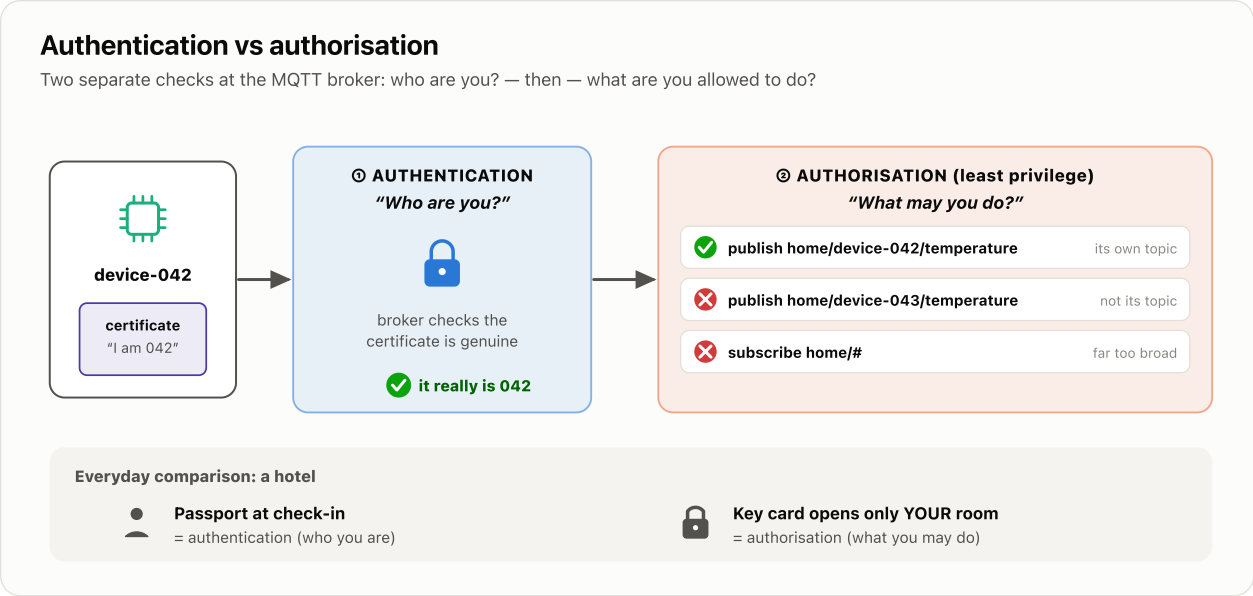


Figure 7.3 — authentication proves who a device is; authorisation limits what it may do.

How devices prove who they are

People log in with a username and password. Devices need a way to log in too. From weakest to strongest:

Method	How it works	Strength
No authentication	Anyone can connect (like the public broker in Lab 3).	None — learning only.
Shared password	Every device uses the same password.	Weak — steal one, and you have them all.
Unique password or key per device	Each device has its own secret (like the Write API key for a ThingSpeak channel).	Better — one leak affects only one device.
Digital certificate per device	Each device holds a unique digital certificate, and a secret key that never leaves the device.	Strongest — the professional standard.

A **digital certificate** is like a passport for a device. It's issued by a trusted authority and proves the device is genuine. Platforms like AWS IoT Core use certificates, and they work together with encryption (see below) so that **both** sides check each other: the device checks it's talking to the real server, and the server checks it's talking to a real device. This is called **mutual authentication**.

Authorisation: least privilege

Once a device is authenticated, it should only be allowed to do what it genuinely needs to do. This is the **principle of least privilege**.

For MQTT, that means each device should only be allowed to publish and subscribe to **its own topics**:

```
Allowed:      home/device-042/temperature
Not allowed:  home/device-043/temperature   (another device's topic)
Not allowed:  home/#                        (everyone's topics)
```

If an attacker takes over device 042, least privilege stops them from controlling device 043 — or every lock in the building.

Protecting the humans too

Devices are often managed through apps and web dashboards, so the people logging in need protecting as well:

- **No universal default passwords.** Every device should have a unique password, or force the user to set one during setup.
- **Strong passwords**, and **multi-factor authentication** (a second check, such as a code sent to your phone) for important accounts.
- **Lock out** repeated failed login attempts, so passwords can't simply be guessed over and over.

Defence 2: data encryption and secure communication

Encryption scrambles data using a mathematical key, so that only someone with the right key can unscramble it. To anyone else, it looks like random nonsense.

It's like sending a letter in a locked box instead of on a postcard. Anyone handling a postcard can read it; a locked box can only be opened by the person with the key.

Data in transit and data at rest

Data needs protecting in two situations:

- **In transit** — while it's travelling across a network. The standard protection is **TLS** (Transport Layer Security), the same technology that puts the padlock in your browser's address bar. HTTPS is HTTP protected by TLS; secure MQTT uses TLS on port **8883**.
- **At rest** — while it's stored on the device or in the cloud. Stored data should be encrypted too, so a stolen device or a leaked database doesn't reveal everything.

	Insecure	Secure
Web requests	`http://` (port 80)	`https://` (port 443)
MQTT	port 1883	port 8883, with TLS
Stored data	Plain text	Encrypted

Keep secrets out of code

Wi-Fi passwords, API keys, and certificates are **secrets**. They should never be written directly into code that's shared — and especially not uploaded to a public website like GitHub, where automated tools constantly scan for leaked keys. Keep secrets in a separate file that's never shared, or in the device's protected storage.

> **WARNING**
>
> Encryption without authentication is only half the job. If your device encrypts its data perfectly — but sends it to an attacker pretending to be your server — the attacker simply decrypts it. That's why TLS checks the server's certificate. Always use **both** encryption and authentication.

Defence 3: firmware updates and patch management

Firmware is the software that runs on a device — like the sketches you ran on the ESP32 in the labs. No software is perfect. New weaknesses are discovered all the time, even in well-made products. The fix is a **patch** — an update that closes the hole.

So every connected device needs a way to be updated after it's installed. Updates delivered over the network, without anyone touching the device, are called **over-the-air (OTA) updates**.

But an update system is also a powerful door into the device. If an attacker can send a fake update, they own the device completely. A **secure update** process looks like this:

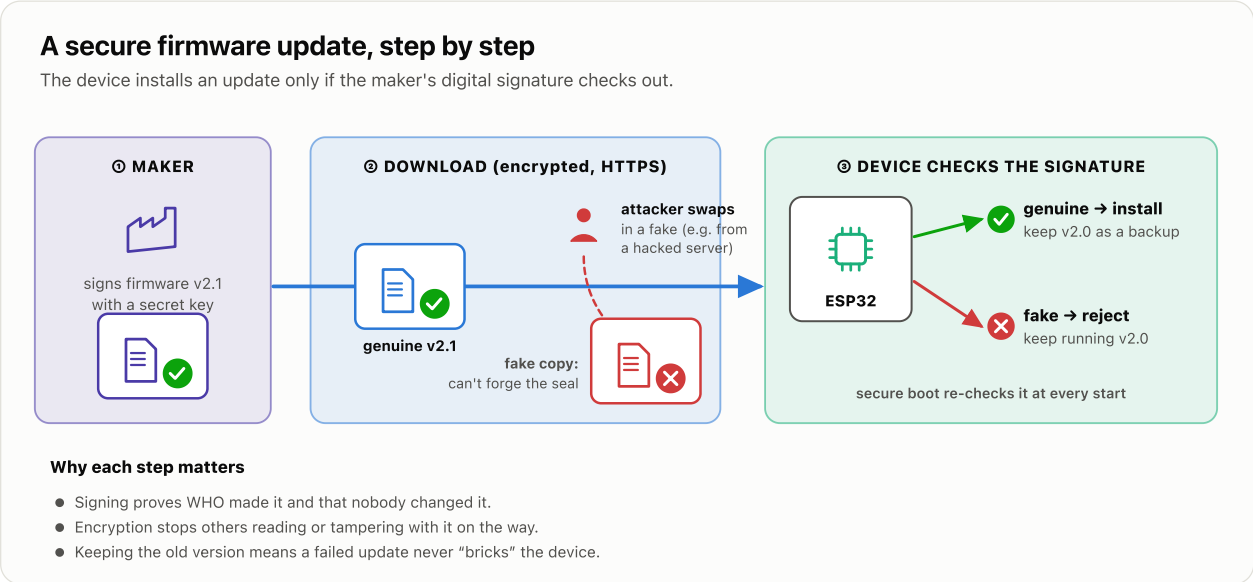


Figure 7.4 — a secure update. The device trusts the signature, not the download.

The key ideas:

- **Code signing.** The maker adds a **digital signature** to each update — a mathematical seal that proves who made it and that nobody has changed it. The device checks the seal before installing anything.
- **Encrypted delivery.** Updates are downloaded over HTTPS or another encrypted connection.
- **Safe fallback.** If an update fails halfway (for example, the power cuts out), the device can go back to its previous working version instead of becoming a useless "brick".
- **Secure boot.** Each time it starts, the device checks that its own firmware is genuine before running it.
- **A stated support period.** Makers should say clearly how long they'll provide security updates — so buyers know when a device will stop being protected.

Patch management is the organisational side: keeping a list of every device and its firmware version, testing updates, rolling them out, and checking they worked. Without it, you get the problem OWASP #8 ("lack of device management") warns about.

Defence 4: a secure-by-design mindset

The cheapest time to add security is at the very start of a design, not after a product is attacked. This is called **secure by design**. A few habits go a long way:

- **Minimise.** Switch off every feature, port, and service you don't need. Every open door is one more thing to guard.
- **Defence in depth.** Use several layers of protection, so if one fails, another still holds — like a building with a gate, a locked door, *and* a safe.
- **Collect only what you need.** Personal data you never collect can never be leaked. This is also a legal requirement in many countries.
- **Plan for the whole life of the device.** Think about how it's set up, updated, and finally retired — including wiping its data.
- **Make it easy to report problems.** Publish a way for security researchers to report weaknesses they find.

Security frameworks, standards, and laws

You don't have to invent IoT security from scratch. Experts have written **frameworks** (structured sets of guidance) and **standards** (agreed requirements), and more and more countries are turning them into **laws**. Here are the most important ones for IoT today.

International standards and guidance

Name	From	What it is
OWASP IoT Top 10	OWASP (non-profit)	The ten most common IoT weaknesses — a great checklist to start with.
ETSI EN 303 645	ETSI, the European standards body	The leading baseline for **consumer** IoT security. Its 13 top-level provisions start with "no universal default passwords", and include having a way to report vulnerabilities, keeping software updated, and protecting personal data.
NIST IR 8259 series	NIST, the US standards agency	Guidance for device makers. Its core baseline says a device should support: device identification, device configuration, data protection, logical access to interfaces, software updates, and cybersecurity state awareness. A revision, NIST IR 8259 Rev. 1, was published in April 2026.
IEC 62443	International Electrotechnical Commission	The main standard series for **industrial** control systems and industrial IoT. It divides a site into zones and sets security levels for each.

Laws in other countries

Around the world, IoT security is moving from "nice to have" to "required by law":

- **United Kingdom** — The **Product Security and Telecommunications Infrastructure (PSTI) Act**, in force since **29 April 2024**, bans universal or easily guessable default passwords on consumer devices, requires makers to publish how to report security problems, and requires them to state how long they'll provide security updates.
- **European Union** — The **Cyber Resilience Act** entered into force in December 2024. Since **11 September 2026**, makers must report actively exploited vulnerabilities and severe incidents quickly. From **11 December 2027**, products with digital elements sold in the EU must meet its full security requirements.
- **United States** — The **U.S. Cyber Trust Mark** is a voluntary security label for consumer smart devices, launched by the FCC in 2025. After a change of programme administrator in 2026, it is still being rolled out, so few products carry it yet.

If you ever build a product for export, these rules will apply to you.

Malaysia

In Malaysia, several laws and technical codes are relevant to IoT:

- **Cyber Security Act 2024 (Act 854)** — in force since **26 August 2024**, and overseen by the National Cyber Security Agency (**NACSA**). It applies to organisations running **national critical information infrastructure** in 11 sectors, including energy, water,

healthcare, transport, and agriculture and plantation. These organisations must carry out risk assessments and audits and report cyber incidents. IoT and industrial systems used in those sectors fall within its scope.

- **Personal Data Protection Act (PDPA), as amended in 2024** — applies to any IoT system that collects **personal data**, such as cameras, wearables, or smart meters linked to a person. Since **1 June 2025**, organisations must appoint a data protection officer and notify the Personal Data Protection Commissioner of a personal data breach within **72 hours**. Biometric data (such as face-recognition data or fingerprints) is now treated as sensitive personal data.
- **MCMC technical codes** — the Malaysian Communications and Multimedia Commission (MCMC) has published technical codes on IoT security, including **MCMC MTSFB TC G013:2018** (IoT Security Management) and **MCMC MTSFB TC G031:2021** (IoT Application Security Requirements).

> **NOTE**
>
> This is a summary to help you find your way, not legal advice. Laws change, and how they apply depends on the product and the organisation. For a real product, always check the latest official sources, or speak to a legal or compliance specialist.

Common misconceptions

"My device is too small and boring to be attacked." Attackers rarely pick devices one by one. Automated tools, like Mirai, scan the whole internet for *any* device with a known weakness. Your "boring" device can be recruited into a botnet to attack someone else.

"Hiding the Wi-Fi network name makes it secure." Hidden networks are easy to find with free tools. Real protection comes from strong passwords and encryption, not from hiding.

"Security is only the device maker's problem." It's shared. Makers must design securely, but installers and users must change default passwords, apply updates, and protect their accounts.

"Encryption solves everything." Encryption protects data from being read. It doesn't stop weak passwords, fake updates, excessive permissions, or a stolen API key. Security needs several layers.

"Once it's secure, it stays secure." New weaknesses are found all the time. A device that can't be updated becomes less secure every year it stays in service.

Real-world example: securing our smart room monitor

Let's turn our lab project into something you could safely sell. Here's how each shortcut would change:

In the labs	In a real product
Public MQTT broker, anyone can join	Private broker (or a platform like AWS IoT Core)
No authentication	A unique certificate for each device
Port 1883, unencrypted	Port 8883 with TLS encryption
Any client can read `iot-workshop/#`	Each device may only use its own topics
Plain HTTP to ThingSpeak	HTTPS
Wi-Fi password and API key in the code	Secrets stored separately, set during setup, never shared
No way to update the device	Signed OTA updates, with a stated support period
—	A published way to report security problems

None of these are exotic. They're standard practice — and increasingly, the law.

Key takeaways

- IoT is a big target because of **scale, neglect, rushed design, and physical impact**. Mirai showed how much damage **default passwords** alone can do.
- Security protects **confidentiality, integrity, and availability** — and in IoT, **safety** too.
- The **OWASP IoT Top 10** lists the most common weaknesses; most come from **defaults and neglect**, not advanced attacks.
- **Authentication** proves who a device is; **authorisation** limits what it can do. Aim for **unique per-device identities** and **least privilege**.
- **Encrypt** data **in transit** (TLS, HTTPS, MQTT on 8883) and **at rest**, and keep **secrets out of code**.
- **Firmware updates** must be **signed**, delivered **securely**, and able to **fall back** safely.
- Key standards: **ETSI EN 303 645, NIST IR 8259, IEC 62443**. Key laws: **UK PSTI Act, EU Cyber Resilience Act**, and in Malaysia the **Cyber Security Act 2024** and the amended **PDPA**.

Check your understanding

1. Why did the Mirai botnet succeed in taking over so many devices?

→ Answer A2 is in the answer key at the end of this chapter. Try it yourself first!

2. What's the difference between authentication and authorisation? Give an IoT example of each.

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

3. Why is a unique password or certificate per device better than one shared password for all devices?

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

4. What's the difference between data in transit and data at rest, and how do you protect each?

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

5. Name three features of a secure firmware update process.

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

6. A Malaysian company is building a smart camera that recognises residents' faces. Which Malaysian law is most directly relevant, and why?

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
Attack surface	All the points where an attacker could try to get into a system.
Authentication	Proving who (or what) you are.
Authorisation	Deciding what an authenticated user or device is allowed to do.
Botnet	A network of hijacked devices controlled by an attacker.
CIA triad	The three goals of security: confidentiality, integrity, availability.
Code signing	Adding a digital signature to software to prove who made it and that it hasn't been changed.
Data at rest / in transit	Data stored on a device or server (at rest), or travelling across a network (in transit).
Defence in depth	Using several layers of protection, so one failure isn't fatal.
Digital certificate	An electronic "passport" that proves a device or server's identity.
Digital signature	A mathematical seal that proves who made a file and that nobody has changed it.
Encryption	Scrambling data so only someone with the right key can read it.
Firmware	The software that runs on a device.

Term	Meaning
Least privilege	Giving each user or device only the permissions it truly needs.
Multi-factor authentication	Logging in with two or more kinds of proof, such as a password plus a phone code.
Mutual authentication	Both sides of a connection proving their identity to each other.
OTA update	An over-the-air update, delivered to a device over the network.
OWASP IoT Top 10	A list of the ten most common IoT security weaknesses.
Patch	A software update that fixes a bug or security weakness.
Patch management	Keeping track of every device and its software version, and rolling out updates safely.
Secure boot	A device checking its own firmware is genuine every time it starts.
Secure by design	Building security in from the start, rather than adding it later.
TLS	Transport Layer Security — the standard way to encrypt internet connections.

Group activity: spot the vulnerability

Now put this chapter to work. Open **Spot the Vulnerability** (about 20 minutes, no coding).

On your own or in a small group, you'll read five short, realistic snippets of code and configuration — a Wi-Fi password in public code, a camera with a default login, readings sent over plain HTTP, an MQTT topic every lock obeys, and an unsigned firmware update. For each one, discuss:

1. **What's wrong?**
2. **What's the realistic worst case?**
3. **How would you fix it?** (Hint: every fix is somewhere in this chapter.)

If you're in a group, share your answers — one sentence each. If you're on your own, write yours down, then compare them with the notes under each snippet.

Up next

In Chapter 8 — Real-World IoT Applications and Project Showcase, you'll pull the whole day together. You'll walk through complete IoT systems from real industries, learn how professionals take an idea from prototype to deployment, and build your capstone project in Lab 6.

Going further (optional)

- OWASP Internet of Things Project — the IoT Top 10 and related guides.
- ETSI EN 303 645 — the consumer IoT security baseline.
- NIST Cybersecurity for IoT Program — including the NIST IR 8259 series.
- NACSA — Cyber Security Act 2024 (Act 854) — Malaysia's cybersecurity law.
- OWASP IoT Security Testing Guide — for anyone curious about how security testers examine devices.

Answer key

A1 · Pause and check — Look back at the shortcuts we took in the labs (public broker, no encryption, secrets in code). Which OWASP items do they match?

Mainly **#7 Insecure data transfer** (plain MQTT on port 1883 and plain HTTP), **#3 Insecure ecosystem interfaces** (a public broker anyone can use, with no authentication), and **#1 Hardcoded passwords/secrets** (Wi-Fi details and the API key written into the code).

A2 · Question 1 — Why did the Mirai botnet succeed in taking over so many devices?

Because huge numbers of devices still used their **factory default usernames and passwords**. Mirai simply tried a short list of common defaults and logged in.

A3 · Question 2 — What's the difference between authentication and authorisation? Give an IoT example of each.

Authentication proves *who* you are — e.g. a device presenting its certificate to the cloud. **Authorisation** decides *what* you may do — e.g. that device being allowed to publish only to its own MQTT topic.

A4 · Question 3 — Why is a unique password or certificate per device better than one shared password for all devices?

If one shared password leaks, an attacker can access **every** device. With unique credentials, a leak affects only **one** device, and that single device can be blocked without affecting the others.

A5 · Question 4 — What's the difference between data in transit and data at rest, and how do you protect each?

In transit is data moving across a network — protect it with **TLS** (HTTPS, MQTT on port 8883). **At rest** is data stored on a device or in the cloud — protect it by **encrypting the stored data**.

A6 · Question 5 — Name three features of a secure firmware update process.

Any three of: updates are **digitally signed** and the device **checks the signature**; they're delivered over an **encrypted** connection; the device can **fall back** to the previous version if an update fails; **secure boot** checks the firmware at every start; and the maker states a **support period**.

A7 · Question 6 — A Malaysian company is building a smart camera that recognises residents' faces. Which Malaysian law is most directly relevant, and why?

The **Personal Data Protection Act (PDPA)**, as amended in 2024. Face images are **biometric data**, now treated as **sensitive personal data**, and the company must protect it and notify the Commissioner of any breach within 72 hours. (If the camera were used in a critical infrastructure sector, the Cyber Security Act 2024 could also apply.)

CHAPTER 8

Real-World IoT Applications and Project Showcase

You now know every part of an IoT system. This final chapter shows how professionals put those parts together: how to design a complete system step by step, how real industries use IoT, how to prototype an idea quickly and cheaply, and what goes wrong when projects move into the real world. Then you'll build and show off your capstone project in Lab 6.

At a glance	
Reading time	About 25 minutes
Lab	Lab 6 — Capstone: Smart Environment Station
Before this	All of Chapters 1–7, and Labs 1–5
You'll need	Your Wokwi account, the HiveMQ web client from Lab 3, and, if you like, someone to show your project to

What you'll learn

By the end of this chapter, you'll be able to:

1. **Follow** a step-by-step method to design an IoT system from a real-world problem.
2. **Explain** the design choices behind real IoT systems in smart homes, industry, and agriculture.
3. **Describe** how teams prototype an IoT idea quickly and prove it works before spending big.
4. **Identify** the main challenges of deploying IoT outside the lab, and how to plan for them.
5. **Build and present** a complete IoT project that combines sensing, a local display, edge automation, and cloud messaging.

Where we are

Look how far you've come:

Chapter	You learned	You built
1	What IoT is: sense → send → decide → act	—
2	The five layers of every IoT system	—
3	Sensors, actuators, analog and digital	Labs 1 and 2: LEDs, a button, a dial, a DHT22
4	Connectivity and protocols	Lab 3: Wi-Fi and MQTT
5	Cloud platforms and dashboards	Lab 4: a ThingSpeak live graph
6	Processing and automation	Lab 5: an edge rule that switches a relay
7	Security	Spotted realistic vulnerabilities

Each of those was one piece of the puzzle. Chapter 8 is about the whole picture — seeing how real systems are designed from end to end, and then assembling all your pieces into one working project.

The big picture: from problem to deployed system

Real IoT projects don't start with "let's use LoRaWAN" or "let's buy some sensors". They start with **a problem someone wants solved**. Then the team works through a series of stages, learning and adjusting at each one:

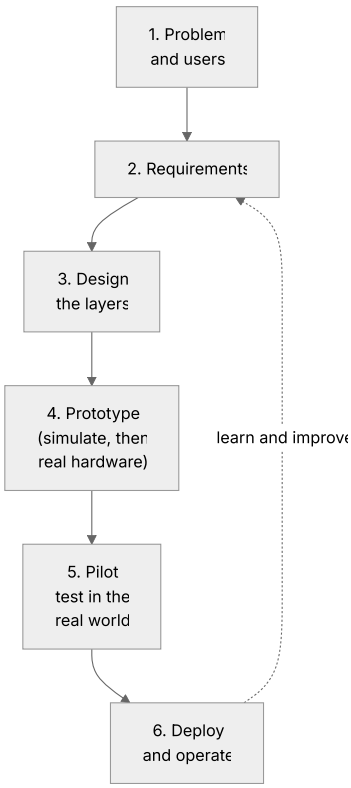


Figure 8.1 — six stages — problem, requirements, design, prototype, pilot, and deploy and operate — with a loop back from deployment to requirements, because lessons from the real world feed the next version.

The dotted arrow matters. IoT projects are rarely right first time. Each stage teaches you something that improves the next version.

A design method you can reuse

Here's a practical, step-by-step method for designing any IoT system. It turns the five layers from Chapter 2 into a series of questions.

Step 1: Start with the problem and the people

Ask:

- **What problem are we solving,** and how will we know it's solved?
- **Who** will use the system, and what do they need to *do* differently?
- **What's it worth?** How much money, time, water, or risk will it save?

A good problem statement sounds like this: *"Our greenhouse loses about 10% of its seedlings each summer because nobody notices when it overheats at weekends."* It's specific, and it tells you what success looks like.

Step 2: Turn it into requirements

Now list what the system must do, and the conditions it must work in:

Question	Why it matters
What must we **measure** (or control)?	Chooses the sensors and actuators.
How often, and how **accurately**?	Affects battery life, data volume, and sensor choice.
Where will devices be — indoors, outdoors, far away?	Affects connectivity, enclosures, and power.
How are they **powered**? How long must a battery last?	Often the hardest constraint of all.
How fast must the system react?	Decides what runs at the edge versus the cloud.
How many devices, now and in future?	Affects cost, management, and platform choice.
What's the **budget** per device, and per month?	Rules options in or out quickly.
What **data** is personal or sensitive, and what laws apply?	Shapes security and privacy design.

Step 3: Design each layer

With the requirements in hand, choose an option for each of the five layers — and write down **why**. The "why" is the real skill:

1. **Devices:** which sensors, actuators, and microcontroller?
2. **Connectivity:** which radio and which protocol? (Chapter 4's four questions: how far, how powered, how much data, what exists already.)
3. **Edge:** which decisions must happen locally? (Chapter 6: urgent, safety-critical, or must work offline.)
4. **Cloud platform:** where is data stored and analysed, and which rules run there?
5. **Application:** who sees what, on which screen? (Chapter 5's dashboard rules.)

And across every layer, apply **security** (Chapter 7): unique device identities, encryption, least privilege, and a way to update devices.

Step 4 onwards: prototype, pilot, deploy

We'll cover these stages in the prototyping and deployment sections below.

End-to-end walkthrough: a smart irrigation station

Let's use the method on a complete example: helping a small farm water its crops more efficiently.

The problem. A vegetable farm spread over several fields waters on a fixed timetable. Some fields get too much water, some too little, and pumping costs money. The owner wants to water each field only when it actually needs it.

The requirements.

- Measure **soil moisture** in each field, plus the temperature.
- A reading every **15–30 minutes** is plenty — soil dries slowly.
- The fields are up to **3 km** from the farmhouse, with **no mains power** and **no Wi-Fi**.
- Sensors must run on batteries for **at least a year**.
- If the soil is too dry, **open the irrigation valve**. It must work even if the internet is down.
- The owner wants a **phone view** and an alert if something goes wrong.

The design, layer by layer:

Layer	Choice	Why
Devices	A soil-moisture sensor and a temperature sensor on a low-power microcontroller in each field; a relay-controlled valve by the pump	Measures what matters; the relay lets a tiny controller switch the valve safely
Connectivity	**LoRaWAN** from the fields to one gateway at the farmhouse; the gateway uses the farm's internet connection	Kilometres of range, tiny data, years on a battery — Wi-Fi couldn't reach, and would drain batteries
Edge	The valve controller decides locally: open below 30% moisture, close above 45% (with hysteresis, from Chapter 6)	Keeps working if the internet drops; avoids the valve flickering on and off
Cloud platform	Stores every reading; a rule alerts the owner if a sensor goes silent for 2 hours, or a field stays dry after watering	Big-picture checks across all fields, plus a history to learn from
Application	A phone dashboard: one status tile per field ("OK" / "DRY — watering"), plus a line chart of moisture over the week	The owner can see what needs attention in seconds, without reading numbers
Security	Each sensor has its own keys (LoRaWAN encrypts messages by design); the dashboard uses a strong password and multi-factor login; devices can receive signed updates	Makes it very hard to fake readings or open valves remotely; the system can be patched

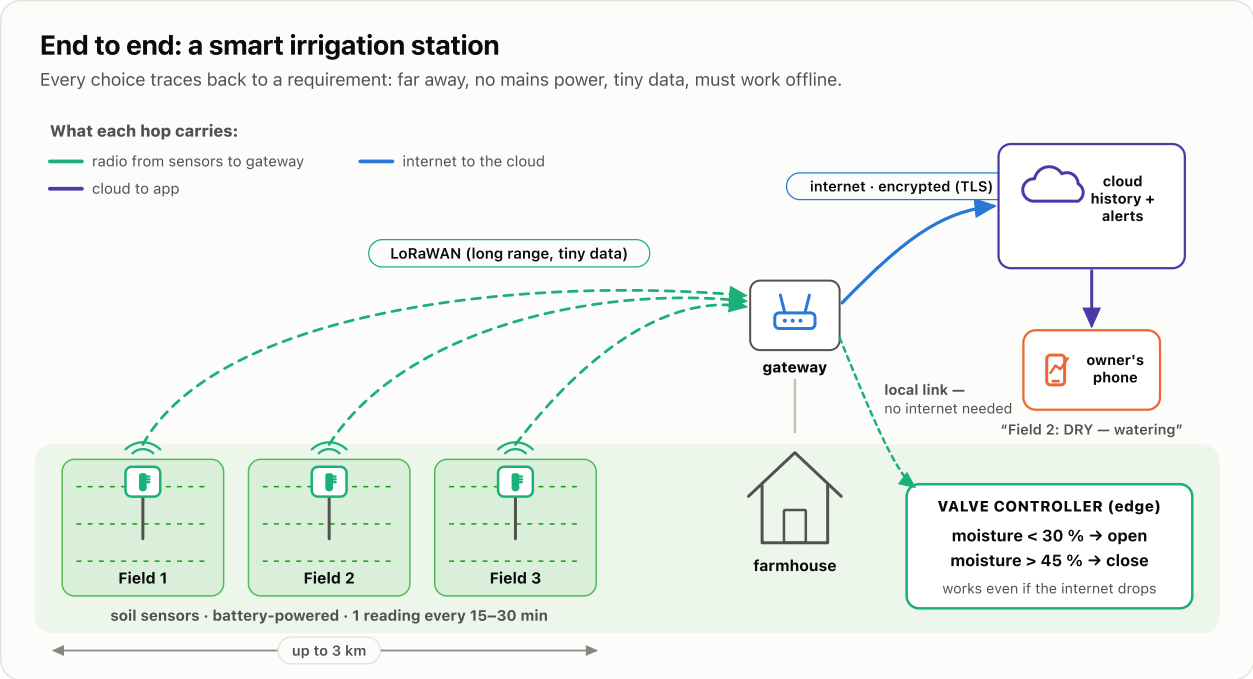


Figure 8.2 — the smart irrigation station, end to end. The watering rule runs locally, not in the cloud.

Notice that **every choice traces back to a requirement**. That's the sign of a good design.

> **TIP**
>
> **Pause and check.** In this design, why is the "open the valve" rule at the edge, while the "sensor has gone silent" rule is in the cloud?
>
> *→ Answer A1 is in the answer key at the end of this chapter. Try it yourself first!*

Case studies from real industries

Here are three common kinds of IoT system. For each one, pay attention to **why** each choice suits its environment. That's the real lesson.

Case study 1: Smart home automation

The goal: comfort, convenience, security, and lower energy bills.

- **Devices:** smart lights, plugs, thermostats, door and window sensors, motion sensors, locks, cameras, and a voice assistant or hub.
- **Connectivity:** a mix. Plugged-in devices like cameras use **Wi-Fi** (lots of data, mains power). Battery devices like door sensors use **Zigbee** or **Thread** (very low power, and the mesh covers the whole house). Phones set up devices over **Bluetooth**.
- **Working together:** the **Matter** standard lets devices from different brands work in the same home. Recent versions of Matter have added support for more device types, including cameras.

- **Edge vs cloud:** a good smart home keeps working when the internet is down — a local hub runs rules like "motion in the hallway at night → turn on the light at 20%". The cloud handles remote access and history.
- **Special challenges: privacy** (cameras and microphones in private spaces), **ease of setup** for non-technical people, and devices from many brands needing to cooperate.

Case study 2: Industrial equipment monitoring

The goal: avoid expensive unplanned breakdowns, and run machines more efficiently.



A wireless vibration sensor fixed to an industrial motor. Photo: Cedric Leturcq, CC BY-SA 4.0, via Wikimedia Commons.

- **Devices:** vibration, temperature, and current sensors fitted to motors, pumps, and conveyor belts. A failing bearing often vibrates differently weeks before it breaks.
- **Connectivity:** often **wired** (Ethernet and industrial networks) for reliability, with wireless sensors where cables are impractical. Data goes to **gateways** on the factory floor.
- **Edge vs cloud:** vibration data is produced very fast — far too much to send it all to the cloud. So **edge gateways** analyse it on site and send only summaries and warnings. The cloud compares machines across the factory, and runs **predictive maintenance** models that forecast failures.
- **The payoff:** fixing a machine *before* it breaks, at a planned time. The consultancy McKinsey reports that predictive maintenance typically cuts machine downtime by **30% to 50%** and extends machine life by 20% to 40%.
- **Special challenges:** connecting to **old machines** that were never designed to share data, harsh conditions (heat, dust, electrical noise), and strict **security**, because a hacked factory can cause physical harm. Industrial sites typically follow the **IEC 62443** security standards from Chapter 7.

Case study 3: Smart agriculture

The goal: grow more with less water, fertiliser, and labour.



A soil sensor installed in a field. It reports conditions so watering happens only when needed. Photo: Christianjhart1, CC0, via Wikimedia Commons.

- **Devices:** soil-moisture and nutrient sensors, weather stations, water-tank level sensors, and controllers for irrigation and **fertigation** (feeding plants through the irrigation water).
- **Connectivity: LoRaWAN** or **NB-IoT**, because farms are large and often far from Wi-Fi. Messages are small and infrequent, which suits these low-power networks perfectly.
- **Edge vs cloud:** irrigation decisions run locally, so crops are watered even if the connection drops. The cloud combines sensor data with weather forecasts ("rain is coming tomorrow — skip watering today").
- **In Malaysia:** there is growing interest in IoT-based fertigation for vegetables and fruit, and in monitoring conditions across large plantations, where distances are huge and manual checks are slow and costly.
- **Special challenges: power** (solar panels and long battery life), weatherproofing against heat, rain, and humidity, patchy connectivity in rural areas, and keeping costs low enough for smaller farms.

What the case studies have in common

	Smart home	Industrial monitoring	Smart agriculture
Main goal	Comfort, security, energy	Avoid breakdowns	Save water and labour
Typical connectivity	Wi-Fi, Zigbee/Thread, Bluetooth (BLE)	Wired + gateways	LoRaWAN, NB-IoT
At the edge	Local hub rules	Vibration analysis on gateways	Irrigation control
In the cloud	Remote access, history	Cross-factory analysis, prediction	Weather-aware planning
Biggest challenge	Privacy and ease of use	Old machines and safety	Power and remote locations

Same five layers, every time. Different requirements lead to different choices at each layer.

Rapid prototyping and proof of concept

Building thousands of IoT devices is expensive. Discovering a design flaw *after* you've built them is much more expensive. So professional teams test their ideas as **cheaply and quickly as possible first**. This approach is called **rapid prototyping**.

The prototyping ladder

Most IoT products climb a ladder like this, getting more real — and more expensive — at each rung:

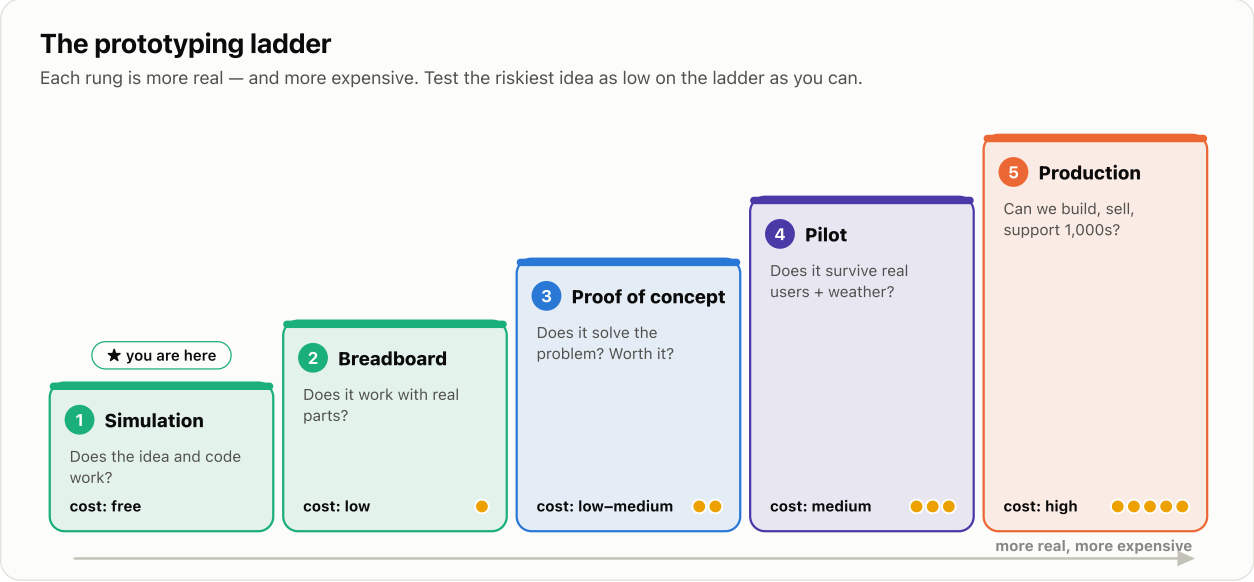
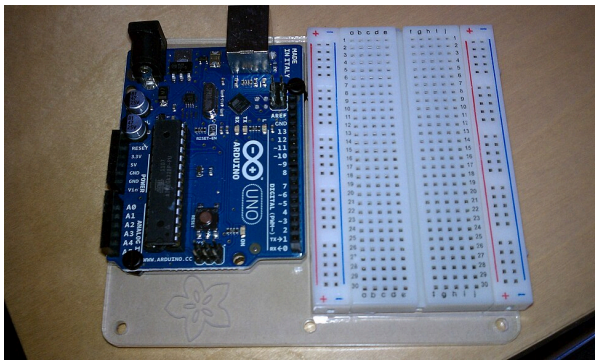


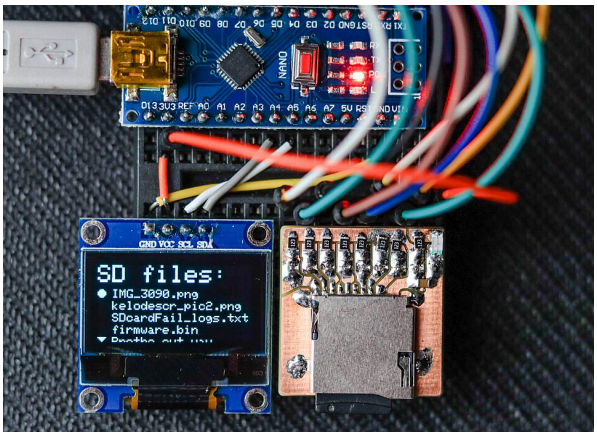
Figure 8.3 — the prototyping ladder. So far you've been on the first rung — the cheapest place to make mistakes.

Stage	What it is	Question it answers	Typical cost
1. Simulation	Build the circuit and code in a simulator, like Wokwi	"Does the idea and the code work at all?"	Free
2. Breadboard prototype	Real development boards and sensors, pushed into a breadboard with jumper wires	"Does it work with real parts?"	Low
3. Proof of concept (PoC)	A small working system tested on the real problem, often with a few devices	"Does it actually solve the problem, and is it worth it?"	Low to medium
4. Pilot	A small real deployment — say 20 devices on one site — run for weeks or months	"Does it survive the real world and real users?"	Medium
5. Production	Custom circuit boards, enclosures, certification, manufacturing at scale	"Can we build, sell, and support thousands?"	High

So far, you've been working at stage 1. That's exactly how real teams start. And because the Wokwi simulator copies a real ESP32 closely, your code would run on a real breadboard prototype (stage 2) with the same parts wired the same way — changing only the Wi-Fi name and password, because `Wokwi-GUEST` exists only inside the simulator.



Rung 2: a development board and a breadboard, ready for jumper wires. Photo: TreyDanger, CC BY 2.0, via Wikimedia Commons.



A real breadboard prototype with a small OLED screen, like Lab 6's. Photo: Turbospok, CC BY-SA 4.0, via Wikimedia Commons.

What a proof of concept must prove

A **proof of concept (PoC)** is a small experiment to show that an idea is worth pursuing. A good PoC answers two questions:

1. **Can it be done?** Does the technology work in the real conditions — does the radio reach, does the battery last, are the readings accurate?
2. **Is it worth doing?** Does it actually save the money, water, or time we hoped? Will people use it?

A PoC that works technically but saves nobody any money is still a failed PoC — and that's a *useful* result, found cheaply.

Tips for prototyping well

- **Build the smallest thing that tests your riskiest idea.** If you're not sure LoRaWAN will reach the far field, test *that* first, before designing a beautiful dashboard.
- **Use development boards and ready-made modules.** Don't design custom circuit boards until the idea is proven.
- **Test in the real environment early.** A sensor that works on your desk may fail in a hot, humid greenhouse.
- **Put it in front of real users early.** Their feedback will surprise you.
- **Keep notes** on what you tried and what you learned. It saves the next person from repeating your mistakes.

Challenges when deploying IoT in the real world

Moving from a working prototype to hundreds of devices in the field brings a whole new set of problems. Here are the ones that catch teams out most often.

Power

Battery life is often the hardest problem in IoT. Radios, sensors, and frequent messages all drain power. Real devices spend most of their time in **deep sleep**, waking briefly to take a reading and send it. Outdoor devices may add a small solar panel. Always measure battery life on the real hardware — estimates are often too optimistic.

Connectivity

The signal that was perfect during testing may be patchy in a basement, a metal warehouse, or a remote field. Good devices **store readings while offline and send them later**, and the system should notice when a device goes quiet.

Scale and management

Setting up 5 devices by hand is easy. Setting up 5,000 isn't. At scale, you need automatic **provisioning** (giving each device its identity and settings), a way to monitor device health, and a way to **update** thousands of devices safely (Chapter 7).

The physical environment

Real devices face heat, humidity, rain, dust, vibration, insects, and curious people. They need suitable **enclosures** — often rated with an "IP rating" (ingress protection, a code showing how well a case keeps out dust and water). In a tropical climate like Malaysia's, heat

and humidity are constant enemies of electronics.

Data quality

Sensors drift over time, get dirty, or fail. Plan for **calibration**, **validation** (Chapter 6), and ways to spot a faulty sensor before its bad data drives bad decisions.

Interoperability

Devices, gateways, and platforms from different makers must work together. Using **open standards** — like MQTT, LoRaWAN, and Matter — reduces the risk of being locked into one supplier.

Cost — the whole cost

The price of the device is only the start. The **total cost of ownership** includes connectivity fees, cloud and platform charges, installation, maintenance visits, battery replacements, and support — often over many years.

Security, privacy, and regulation

Everything in Chapter 7 applies, and gets harder at scale. There are also product rules. In many countries, including Malaysia, **radio equipment must be certified** before it can be sold, and products that collect personal data must follow data protection law, such as the PDPA.

People

Finally, IoT changes how people work. A system the staff don't trust, don't understand, or find annoying will be ignored or switched off. Involve the users early, train them, and make the system genuinely helpful to them.

> **NOTE**
>
> A helpful way to remember these: **power, signal, scale, weather, data, standards, cost, security, and people.** Almost every IoT deployment problem falls into one of them.

Common misconceptions

"If the prototype works, the product is almost finished." A working prototype is often less than half the journey. Enclosures, certification, security, manufacturing, deployment, and support take much more time and money than most people expect.

"You should design the perfect system before building anything." Real teams learn by building. Cheap, fast prototypes reveal problems that no amount of planning would find.

"The most advanced technology gives the best result." The best design is the simplest one that meets the requirements. A plain LoRaWAN sensor often beats a fancy 5G one for a farm.

"Once devices are installed, the project is done." Deployment is the *start* of a device's working life. Devices need monitoring, updates, repairs, and eventually safe retirement — often for years.

Key takeaways

- Good IoT design starts with **the problem and the people**, then turns them into **requirements**, then chooses each **layer** — always writing down **why**.
- Every good design choice **traces back to a requirement**.
- Real systems in **homes, industry, and agriculture** use the same five layers, with different choices driven by their environment.
- **Rapid prototyping** climbs a ladder: simulation → breadboard → proof of concept → pilot → production. Test the riskiest idea first, cheaply.
- A **proof of concept** must show an idea is both **possible** and **worth doing**.
- Deployment challenges: **power, connectivity, scale, environment, data quality, interoperability, total cost, security and regulation, and people**.

Check your understanding

1. Why should an IoT design start with the problem, rather than with a technology?

→ *Answer A2 is in the answer key at the end of this chapter. Try it yourself first!*

2. In the irrigation example, why was LoRaWAN chosen instead of Wi-Fi?

→ *Answer A3 is in the answer key at the end of this chapter. Try it yourself first!*

3. In industrial monitoring, why is vibration data usually analysed at the edge?

→ *Answer A4 is in the answer key at the end of this chapter. Try it yourself first!*

4. Put these prototyping stages in order: pilot, simulation, production, proof of concept, breadboard prototype.

→ *Answer A5 is in the answer key at the end of this chapter. Try it yourself first!*

5. A proof of concept shows the sensors work perfectly, but the farm saves almost no water. Was the PoC a success or a failure?

→ *Answer A6 is in the answer key at the end of this chapter. Try it yourself first!*

6. Name four challenges that often appear when moving from a prototype to hundreds of deployed devices.

→ *Answer A7 is in the answer key at the end of this chapter. Try it yourself first!*

Key terms

Term	Meaning
Breadboard	A reusable board for building circuits by pushing in parts and wires, with no soldering.
Development board	A ready-made circuit board with a microcontroller and handy connections, used for prototyping.
Enclosure	The protective case around a device.
Fertigation	Feeding plants by adding fertiliser to irrigation water.
Interoperability	The ability of devices and systems from different makers to work together.
IP rating	A code showing how well an enclosure keeps out dust and water.
Pilot	A small, real-world deployment to test a system before full rollout.
Proof of concept (PoC)	A small experiment to show an idea is technically possible and worth pursuing.
Provisioning	Setting up a new device with its identity, credentials, and settings.
Rapid prototyping	Testing ideas quickly and cheaply before investing in a full product.
Requirements	A clear list of what a system must do and the conditions it must work in.
Total cost of ownership	The full cost of a system over its life — not just the device price.

Into the lab: the capstone project

It's time to put everything together. Open **Lab 6 — Capstone: Smart Environment Station** (about 45 minutes).

You'll build a complete **Smart Environment Station**: the finished version of the smart room monitor we've followed throughout. It combines:

Part of the station	Where you learned it
DHT22 temperature and humidity readings	Chapter 3, Lab 2
An OLED screen showing live readings and status (over I²C)	Chapter 3
Wi-Fi and MQTT publishing, including a `status` topic	Chapter 4, Lab 3
An edge rule: above 28 °C, switch on the relay and buzzer	Chapter 6, Lab 5

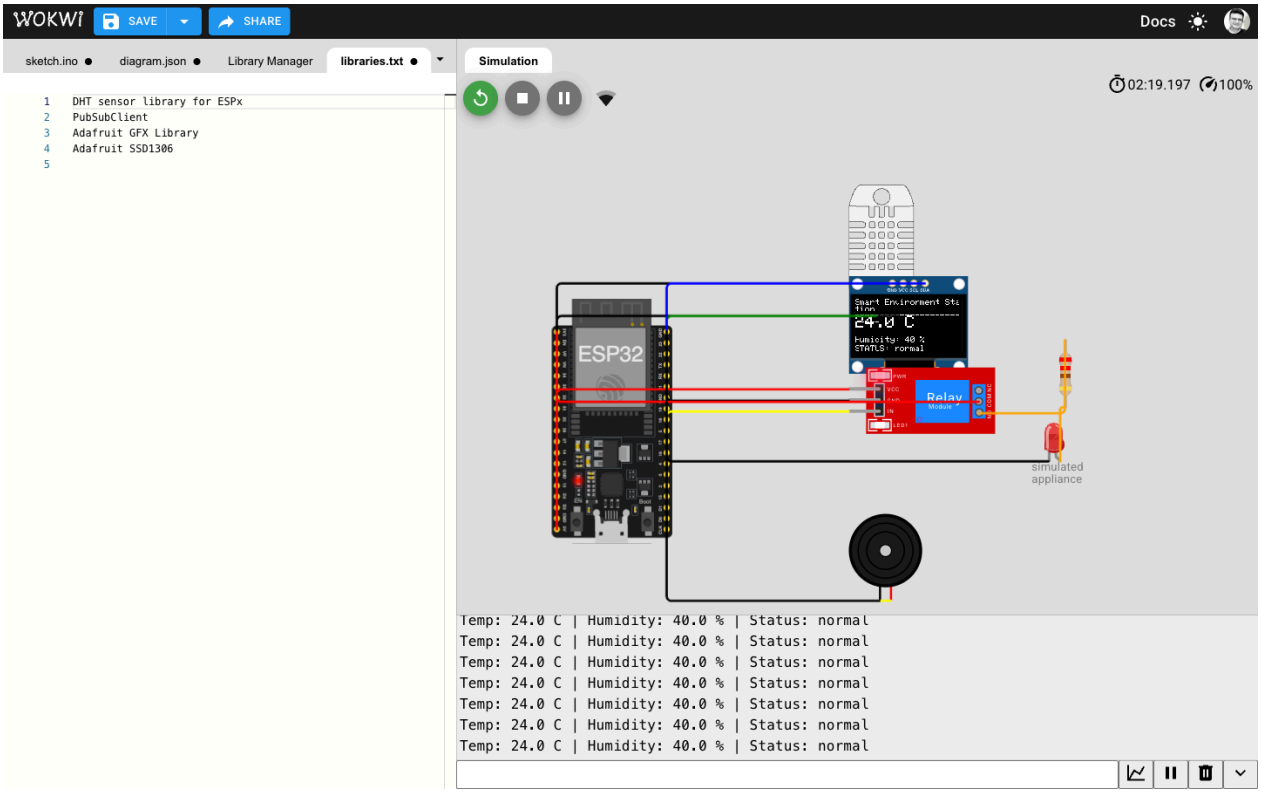


Figure 8.4 — the finished Smart Environment Station running in Wokwi, with the OLED showing live readings.

There's almost nothing new here — and that's the point. You're assembling pieces you already understand, which is exactly what building real IoT systems feels like.

Remember to **change** `YOUR_NAME` before pressing Play. This lab uses four libraries, so the first compile is the slowest yet — be patient, and press Play again if you see "Build Servers Busy".

Showcase: present your project in two minutes

When your station is running, show it to someone (or record a short video). A great two-minute showcase covers four things:

1. **The problem.** "This station protects a room from overheating."

2. **The layers.** Point to each one: the sensor and relay (**devices**), Wi-Fi and MQTT (**connectivity**), the 28 °C rule (**edge**), the public broker (standing in for a **cloud** platform), and the OLED and the HiveMQ client (**application**).
3. **The demo.** Push the temperature above 28 °C, and show everything react at once: the OLED shows `STATUS: ALARM!`, the relay and buzzer switch on, and an `ALARM` message arrives on your status topic.
4. **One improvement.** Using Chapter 7, name one thing you'd change before trusting it in a real home — for example, a private broker with a password and TLS encryption instead of a public, unencrypted one.

Wrapping up

Across these eight chapters, you've travelled the whole IoT stack: from the physics of a sensor, through Wi-Fi and MQTT, into the cloud, back out as automated action, and around again with security in mind. You've built — and run — a real IoT system from end to end.

Where to go next

You don't need to stop here. Here's a learning path for what comes next:

1. **Keep experimenting in Wokwi.** Your projects are saved in your free account. Try the challenge tasks at the end of each lab, like adding hysteresis to Lab 5.
2. **Try real hardware.** An ESP32 development board, a DHT22, a breadboard, and some jumper wires cost very little. The lab code runs on real hardware too — just put your own Wi-Fi name and password in place of `Wokwi-GUEST`.
3. **Secure your project.** Rebuild Lab 3 using an MQTT broker with a password and TLS encryption.
4. **Explore visual tools.** Try **Node-RED** for building data flows and rules, or **Home Assistant** for a smart home you control locally.
5. **Go deeper on one layer.** Pick the part you enjoyed most — electronics, networking, cloud, data, or security — and follow it further.

The further resources page has links for every one of these.

> **TIP**
>
> **One last question to take home:** what's one problem in your own home, workplace, or community that an IoT system could help solve? Try running it through this chapter's design method: problem, requirements, and one choice for each layer. That's how every real IoT project begins.

Going further (optional)

- Further resources — the full list of tools, tutorials, and communities.
- Random Nerd Tutorials — ESP32 projects — hundreds of real projects to try next.
- Node-RED — visual flows and rules for IoT.
- Home Assistant — a free, local-first smart home platform.
- The Things Network — a community LoRaWAN network and learning resources.

Answer key

A1 · Pause and check — In this design, why is the "open the valve" rule at the edge, while the "sensor has gone silent" rule is in the cloud?

Opening the valve must work **even if the internet is down**, so it runs locally at the edge. Noticing that a sensor has gone silent needs a view from **outside** the device — a silent sensor can't report its own silence — so the cloud watches for missing data.

A2 · Question 1 — Why should an IoT design start with the problem, rather than with a technology?

Because the right technology depends entirely on the problem's requirements — distance, power, data, cost, and who uses it. Starting with a technology risks building something impressive that doesn't solve the real problem, or costs too much.

A3 · Question 2 — In the irrigation example, why was LoRaWAN chosen instead of Wi-Fi?

The fields are up to 3 km away with no Wi-Fi and no mains power, and the sensors must run on batteries for at least a year while sending small, infrequent readings. LoRaWAN offers long range and very low power; Wi-Fi has short range and uses too much power.

A4 · Question 3 — In industrial monitoring, why is vibration data usually analysed at the edge?

Vibration sensors produce data very fast — far too much to send it all to the cloud cheaply. Edge gateways analyse it on site and send only summaries and warnings, which saves bandwidth and cost, and gives faster alerts.

A5 · Question 4 — Put these prototyping stages in order: pilot, simulation, production, proof of concept, breadboard prototype.

Simulation → breadboard prototype → proof of concept → pilot → production.

A6 · Question 5 — A proof of concept shows the sensors work perfectly, but the farm saves almost no water. Was the PoC a success or a failure?

It **failed to prove the idea is worth doing** — but it was still a valuable, cheap lesson. A PoC must show that an idea is both technically possible **and** worth the cost. Finding out early saves a much bigger failure later.

A7 · Question 6 — Name four challenges that often appear when moving from a prototype to hundreds of deployed devices.

Any four of: **power** and battery life; patchy **connectivity**; **scale** (provisioning, monitoring, updating); the physical **environment** (heat, humidity, dust, water); **data quality** and calibration; **interoperability**; **total cost** of ownership; **security, privacy, and regulation**; and **people** trusting and using the system.

The hands-on labs

Every lab runs free in your web browser in the Wokwi simulator. Each one has step-by-step instructions, screenshots, the complete code and circuit files, and troubleshooting notes. Scan a code with your phone, or type the address, to open a lab. All the labs live together at github.com/datatweets/intro-to-iot-workshop, where they are kept up to date as the tools change.

Scan	Lab	Goes with
	**Quickstart** — Open Wokwi and run your first simulation `github.com/datatweets/intro-to-iot-workshop/tree/main/1-getting-started`	Chapter 3
	**Lab 1** — Digital vs analog: an LED, a button and a dial `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/lab-1-digital-vs-analog`	Chapter 3
	**Lab 2** — Read a real sensor: DHT22 temperature and humidity `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/lab-2-temperature-and-humidity`	Chapter 3
	**Lab 3** — Get online: Wi-Fi and MQTT `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/lab-3-wifi-and-mqtt`	Chapter 4
	**Lab 4** — A live cloud dashboard with ThingSpeak `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/lab-4-cloud-dashboard`	Chapter 5
	**Lab 5** — Automation: a relay and buzzer that react on their own `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/lab-5-automation`	Chapter 6
	**Activity** — Spot the vulnerability (no coding) `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/activity-spot-the-vulnerability`	Chapter 7
	**Lab 6** — Capstone: the Smart Environment Station `github.com/datatweets/intro-to-iot-workshop/tree/main/2-labs/lab-6-smart-environment-station`	Chapter 8

Before Lab 1, create your free accounts at wokwi.com and thingspeak.com (ThingSpeak is needed from Lab 4).

Glossary

Every key term from the eight chapters, in one place, in alphabetical order. Each term links to the chapter where it's explained in full, so you can go back for the longer explanation, examples, and diagrams.

A

Term	Meaning	Learn more
Actuator	A part that turns an electrical signal into a physical action, like light, sound, or movement.	Chapter 2, Chapter 3
ADC (analog-to-digital converter)	A part of the microcontroller that turns an analog voltage into a number. On the ESP32, the number is 0–4095.	Chapter 3
Aggregate	To summarise many readings into one value, like an hourly average.	Chapter 6
Analog signal	A signal that can take any value within a range, like a dimmer switch.	Chapter 3
Analytics	Examining data to find useful information — descriptive, diagnostic, predictive, or prescriptive.	Chapter 5
API	Application programming interface — a doorway that lets programs talk to each other.	Chapter 4
API key	A secret code that gives permission to read or write data through an API.	Chapter 5
Application domain	An area where IoT is used, such as homes, cities, or agriculture.	Chapter 1
Application layer	The part of an IoT system people see and use — dashboards, apps, alerts.	Chapter 2
Attack surface	All the points where an attacker could try to get into a system.	Chapter 7
Authentication	Proving who (or what) you are, e.g. with a password or certificate.	Chapter 4, Chapter 7
Authorisation	Deciding what an authenticated user or device is allowed to do.	Chapter 7
Automation rule	An instruction of the form IF condition THEN action, run automatically.	Chapter 6

B

Term	Meaning	Learn more
Bandwidth (data rate)	How much data can be sent per second.	Chapter 4
BLE	Bluetooth Low Energy — a very-low-power, short-range radio.	Chapter 4
Botnet	A network of hijacked devices controlled by an attacker.	Chapter 7
Breadboard	A reusable board for building circuits by pushing in parts and wires, with no soldering.	Chapter 8
Broker	The central MQTT server that receives messages and delivers them to subscribers.	Chapter 4
Buzzer	An actuator that beeps or sounds a tone when it receives a signal.	Chapter 3

C

Term	Meaning	Learn more
Calibration	Checking a sensor against a trusted reference and adjusting it so its readings are accurate.	Chapter 3
Cellular IoT	IoT connectivity over mobile networks, such as NB-IoT, LTE-M and 5G RedCap.	Chapter 4
Channel (ThingSpeak)	A container for one project's data, holding up to 8 fields.	Chapter 5
CIA triad	The three goals of security: confidentiality, integrity, availability.	Chapter 7
Client / server	A client makes requests; a server listens and responds.	Chapter 4
Cloud automation	Rules that run on cloud servers, using data sent from devices.	Chapter 6
Cloud computing	Renting computing power and storage over the internet instead of owning it.	Chapter 1, Chapter 2, Chapter 5
Cloud platform (IoT platform)	A cloud service built to store, process, and act on device data.	Chapter 2, Chapter 5
CoAP	Constrained Application Protocol — a lightweight, UDP-based request–response protocol for tiny devices.	Chapter 4
Code signing	Adding a digital signature to software to prove who made it and that it hasn't been changed.	Chapter 7

Term	Meaning	Learn more
Connectivity	How data gets off a device and commands get back to it — the radio and the "language" it speaks.	Chapter 1, Chapter 2

D

Term	Meaning	Learn more
Dashboard	A screen showing the most important information at a glance.	Chapter 5
Data at rest / in transit	Data stored on a device or server (at rest), or travelling across a network (in transit).	Chapter 7
Data pipeline	The series of steps data passes through, from being sensed to being shown to a person.	Chapter 2
Data processing workflow	The series of steps that turns raw readings into useful information.	Chapter 6
Data retention	How long a platform keeps your data.	Chapter 5
Data visualisation	Presenting data as charts, gauges, maps, and other pictures.	Chapter 5
Datasheet	The technical document from a part's maker, listing its limits and how to use it.	Chapter 3
Deep sleep	A very-low-power mode in which a microcontroller switches almost everything off between jobs.	Chapter 2
Defence in depth	Using several layers of protection, so one failure isn't fatal.	Chapter 7
Development board	A ready-made circuit board with a microcontroller and handy connections, used for prototyping.	Chapter 8
Device management	Keeping track of devices, their credentials, status, and settings.	Chapter 5
Device shadow	A cloud copy of a device's latest state (AWS IoT).	Chapter 5
Digital certificate	An electronic "passport" that proves a device or server's identity.	Chapter 7
Digital signal	A signal with only two states: HIGH/LOW, 1/0, on/off.	Chapter 3
Digital signature	A mathematical seal that proves who made a file and that nobody has changed it.	Chapter 7
DNS	Domain Name System — turns names like `broker.hivemq.com` into IP addresses.	Chapter 4

E

Term	Meaning	Learn more
Ecosystem	The network of companies and people who together make an IoT product work.	Chapter 2
Edge automation	Rules that run on the device or a nearby gateway, without needing the cloud.	Chapter 6
Edge computing	Processing data and making decisions close to the device, instead of in a distant cloud.	Chapter 2
Enclosure	The protective case around a device.	Chapter 8
Encryption	Scrambling data so only someone with the right key can read it.	Chapter 4, Chapter 7
ESP32	A low-cost microcontroller from Espressif with Wi-Fi and Bluetooth built in. Used in all of the labs.	Chapter 2
Event	A meaningful change, such as a value crossing a limit or a door opening.	Chapter 6
Event-driven architecture	A design where parts of a system react to events as they happen.	Chapter 6

F

Term	Meaning	Learn more
Fertigation	Feeding plants by adding fertiliser to irrigation water.	Chapter 8
Field (ThingSpeak)	One value within a channel, such as temperature.	Chapter 5
Firmware	The software that runs on a device.	Chapter 7
Floating input	An input pin that isn't connected to anything, so it gives random readings.	Chapter 3

G

Term	Meaning	Learn more
Gateway	A middleman device that connects nearby devices to the internet.	Chapter 2

H

Term	Meaning	Learn more
HTTP / HTTPS	The request–response protocol of the web. HTTPS is the encrypted version.	Chapter 4
Hysteresis	Using two thresholds (an “on” point and an “off” point) to stop a rule flickering.	Chapter 6

I

Term	Meaning	Learn more
I²C	A two-wire protocol that lets many devices share the same connection. Used by the OLED display in Lab 6.	Chapter 3
Industrial IoT (IIoT)	IoT used in factories, energy, mining, and other industries.	Chapter 1
Ingestion	Receiving and accepting data from devices into a platform.	Chapter 5
Interfacing	Connecting a sensor or actuator to a microcontroller so they work together correctly.	Chapter 3
Internet of Things (IoT)	The network of physical objects with sensors, software, and connectivity that collect, share, and act on data.	Chapter 1
Interoperability	The ability of devices and systems from different makers to work together.	Chapter 8
IoT loop	The four steps every IoT system follows: sense, send, decide, act.	Chapter 1
IP address	A device’s address on a network.	Chapter 4
IP rating	A code showing how well an enclosure keeps out dust and water.	Chapter 8

L

Term	Meaning	Learn more
LAN	Local area network — a network in one place, such as your home Wi-Fi.	Chapter 4
Last Will and Testament	An MQTT message a device leaves with the broker, which the broker sends for it if the device drops off unexpectedly.	Chapter 4
Latency	How long a message takes to arrive.	Chapter 4
Least privilege	Giving each user or device only the permissions it truly needs.	Chapter 7
LED	Light-emitting diode — a small, efficient light, often used as a status indicator.	Chapter 3
Library	Ready-made code that handles a complicated job, like talking to a particular sensor.	Chapter 3
Lock-in	Becoming so dependent on one provider that switching is hard.	Chapter 5
LoRaWAN	A long-range, very-low-power wide-area network for small, infrequent messages.	Chapter 4

M

Term	Meaning	Learn more
Machine learning	Software that learns patterns from data to make predictions.	Chapter 5
Matter	A smart-home standard that lets devices from different brands work together.	Chapter 1
Mesh network	A network where devices pass messages on for each other to extend coverage.	Chapter 4
Microcontroller (MCU)	A complete, tiny, low-power computer on one chip that runs one program.	Chapter 2
Microprocessor (MPU)	A more powerful processor that needs extra chips and usually runs a full operating system.	Chapter 2
Motor driver	A helper circuit that lets a microcontroller control a motor safely.	Chapter 3
MQTT	A lightweight publish–subscribe messaging protocol widely used in IoT.	Chapter 4
Multi-factor authentication	Logging in with two or more kinds of proof, such as a password plus a phone code.	Chapter 7
Mutual authentication	Both sides of a connection proving their identity to each other.	Chapter 7

N

Term	Meaning	Learn more
NB-IoT / LTE-M	Low-power versions of mobile (cellular) networks, designed for IoT.	Chapter 4
Noise	Small, random wobbles in sensor readings.	Chapter 3

O

Term	Meaning	Learn more
OTA update	An over-the-air update, delivered to a device over the network.	Chapter 7
OWASP IoT Top 10	A list of the ten most common IoT security weaknesses.	Chapter 7

P

Term	Meaning	Learn more
Packet	A small chunk of data sent across a network.	Chapter 4
Patch	A software update that fixes a bug or security weakness.	Chapter 7
Patch management	Keeping track of every device and its software version, and rolling out updates safely.	Chapter 7
Photoresistor (LDR)	A light sensor whose resistance drops as more light falls on it.	Chapter 3
Pilot	A small, real-world deployment to test a system before full rollout.	Chapter 8
PIR sensor	A passive infrared motion sensor that detects warm bodies moving.	Chapter 3
Polling	Checking a value again and again on a fixed schedule.	Chapter 6
Port	A number that identifies a particular service on a server.	Chapter 4
Potentiometer	A knob or dial whose position gives an analog signal.	Chapter 3
Precision agriculture	Using sensors and data to give crops exactly the water and nutrients they need.	Chapter 1
Predictive maintenance	Using sensor data to fix machines before they break down.	Chapter 1
Proof of concept (PoC)	A small experiment to show an idea is technically possible and worth pursuing.	Chapter 8
Protocol	An agreed set of rules for how messages are formatted and exchanged.	Chapter 4

Term	Meaning	Learn more
Provisioning	Setting up a new device with its identity, credentials, and settings.	Chapter 8
Publish / subscribe	A messaging pattern where senders publish to topics, and receivers subscribe to topics.	Chapter 4
Pull-up resistor	A resistor that holds an input pin HIGH until something pulls it LOW.	Chapter 3
PWM (pulse-width modulation)	Switching a pin on and off very fast to imitate an in-between value, e.g. to dim an LED.	Chapter 3

Q

Term	Meaning	Learn more
QoS	Quality of Service — MQTT's setting for how hard to try to deliver a message (0, 1, or 2).	Chapter 4

R

Term	Meaning	Learn more
Rapid prototyping	Testing ideas quickly and cheaply before investing in a full product.	Chapter 8
Raspberry Pi	A popular small computer board built around a microprocessor; it runs a full operating system.	Chapter 2
Relative humidity	How much water vapour the air holds, as a percentage of the most it could hold.	Chapter 3
Relay	An electrically controlled switch that lets a small signal switch a bigger circuit.	Chapter 3
Requirements	A clear list of what a system must do and the conditions it must work in.	Chapter 8
Resistor	A part that limits the flow of electricity, e.g. to protect an LED.	Chapter 3
REST	A popular style of building web APIs using HTTP methods like GET and POST.	Chapter 4
Retained message	The last message on an MQTT topic, kept by the broker and given to new subscribers straight away.	Chapter 4
RFID	Radio-frequency identification — small tags that let objects be identified by radio.	Chapter 1

Term	Meaning	Learn more
Router	A device that passes data between networks, such as your home network and the internet.	Chapter 4
Rules engine	Software that stores and runs many automation rules.	Chapter 6

S

Term	Meaning	Learn more
Secure boot	A device checking its own firmware is genuine every time it starts.	Chapter 7
Secure by design	Building security in from the start, rather than adding it later.	Chapter 7
Sensor	A part that measures something in the physical world and turns it into an electrical signal.	Chapter 2, Chapter 3
Serial Monitor	A panel that shows text messages sent from the device with `Serial.print()`.	Chapter 3
Servo motor	A motor that turns to an exact angle.	Chapter 3
Sketch	An Arduino program. It always has a `setup()` and a `loop()`.	Chapter 3
Smart meter	An electricity (or water) meter that measures use and reports it automatically.	Chapter 1
SPI	A fast, four-wire protocol used by displays, SD cards and similar parts.	Chapter 3
Stakeholder	Any person or organisation with an interest in how an IoT system turns out.	Chapter 2
Status code	The number a server sends back to say how a request went, such as 200 (success) or 404 (not found).	Chapter 4

T

Term	Meaning	Learn more
TCP / UDP	Rules for sending packets: TCP is reliable and ordered; UDP is lighter with no guarantees.	Chapter 4
Thermistor	A part whose resistance changes with temperature, used as a simple temperature sensor.	Chapter 3
Threshold	The limit a value is compared against in a rule, like 28 °C.	Chapter 6
Time-series data	A sequence of values, each recorded with a timestamp.	Chapter 5
Time-series database	A database built to store huge numbers of timestamped readings and answer time-based questions quickly.	Chapter 5
TLS	Transport Layer Security — the standard way to encrypt internet connections.	Chapter 4, Chapter 7
Topic	The label on an MQTT message, organised in levels like `home/kitchen/temperature`.	Chapter 4
Total cost of ownership	The full cost of a system over its life — not just the device price.	Chapter 8

U

Term	Meaning	Learn more
UART	A simple two-wire serial link, one wire each way. The Serial Monitor uses it.	Chapter 3

V

Term	Meaning	Learn more
Validation	Checking that a reading is realistic, and discarding it if not.	Chapter 6

W

Term	Meaning	Learn more
Wildcard	A symbol in an MQTT subscription that matches several topics: + for exactly one level, # for everything below.	Chapter 4
Wokwi	A free online simulator that runs a real ESP32 and its circuit in your web browser.	Chapter 3

Z

Term	Meaning	Learn more
Zigbee / Thread	Low-power mesh radio technologies used in smart homes and buildings.	Chapter 4

Image credits

Diagrams

Every diagram in this book was drawn for it. They use one shared visual language, so the same colour always means the same thing:

Colour	Meaning
Blue	Data moving up, from devices towards people
Orange	Commands and actions, such as switching something on
Aqua	Work done on or near the device (the edge)
Violet	The cloud
Green ✓ / red ✕	OK / problem, always shown with an icon and words, never colour alone

Photographs

All photographs in this book come from Wikimedia Commons and are free to reuse, including commercially, under the licences listed below. Some have been resized for faster loading; no other changes were made. Photos under a Creative Commons BY or BY-SA licence must keep this attribution if you reuse them.

Shows	Author	Licence	Source
Arduino board beside a breadboard	TreyDanger	CC BY 2.0	Commons page
Rows of server racks in a data centre	BalticServers.com	CC BY-SA 3.0	Commons page
Technicians working at a data-centre rack (CMC Telecom, Vietnam)	Daoducquan	CC BY-SA 4.0	Commons page
DHT22 (AM2302) sensor	Ubahnverleih	CC0	Commons page
ESP32 development board	Edwiyanto	CC BY-SA 4.0	Commons page
Soil-moisture sensor in a field	Christianjhart1	CC0	Commons page

Shows	Author	Licence	Source
Kevin Ashton in 2015	Larry D. Moore	CC BY 4.0	Commons page
LoRa radio module	Cnd	CC0	Commons page
LoRaWAN gateway antenna on a mast	Fabian Horst	CC BY 4.0	Commons page
Breadboard prototype with a small OLED display	Turbospok	CC BY-SA 4.0	Commons page
PIR motion sensor module	Sjilla	CC BY-SA 4.0	Commons page
Rotary potentiometer	Iainf	CC BY 2.5	Commons page
Raspberry Pi 4 Model B	Laserlicht	CC BY-SA 4.0	Commons page
5 V single-channel relay module	Suyash Dwivedi	CC BY-SA 4.0	Commons page
SG90 micro servo motor	Suyash Dwivedi	CC BY-SA 4.0	Commons page
Smart electricity meter	277volts	CC BY-SA 4.0	Commons page
Smart streetlight pilot, Hannover	Ra Boe / Wikipedia	CC BY-SA 3.0 DE	Commons page
Smart thermostat	Raysonho @ Open Grid Scheduler / Grid Engine	CC0	Commons page
Telecommunication towers in Kuala Terengganu, Malaysia	pslim	CC BY 2.0	Commons page
Wireless vibration sensor on an industrial motor	Cedric Leturcq	CC BY-SA 4.0	Commons page

Product and brand names visible in photos (for example, Nest, Raspberry Pi, Arduino) are trademarks of their owners. Their appearance here is only to illustrate a type of device, and does not imply any endorsement.

Understand the Internet of Things — and build it yourself.

From smart homes to smart farms, billions of everyday objects now sense the world, share what they sense, and act on it. This book explains how — clearly, step by step, and without assuming any background in electronics, networking or programming.

You'll learn how to:

- explain what IoT is, how an IoT system is built, and where it is used;
- connect and read sensors, and control actuators such as relays and buzzers;
- compare Wi-Fi, Bluetooth, LoRaWAN and cellular IoT, and send data with MQTT and HTTP;
- build a live cloud dashboard for your own sensor data;
- make a device act on its own with automation rules at the edge;
- recognise common IoT security threats — and the defences that stop them;
- design a complete IoT system, from idea to prototype.

Six hands-on labs run free in your web browser — no hardware to buy, nothing to install.

Hands-on labs and code
github.com/datatweets/intro-to-iot-workshop

Mehdi Lotfinejad